

Methodology and Git Strategy

08/06/2026 16:57:12

lecture_two.pdf Methodology and original documentation: help us form our methodology (git and project) base...

Thinking 🌐 Searching the web docs.gitea.com ● Designing project workflow I'm shaping a formal, lightweight...
ok wait, u went too fast and did tooo much and Im lost. we need to ensure we answer all the qs on the last slid...

Thinking ● Planning the document I'll create one concise Git methodology document for your team, following t...
Ok, go for it and make Git Methodology. we want to link branches/commits/pulls to issues i think?

Thinking ● Designing Git methodology I'm preparing a standalone Git Methodology section covering repository...
1. mak it formal for submission 2. i feel like tis a bit vague?

Thinking ● Structuring the methodology I'm tightening the submission by replacing broad intentions with...
now give it to me in markdown as a submission ready doc (no ref lecture 2 etc) also, add that we will not chan...

Thinking ● Evaluating methodology fit I'm treating this as a lightweight academic project, where consistency a...
why did u stop markdown?? give entire thing in markdown, or a .md to download?

You're right. Here is the complete Markdown file: Download the submission-ready Git Methodology

ok, now do same for project methodology, first on the chat so we can discuss and then a .md

Thinking ● Structuring the workflow I'm organizing the project brief into Basic, Intermediate, and Advanced tie...
standups once a week? meet stakeholders on tuesday, standups on thursday? or something like that? do we...

Thinking ● Designing a lightweight cadence I'm recommending a weekly Thursday stand-up, with Tuesday...
ok, give me the .md doc

Download the submission-ready Project Methodology

Pasted markdown.md do we need the sprint reviews and retrospectives etc??

Thinking ● Clarifying sprint practices I'm recommending a lightweight approach: use the existing Tuesday...
can u give me the whole .md then (keeping my changes)

Download the revised Project Methodology I kept your changes, replaced the separate sprint review and...
ok, now hoqw to push these?? the repo is completley empty, and want to follow all the methodology we spoke...

Thinking 🌐 Searching the web docs.gitea.comdictionary.cambridge.orgpullapart.combyn.skincdc.govgithub.co...
377aeea9-5cf1-4c08-81e6-0ad278215992.png how to create the project board columns?

Thinking 🌐 Searching the web docs.gitea.com ● Searched 1 website ✓ Worked for 18s You are currently on th...
3e67ad77-a6da-4cc0-9c81-ad1ca8668982.png uhm, now im out of sprint 1?

Yes, you are now on the Projects page, which is separate from the Sprint 1 milestone page. You have not delete...
160113d3-0538-48ba-aa2a-c86153b64041.png repo has this, ive added the board.. now give ,me instructions...

Thinking 🌐 Searching the web docs.gitea.com ● Planning the next steps I'm starting with issue creation...
in step 4, dont i need to say AI?

Thinking ● Clarifying AI attribution I'm interpreting the policy conservatively: if AI helped draft the issue, add a...
dbce711e-9657-4b28-a191-874c6b20e2f5.png step 8, wheres repo settings?

Thinking 🌐 Searching the web docs.gitea.com ● Searched 1 website ✓ Worked for 19s The Settings tab is...
cffe7f1f-962f-4fd2-a85e-20e92ca80b51.png are these namings etc correct?

Almost correct. One filename does not match the link in your README. Correct docs/git-methodology.md This...
Purpose Add the approved Git Methodology and Project Methodology documents to the repository. ##...

Thinking ● Clarifying disclosure needs I'm distinguishing documentation disclosures from the pull-request...
can u assign "any 1 reviewer" or do u need to chose a specific person?

Thinking 🌐 Searching the web docs.gitea.com ● Searched 1 website ✓ Worked for 14s You can configure the...
but i cant do that if im not admin right?

Thinking 🌐 Searching the web docs.gitea.com ● Searched 1 website ✓ Worked for 7s Correct. Without Admin...

bed4715d-1ec9-416a-9da3-40f59628ece4.png uhmm, why does it say 0 commits?

Thinking ● Diagnosing the pull request The pull request shows no commits because your branch likely has no... ohh, someone merged it before i had the change to write it.. s how should i close this then?

Thinking ● Handling the duplicate pull request I'm inferring the documentation is already merged elsewhere. I'... oyyy, the git methodology was empty, how to follow rules and reupload that doc?

Do not edit main directly and do not reopen the old empty Pull Request. Because the earlier change was ahead... ok, now dean made this layout, can u access from the link for gitea or not? b4 i ask my q...

Thinking 🌐 Searching the web ● Searching the web ✓ Worked for 9s I tried the link, but I cannot access the... Sport-Analytics-Tool.zip 1. are there clashes with the things we pushed and this? 2. using spruint 1 rubric etc,...

Thinking 🌐 Remembering ● Reviewed repository conventions and aligned Sprint 1 setup issues ● Reviewing... we havent discussed actual project flow yet, so prefer not that, rather things like this? we can start OAuth Azur...

Thinking 🌐 Inspecting Uploaded Repository Files ● Aligning repository governance I'm drafting one cleanup... we havent discussed actual project flow yet, so prefer not that, rather things like this? we can start OAuth Azur...

Thinking 🌐 Inspecting sport analytics repository files ● Inspected sport analytics repository files, templates,... ok, its issue #7, ive made the issue, now give me step by stepp to make it happen. note: im not changing...

Thinking 🌐 Inspecting repository for issue implementation ● Planning repository instructions I'm preparing a... wait, part 5, give me the whole markdown to paste, itgot split up before

Replace the entire contents of: .gitea/ISSUE_TEMPLATE/feature.md with this: --- name: Feature about: Impleme... i meant part 4 pls

Replace the entire contents of: evidence/decisions/ADR-001-initial-repository-setup.md with this: # ADR-001:... again. from alternatives considered isnt markdown, should it be>?

Yes, it should all be Markdown. The previous block broke because the document itself contained a triple-backtic... pls regive me from part 8-end, ensure markdowns are all correct!

Thinking ● Continuing the workflow I'm providing Parts 8 through the end, keeping Markdown fencing valid an... ok wait, itll be 1 ai chat for everything from methodology to today, sho yesturday and today. how should i go...

Thinking ● Clarifying register entries I'm checking whether one continuous chat can cover multiple work... in part 12 commits, dont i need Ai declarations??

Thinking ● Explained AI attribution requirements for all Part 12 commits ✓ Worked for 20s Yes - for these... uhm?

Thinking ● Clarified AI attribution requirements ✓ Worked for 14s e0dca32e-50d6-4bf5-9d0c-e3fc39e76aad.png uhm?

Thinking ● Clarifying commit declarations You're right to question that. I'm revising my earlier advice: don't ad... i want to undo the commit! how to get out of where i am?

You have not committed yet. You are inside Vim editing the commit message. To cancel the commit and exit:... now give me commits that will work with Ai declaration too

Thinking ● Preparing commit commands I'm giving three PowerShell one-liners in dependency order. Each... a3f9a95a-ab22-4cf0-b1c4-8fd065b59884.png what does this mean?

Thinking ● Interpreting the workflow status I'm reading the clock icons as queued runs: the workflow was... so the merge will be succesfull regardless? then i have admin access so we can do protecti nvng main

Thinking 🌐 Searching the web docs.gitea.com ● Clarifying merge protection Yes, the merge is already in `mai... ok, so now give me recomendatin of 6 issues we should create and work on now (myabe ci one can be one?)

Thinking 🌐 Inspecting repository structure and existing files ● Planning sprint issues I'm mapping six... do we need a gitwiki if we using a docs file? how do we display it from the repo

Thinking 🌐 Searching the web docs.gitea.com ● Searched 1 website ● Clarifying documentation flow I'm... ok, so give me complete issues for the 6 things and how to make them etc and then each teammember will...

Thinking 🌐 Inspecting Repository Structure for Issues ● Organizing issue set I'm preparing six copy-paste-read... for each are they features, bgs, tasks etc?

Use the Task template for all six. None of them is a product feature yet, and none is strictly a bug report. Issue... is Ci a clever one for me to take?

Yes, CI is a clever issue to take, but it is not the easiest one. It is valuable because it: directly supports the Spri... k, im gonna do CI, are you ready to help me?

Yes — ready. We'll do it slowly, one step at a time, and verify each command before moving on. First: Assign th... Sport-Analytics-Tool (2).zip cant find ci?

Thinking 🌐 Locating CI Workflow Files ● Inspected CI workflow files and configuration ✓ Worked for 18s The CI... issue #10

Thinking ● Starting issue #10 I'm treating issue #10 as the next task. First, assign it, move it to ****In Progress***...

PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> Get-Content .gitea/workflows/ci.yml name:...

Good. We have confirmed two things: CI currently uses npm install. There is no package-lock.json, so npm ci...

PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> node --version v24.14.0 PS...

Thinking ● Aligning local setup I'm seeing two separate issues: PowerShell is blocking `npm.ps1`, and your loc...

PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> npm.cmd install npm warn deprecated inflight@1.0....

Thinking 🌐 Inspecting repository files and formatting configuration ● Assessing formatting changes I'm...

PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> git ls-files --eol package.json .gitea/workflows/ci.yml...

Thinking ● Diagnosing attributes I'm checking whether `.gitattributes` actually exists on the current remote...

PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> git fetch origin remote: Enumerating objects: 1, don...

Thinking ● Updating the branch I'm confirming the branch is behind `origin/main`, not ahead. I'd first preserve...

PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> git ls-files --eol package.json .gitea/workflows/ci.yml...

Thinking ● Fixing line endings I'm narrowing this to Git's checkout behavior: the attributes are correct, but...

PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> git config core.autocrlf false PS...

Thinking ● Troubleshooting line endings I'm checking whether Git's attributes are being applied during checko...

PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> git status --short >> npm.cmd run format:check M...

Thinking ● Recovering repository state I'm seeing three tracked files still differ after the reset attempt. I'd avoi...

PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> npx.cmd prettier --check . --end-of-line auto Checki...

Thinking ● Reviewing formatting fixes I'm focusing on the 13 files that CI flagged, avoiding broad rewrites. I'll...

PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> git status --short M .gitea/ISSUE_TEMPLATE/bug.md ...

Thinking ● Fixing remaining files The earlier formatting command likely didn't modify those six documents, sin...

PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> npm.cmd run check > sport-analytics-tool@0.1.0...

Great — formatting now passes. CI has reached a genuine lint error, which is exactly what we needed to uncov...

PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> Get-Content apps/backend/src/middleware/error-...

Thinking ● Fixing lint errors I'm checking the failure rather than changing architecture. The smallest fix is...

sport-analytics-tool@0.1.0 C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool |─┐ @typescript-eslint/eslint-...

Excellent. The lint error is fixed, and the TypeScript compatibility warning is gone. Next step: run the complete...

PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> npm.cmd run check > sport-analytics-tool@0.1.0...

Only .eslintrc.cjs remains unformatted. Run: npx.cmd prettier --write .eslintrc.cjs Then verify formatting:...

PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> npm.cmd install --save-dev --save-exact...

Excellent progress: structure, formatting, linting, type checking and all tests now pass. Only the frontend...

PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> Get-Content apps/frontend/src/api/client.ts export...

Make the smallest type-safe change in apps/frontend/src/api/client.ts. Replace: signal, with: signal: signal ?? nu...

PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> npm.cmd run build --workspace=@sport-...

Excellent — the entire repository check now passes: structure check; formatting; linting; type checking; backen...

PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> git status --short M .eslintrc.cjs M...

npm.cmd ci succeeded, so the new lockfile supports a clean, reproducible installation. The vulnerability and...

PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> git status --short M .eslintrc.cjs M...

That is correct. npm.cmd ci did not create any unexpected changes; the repository still contains only the fixes ...

PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> Get-Content .gitea/workflows/ci.yml name:...

Thinking ● Reviewing local changes I'm treating the checks as successful while noting the dependency warnin...

PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> git diff --check warning: in the working copy of...

Thinking ● Reviewing the diff I'm seeing no whitespace errors; the line-ending warnings are expected...

PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> git branch --show-current ci/10-restore-green-ci PS...

Thinking ● Organizing commits The combined command isn't a problem; it just means the results run together...

next

Commit 2: Reproducible dependencies First confirm the remaining changes: git status --short You should see: M...

next

Commit 3: Allow intentionally unused Express parameters Stage only the ESLint configuration: git add...

next

Commit 4: Fix the frontend production build Stage only the API client change: git add...

next

Commit 5: Use reproducible installation in CI Stage the workflow change: git add .gitea/workflows/ci.yml Revie...

next do we need to update documentation b4 we push?

Thinking ● Reviewing documentation needs Yes, I'm checking this before the push. Because the changes affect...

PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> Get-Content docs/development/setup.md # Local...

Thinking ● Updating documentation Yes—I'm updating three docs before pushing: `setup.md` to require `npm...

Pasted markdown(1).md correct?? also, this ai chat is the same one i used for the previous stuff, so what shoul...

Thinking ● Reviewing the AI entry I'm checking the existing row against the repository evidence. It is correct: t...

AI usage This repository makes use of AI code generation using the following tools: ChatGPT-Web[GPT-5.6...

Yes — the README AI section is correct as written: ## AI usage This repository makes use of AI code generatio...

Date,Team member,Tool,Model,Purpose,Brief task,Output used,Verification or adaptation,Related evidence 202...

Thinking ● Checking CSV quality I'm seeing a solid register entry. I'd remove the blank line between records an...

k, lets commit and push? are we ready?

Thinking ● Finalizing changes Yes, I'm ready to commit and push, provided the working tree contains only the...

PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> git push -u origin ci/10-restore-green-ci Enumeratin...

Great — the branch pushed successfully and is tracking the remote branch. Open the Pull Request link shown i...

caa4dfa3-2e41-4bb9-9b6a-907313c4f019.png

This strongly indicates a runner problem, not a workflow-code problem: the new run is Waiting duration is still ...

5b3426b6-f826-439c-ab4a-15b7cc815a0a.png

Thinking ● Assessing runner availability I'm reading the screenshot as showing no runner currently registered f...

the admin is the lecturer, so write it d=formaly enough for me

Subject: Gitea Actions Runner Required for CI Workflow Dear [Lecturer's Name], I hope you are well. Our team...

dont say issue #10 bc he doesnt know what that is

Subject: Gitea Actions Runner Required for CI Workflow Dear [Lecturer's Name], I hope you are well. Our team...

can i make it that we merge this commit, but dont close the issue?

Thinking 🌐 Searching the web docs.gitea.com ● Searched 1 website ✓ Worked for 7s Yes. Edit the PR...

Subject: Gitea Actions Runner Required for CI Workflow Dear [Lecturer's Name], I hope you are well. Our...

Subject: Gitea Actions Runner Hi [Lecturer's Name], We have set up our CI workflow and tested it locally...

no, ask as if he hasnt seen our repo at all,

Subject: Gitea Actions Runner Hi [Lecturer's Name], We are setting up automated checks for our project... the person doing azure from the 6 issues from b4 wants to do frontend at the same time, make me a frontend... Yes, it makes sense to create a separate frontend Azure issue. The same person may work on both, but keepin... labels?

Use: Task Frontend Infrastructure Azure Deployment Sprint 1 Priority: Medium, unless the backend deployment... noo, like type and area

Use: Type: Task Area: Frontend Because the issue is specifically for deploying the frontend. The Azure and... wait, do it properly, look back to the other one and do it like that!!

Thinking 🌐 Remembering ● Reviewed Azure issues, labels, and Sport Analytics template ✓ Worked for 18s... and the issue decr etc? do thag again

Thinking ● Used personal_context tool ✓ Worked for 22s Issue title Establish Azure frontend deployment... ok, now we couldn figure out how to 'aprove' merges so i changed therule to 0, is that an issue and what do i...

Thinking ● Clarifying review controls I'm distinguishing enforced settings from actual review evidence. Keeping... 96454461-bc87-40d2-b3b6-c6bab44b7c3c.png

That's helpful. It confirms the runner problem is known and the lecturer is arranging one, but it may take a wee... ok, we have our first standup today, what do we need to do/say?

Thinking ● Preparing standup guidance I'm treating this as a practical first standup: align on yesterday, today,... what sprt is bets for us to use

Thinking ● Choosing a suitable sport I'm comparing cricket, baseball, tennis, basketball, and soccer by event... can clouflare link to gitea? and how?

Thinking 🌐 Searching the web developers.cloudflare.commerriam-webster.comapple.comgit-... can you make an issue for me for doing that and connecting to wrangler etc

Thinking 🌐 Searching the web docs.gitea.comdevelopers.cloudflare.com ● Clarifying Cloudflare scope I'm... give me the issue to copy paste, make sure ur markdown works well make sure to add the AI decleration, etc.

Thinking ● Structuring deployment issue I'm drafting a self-contained issue with milestones, acceptance criteri... now make an issue to create foilders for Ai transcripts for each person

Thinking ● Drafting repository issue I'm shaping a documentation issue for organizing AI transcripts while...



lecture_two.pdf

Methodology and original documentation:

help us form our methodology (git and project) based on sprint 1 rubric and the notes here I've added from class

We were thinking to use Gitea projects (we using Gittea not Github) and to reference commits like that or something?

note: he said as long as we justify our choices, and follow them, its ok. so chose effective but easy and lightweight

also add sprint standups etc?

8/4/2026, 3:51:23 PM



Thinking



Searching the web

docs.gitea.com

- **Designing project workflow**

I'm shaping a formal, lightweight Scrumban process with GitHub Flow adapted to Gitea. I'll cover sprint cadence, board states, issue and pull-request rules, branching, commits, reviews, releases, documentation, evidence, and AI-use declarations.

- **Designed project workflow**



Searching the web

- **Structuring the methodology**

I'm tailoring the plan to the sports-analytics project: lightweight Scrumban, fixed sprints, brief standups, weekly client checks, reviews, retrospectives, and Gitea Flow with traceable issues, commits, pull requests, tests, documentation, and original evidence.

- **Structured the methodology**

- **Searched 1 website**

✓ Worked for 2m 15s

Recommended approach

Use two related but separate methodologies:

Project methodology: a lightweight **Scrumban** process

Git methodology: **GitHub Flow adapted for Gitea**

This is a good fit because your deadlines are organised into fixed sprints, but the work itself will change as you clarify requirements with the client. Scrumban gives you sprint planning and reviews without requiring full Scrum administration. GitHub Flow gives you short-lived branches and reviewed pull requests without the extra `develop`, `release`, and `hotfix` branches used by Git Flow.

The lecture requires you to define when you commit, how commits are written, when branches are created and merged, merge requirements, and versioning. It also stresses that consistency is more important than choosing an elaborate system.

[lecture_two.pdf](#)

Yes, include standups. To keep them lightweight, I would use **three short standups per week rather than daily meetings**, with daily standups only near a milestone.

Copy-ready methodology document

You could place this in:



1. Methodology overview

The team will use a lightweight Scrumban methodology supported by Gitea Projects, Issues, Milestones and Pull Requests. Gitea will act as the team's single source of truth for planned work, current progress, decisions and completed work.

Scrumban was selected because the project has fixed sprint deadlines but its detailed requirements will continue to evolve through stakeholder discussions, technical investigation and testing. The methodology combines sprint-based planning and review with a visual Kanban workflow. This provides enough structure to coordinate the team and produce evidence of progress without introducing unnecessary administrative work.

The Git workflow will follow GitHub Flow principles adapted for Gitea. Each change will be connected to a Gitea issue, developed on a short-lived branch, reviewed through a pull request and merged into a protected `main` branch. This was selected instead of Git Flow because the project is being completed by a small team over a short semester and does not require separate long-running `develop` and `release` branches.

The methodology will be reviewed during each sprint retrospective. Any change to the methodology must be agreed upon by the team, recorded in the retrospective notes and updated in this document. Once changed, the new process must be applied consistently.

2. Gitea work tracking

2.1 Milestones

One Gitea milestone will be created for each formal project milestone:

```
Sprint 1
Sprint 2
Sprint 3
Final Submission
```

These correspond with the project deadlines of 25 August, 15 September, 29 September and 11 October 2026.

`project_briefs.pdf`

Every issue that is intended for a sprint must be assigned to the relevant milestone.

2.2 Project board

The Gitea Project board will use the following columns:

```
Backlog
Ready
In Progress
In Review
Blocked
Done
```

Backlog: Work that has been identified but not yet selected for a sprint.

Ready: Work that has sufficient information and acceptance criteria and can be started.

In Progress: Work currently being completed. Each team member should normally have no more than one main development issue in progress.

In Review: A pull request has been opened and the work is awaiting review, testing or changes.

Blocked: Work cannot continue because of a technical, dependency or stakeholder-related blocker.

Done: The issue meets the team's Definition of Done and has been merged.

2.3 Issue requirements

Each significant unit of work must have a Gitea issue before development begins. Very small corrections may be added to an existing relevant issue.

Each issue should contain:

```
## Description
What needs to be completed?

## Motivation
Why is this work required?

## Acceptance criteria
- [ ] Observable result one
- [ ] Observable result two
- [ ] Relevant tests pass
- [ ] Documentation is updated where necessary

## Dependencies
List related or blocking issues.

## Evidence
Links to designs, stakeholder notes, pull requests or test results.
```

Each issue must also have:

- an assignee;
- a sprint milestone;
- an appropriate type and area label;
- acceptance criteria;
- a priority where necessary.

2.4 Labels

Keep the labels limited and useful:

```
type: feature
type: bug
type: documentation
type: testing
type: research

area: api
area: data
area: frontend
area: infrastructure

priority: high
status: blocked
```

Do not create labels that the group will not actually use.

3. Sprint process

3.1 Sprint planning

At the beginning of each sprint, the team will hold a planning meeting of approximately 45 minutes.

During planning, the team will:

1. review the milestone requirements;
2. review stakeholder feedback;
3. select issues from the backlog;
4. confirm acceptance criteria;
5. assign initial owners;
6. identify dependencies and risks;
7. agree on the sprint goal.

The result of planning must be visible through the Gitea milestone and Project board. A brief planning note will be stored under:

```
docs/meetings/YYYY-MM-DD-sprint-planning.md
```

3.2 Standups

The team will hold a ten-minute standup on Monday, Wednesday and Friday.

Each member will answer:

```
What have I completed since the previous standup?  
What will I work on next?  
What is blocking me?
```

Standups are intended to identify problems and coordinate work, not to conduct technical discussions. Longer discussions will take place after the standup with only the relevant members.

To reduce documentation overhead, individual standup minutes are not required. Instead, one short weekly summary will record:

- important progress;
- blockers;
- changes to responsibilities;
- decisions made;
- actions for the following week.

During the final week before a milestone, the team may temporarily hold daily standups.

3.3 Stakeholder interaction

The team will meet or communicate with the assigned client at least once per week where possible.

Stakeholder interaction will be used to:

- clarify requirements;
- demonstrate current work;
- confirm whether acceptance criteria reflect the client's expectations;
- discuss proposed changes;
- prioritise upcoming work.

Important stakeholder feedback will be documented in:

```
docs/meetings/YYYY-MM-DD-client-meeting.md
```

Each note should contain:

```
# Client meeting

**Date:**
**Attendees:**
**Purpose:**

## Requirements discussed

## Feedback received

## Decisions

## Actions

## Related Gitea issues
- #...
```

Feedback that changes the product must result in a new issue or an update to an existing issue. Requirements must not be changed silently.

This provides evidence for the Sprint 1 stakeholder-interaction criterion, which expects requirements to be discussed and identified rather than merely listed. `project_briefs.pdf`

3.4 Sprint review

At the end of each sprint, the team will demonstrate completed work to the stakeholder or tutor.

Only work in the `Done` column will be presented as completed. Incomplete work will be returned to the backlog or moved to the next milestone with a recorded explanation.

The review must record:

- features demonstrated;
- stakeholder feedback;
- accepted work;
- rejected or incomplete work;
- new requirements;
- scope changes.

3.5 Sprint retrospective

After the review, the team will hold a 20-minute retrospective addressing:

```
What worked well?
What caused difficulty?
What should we continue doing?
What should we change during the next sprint?
```

Actions from the retrospective must be specific and assigned to a team member. Changes to the methodology must be recorded in this document.

4. Definition of Ready

An issue may move into `Ready` when:

- its purpose is clear;

- it has acceptance criteria;
- important dependencies are known;
- required stakeholder clarification has been obtained;
- the work is small enough to complete within the sprint;
- the team understands how completion will be verified.

Research or investigation issues may use a defined output, such as a recommendation, prototype or decision record, instead of software acceptance criteria.

5. Definition of Done

An issue may move to `Done` only when:

- the acceptance criteria have been met;
- the code has been committed and pushed;
- a pull request has been reviewed;
- required automated checks pass;
- relevant tests have been added or updated;
- documentation has been updated;
- no unresolved review comments remain;
- the change has been merged into `main`;
- the issue has been closed.

For functionality that affects the user interface, the work should also be checked for responsiveness and accessibility.

Git methodology

6. Repository rules

The same Git methodology will apply to every repository used by the project.

The `main` branch represents the latest integrated and working version of the project. Direct pushes to `main` are not permitted. Gitea branch protection can enforce pull-request merging, approvals and successful status checks. ([Gitea Documentation](#))

Force pushes to `main` are prohibited.

Every repository must contain:

```
README.md
.gitignore
licence information where applicable
setup and development instructions
AI usage declaration
```

Where applicable, repositories should also include formatter, linter and type-checking configuration. This helps address the advanced Sprint 1 version-control requirement for both repository structure and code-quality tools. `project_briefs.pdf`

7. Branching strategy

A new branch will be created for each issue that requires code or substantial documentation changes.

Branch names will use the following format:

```
<type>/<issue-number>-<short-description>
```

Examples:

```
feat/27-event-schema-validation
fix/41-duplicate-event-counting
docs/18-local-development-guide
test/35-submission-api-tests
chore/12-ci-pipeline
```

Allowed branch types are:

```
feat
fix
docs
test
refactor
chore
```

Branches should be short-lived and should focus on one issue. A branch should normally be merged within several days rather than remaining open for an entire sprint.

A new branch should be created from the latest version of `main`:

```
git switch main
git pull
git switch -c feat/27-event-schema-validation
```

8. Commit methodology

Commits must be atomic. Each commit should represent one meaningful and independently understandable change.

The team will use Conventional Commit-style messages:

```
<type>(<scope>): <description>
```

Examples:

```
feat(api): validate submitted event structure
fix(stats): prevent duplicate events from affecting totals
docs(setup): add database migration instructions
test(api): add invalid submission cases
refactor(data): extract statistic calculation service
chore(ci): run API tests on pull requests
```

Commit descriptions should:

- use the imperative form;
- explain the result of the change;
- avoid vague wording such as `updated stuff`;
- remain focused on one unit of work.

8.1 Linking commits to Gitea issues

Use the issue number in the commit body:

```
feat(api): validate submitted event structure

Refs #27
```

The final pull request should close the issue:

```
Closes #27
```

Gitea automatically links references such as `#27` to issues and can close an issue through wording such as `Closes #27` when the pull request is merged. Cross-repository issues can be referenced as `owner/repository#27`. Commit hashes of at least seven characters can also be linked, but issue and pull-request references should be the primary traceability mechanism. ([Gitea Documentation](#))

Therefore, your original idea is correct, but I would use:

```
Issue → Branch → Commits → Pull Request → Merge
```

rather than relying mainly on raw commit hashes.

8.2 AI-assisted commits

Where AI generated code used in a commit, the commit body must include the tool and model:

```
feat(api): validate submitted event structure
```

```
Refs #27
```

```
Assisted-by: <tool>[<model>]
```

Example:

```
Assisted-by: ChatGPT-Web[GPT-5.6 Thinking]
```

Only include tools that were genuinely used for that commit. The course policy also requires the repository README to declare AI code generation, inline editing and code review usage or non-usage. [ai_policy_20_jul.pdf](#)

9. Pull-request process

Every change to `main` must be submitted through a Gitea pull request.

Gitea supports repository-based issue and pull-request templates, including `.gitea/PULL_REQUEST_TEMPLATE.md`, which can automatically provide the required review checklist. ([Gitea Documentation](#))

Use the following template:

```
## Purpose
```

```
Explain why this change is needed.
```

```
## Changes
```

- Change one
- Change two

```
## Related issue
```

```
Closes #...
```

```
## Verification
```

```
Describe how the change was tested.
```

```
## Checklist
```

- [] Acceptance criteria have been met

- [] I reviewed my own changes
- [] Tests have been added or updated where appropriate
- [] All automated checks pass
- [] Documentation has been updated
- [] No unrelated changes are included
- [] AI assistance has been declared where required

9.1 Merge requirements

A pull request may be merged only when:

- it relates to a Gitea issue;
- its purpose and verification steps are documented;
- at least one team member other than the author has reviewed it;
- requested changes have been addressed;
- automated checks pass;
- the branch is not in conflict with `main`;
- relevant documentation and tests are included;
- the acceptance criteria are satisfied.

A pull request that is not ready should begin with:

WIP:

The team will use merge commits consistently so that the branch history, individual commits and pull-request boundary remain visible. The branch will be deleted after merging.

10. Versioning

The project will use descriptive milestone tags because the main externally relevant versions during development correspond to the course milestones.

Tags will be:

```
sprint-1  
sprint-2  
sprint-3  
submission
```

A tag will be created only after the sprint review and after the accepted milestone version has been merged into `main`.

Example:

```
git switch main  
git pull  
git tag -a sprint-1 -m "Sprint 1 milestone submission"  
git push origin sprint-1
```

Each milestone release note should include:

- the sprint goal;
- completed features;
- known limitations;
- linked issues and pull requests;

- setup or deployment information.

This follows the lecture's example of descriptive versions such as `sprint-2` and provides a stable point that can be revisited during marking. `lecture_two.pdf`

Original documentation structure

I recommend keeping the original documentation small but meaningful:

```
README.md

docs/
├─ methodology.md
├─ requirements.md
├─ development-plan.md
├─ architecture.md
├─ tech-stack.md
├─ testing.md
├─ meetings/
│   ├─ 2026-08-05-sprint-planning.md
│   ├─ 2026-08-08-client-meeting.md
│   └─ week-01-summary.md
├─ decisions/
│   ├─ 001-project-methodology.md
│   ├─ 002-git-workflow.md
│   └─ 003-event-storage-model.md
└─ designs/
    ├─ system-context.png
    ├─ database-erd.png
    ├─ api-outline.md
    └─ wireframes/
```

Do not document every informal conversation. Document information that proves:

- what decision was made;
- who was involved;
- why it was selected;
- what alternatives were considered;
- what changed because of the decision.

For example:

```
# Decision 002: Git workflow

## Decision

Use GitHub Flow adapted to Gitea.

## Motivation

The team has fixed sprint deadlines and needs visible peer review, but the
project is too small and short-lived to justify Git Flow's additional
develop and release branches.

## Alternatives considered

- Git Flow
- Trunk-based development
```

- Direct commits to main

Consequences

- Every task requires an issue and short-lived branch.
- Main remains protected.
- Every merge is reviewed through a pull request.
- There is a small increase in process, but substantially better traceability and review evidence.

How this meets the Sprint 1 rubric

Sprint 1 criterion	Evidence you should produce
Version Control	Organised repositories, protected <code>main</code> , formatter/linter configuration, atomic commits from every member
Documentation Site	Deployed documentation with methodology, architecture, requirements and development content
Getting Started / Dev Guides	A working README in every repository with setup, test and run instructions
Work Tracker	Gitea Project board, milestones, assigned issues and visible movement between columns
Git Methodology	This document plus evidence in branch names, commits, pull requests, reviews and tags
Project Methodology	Sprint planning, standups, reviews, retrospectives and board history
Tech Stack	<code>tech-stack.md</code> listing each technology, purpose, alternatives and motivation
Stakeholder Interaction	Client meeting notes, clarified requirements, feedback issues and resulting changes
Initial Design and Dev Plan	Architecture diagram, ERD, API outline, UI wireframes and sprint roadmap
Implementation	Working project scaffolds, CI/CD, database setup and early supporting features

The advanced rubric is not awarded simply because a methodology exists. It expects the Git methodology to be **used** and the project methodology to be followed with evidence and motivation. `project_briefs.pdf`

Suggested Sprint 1 methodology-related issues

Create these in Gitea immediately:

```
Set up Gitea Project and sprint milestones
Protect main branches
Add issue and pull-request templates
Document Git methodology
Document project methodology
Create README and getting-started guide for each repository
Configure formatter and linter
Create documentation-site scaffold
Deploy documentation site
Document and motivate technology stack
Hold initial stakeholder requirements meeting
```

```
Create initial requirements catalogue
Create system architecture diagram
Create database ERD
Create initial API design
Create UI wireframes
Create semester development roadmap
Configure initial CI pipeline
```

For the Sport Analytics Tool, early implementation issues could include:

```
Define the sport event schema
Create the initial fixture and event data model
Prototype event-submission validation
Create basic fixture and event API endpoints
Create a statistic-derivation proof of concept
Record event-to-submission traceability
Create representative sample event data
```

These are directly connected to the project's central requirement that statistics be derived from ordered event data, submissions be validated, and published information remain traceable to its source. `sport_analytics.pdf`

Declaration to add to this document

Because this methodology was prepared with AI assistance, add:

The preceding document was planned and generated with the assistance of ChatGPT-Web[GPT-5.6 Thinking].

Change the wording to "reviewed", "edited", "planned" or "generated" according to how your group ultimately uses and rewrites it.

Sources

[Protected branches | Gitea Documentation](#) docs.gitea.com

8/4/2026, 3:51:26 PM



ok wait, u went too fast and did tooo much and Im lost.
we need to ensure we answer all the qs on the last slide of the lecture in a document, so include those, and explain to me what u will do before u do it

8/4/2026, 4:01:31 PM



Thinking

- **Planning the document**

I'll create one concise Git methodology document for your team, following the eight questions on the final slide exactly: commits, messages, branches, naming, merging, merge requirements, version timing, and versioning rules. `[]filecite[]turn0file0[]`

✓ Worked for 10s

You're right - I moved too quickly.

The **first document should only deal with the Git Methodology questions from the last lecture slide:**

1. When to commit
2. How to write commits
3. When to branch
4. How to name branches
5. When to merge
6. What requirements must be met before merging
7. When to create a version
8. How versions will be named

These are the exact areas the lecture says a Git methodology must define. `lecture_two.pdf`

What I will do

I will create one short, clear document titled:

Git Methodology

For each question, the document will have three parts:

Our decision

What your group has chosen to do.

How it works

A simple explanation of the rule.

Why we chose it

A brief justification showing that the choice is lightweight, practical and suitable for your group.

For example:

When to commit

Our decision:

We will commit whenever one small, complete unit of work has been finished.

How it works:

A commit should contain one logical change, such as adding validation to an endpoint or fixing one bug.

Why we chose it:

Small commits are easier to understand, review and reverse if something goes wrong.

I will use **Gitea terminology**, not GitHub terminology. Therefore, the document can refer to:

- Gitea Issues
- Gitea Projects
- branches
- commits
- Pull Requests
- tags

I will keep the methodology simple. I am likely to recommend:

- atomic commits;
- conventional-style commit messages;
- one short-lived branch per Gitea issue;
- Pull Requests before merging into `main`;
- one reviewer before merging;
- descriptive tags such as `sprint-1`, `sprint-2` and `final-submission`.

I will not yet add sprint planning, standups, retrospectives or the full project methodology. Those belong in a separate **Project Methodology** section. We can do that after the Git Methodology is clear and agreed upon.

The final Git Methodology document should be approximately one to two pages, not the long document I produced previously.

8/4/2026, 4:01:43 PM



Ok, go for it and make Git Methodology.
we want to link branches/commits/pulls to issues i think?

8/4/2026, 4:03:11 PM



Thinking

- **Designing Git methodology**

I'm preparing a standalone Git Methodology section covering repository roles, branch protection, commit conventions, issue linkage, pull requests, reviews, merging, releases, and evidence needed for assessment.

- **Designed a Git methodology**

✓ Worked for 16s

Git Methodology

1. Purpose

This Git methodology defines how our team will use Git and Gitea throughout the Sport Analytics project. Its purpose is to keep development organised, make each change traceable, support code review, and ensure that all team members follow the same process.

We have chosen a lightweight branch and Pull Request workflow suitable for a small student team. The methodology answers each of the questions listed on the final slide of Lecture 2. [lecture_two.pdf](#)

2. When will we commit?

We will commit whenever a small, complete and meaningful unit of work has been completed.

A commit should represent one logical change, such as:

- adding one API endpoint;
- fixing one bug;
- adding validation;
- updating one section of documentation;
- adding tests for one feature.

We will avoid commits that contain several unrelated changes.

We chose this approach because small, atomic commits are easier to understand, review, test and reverse if a problem is introduced.

3. How will we write commit messages?

We will use a simplified Conventional Commit format:

```
<type>: <short description>
```

The main commit types will be:

```
feat:    new functionality
fix:     bug fix
docs:    documentation changes
test:    adding or changing tests
```

```
refactor:  restructuring code without changing behaviour
chore:     setup, configuration or maintenance work
```

Examples:

```
feat: add fixture event submission endpoint
fix:  prevent duplicate events from being counted
docs: add local database setup instructions
test: add tests for invalid event submissions
```

Commit descriptions should clearly explain what changed. We will avoid unclear messages such as:

```
updated files
changes
fixed stuff
work done
```

Where useful, a commit may include more detail in its body.

4. When will we create a branch?

A branch will be created when a team member begins work on a Gitea issue.

Each branch should normally focus on one issue or one clearly defined unit of work. This makes it easier to understand what the branch contains and prevents unrelated changes from being mixed together.

Branches will be created from the latest version of `main`.

The `main` branch will contain the most recent reviewed and working version of the project. Team members will not develop directly on `main`.

5. How will we name branches?

Branch names will include:

1. the type of work;
2. the related Gitea issue number;
3. a short description.

The format will be:

```
<type>/<issue-number>-<short-description>
```

Examples:

```
feature/12-event-submission
fix/18-duplicate-statistics
docs/21-api-documentation
test/24-event-validation-tests
chore/7-ci-setup
```

This naming system makes it immediately clear what the branch is for and which Gitea issue it relates to.

6. How will branches, commits and Pull Requests link to issues?

Every significant task will first be recorded as a Gitea issue.

The issue will act as the starting point for the work and will contain the task description, acceptance criteria and assigned team member.

The issue number will then be included throughout the development process.

Branch

```
feature/12-event-submission
```

Commit

```
feat: validate submitted event data
```

```
Issue: #12
```

Pull Request

```
#12 Add event submission validation
```

The Pull Request description will also state:

```
Related issue: #12
```

Once the Pull Request has been reviewed, accepted and merged, the related issue can be closed.

This gives us a clear traceability chain:

```
Gitea issue → branch → commits → Pull Request → merge
```

We chose this system because it makes it possible to trace a completed feature back to its original requirement and development history.

7. When will we merge?

A branch will be merged when:

- the issue's acceptance criteria have been completed;
- the work is ready for review;
- a Pull Request has been opened;
- the required review has been completed;
- any requested changes have been addressed;
- relevant automated checks pass;
- there are no unresolved merge conflicts.

Branches will be merged into `main` through a Gitea Pull Request.

The branch will be deleted after it has been successfully merged, unless there is a clear reason to keep it.

8. What are the requirements for merging?

Before a Pull Request may be merged:

1. It must be linked to a Gitea issue.
2. It must clearly explain what was changed.
3. The author must review their own changes first.
4. At least one other team member must review the Pull Request.
5. All review comments must be resolved.
6. Relevant tests must pass.

7. Automated checks must pass where available.
8. Documentation must be updated where necessary.
9. The Pull Request must not contain unrelated changes.
10. The code must not contain unresolved merge conflicts.

A Pull Request should answer the questions highlighted in the lecture:

- What does the code do?
- Does it actually do that?
- Has someone else reviewed it?
- Does it pass automated testing?
- Where is it being merged? `lecture_two.pdf`

9. When will we create a version?

A version will be created when the code reaches an important, stable project milestone.

For this project, versions will be created after:

- Sprint 1;
- Sprint 2;
- Sprint 3;
- the final project submission.

A version will only be created once the milestone work has been reviewed and merged into `main`.

We will use Git tags to identify these versions so that the exact state of the project at each milestone can be revisited later.

10. How will we name versions?

We will use descriptive version names linked to the project milestones:

```
sprint-1  
sprint-2  
sprint-3  
final-submission
```

If a corrected version of a milestone must be created, a revision number may be added:

```
sprint-1.1  
sprint-1.2
```

We selected descriptive versioning rather than semantic versioning because the most important releases during this project are the formal sprint submissions. The lecture identifies sprint-based names as a valid descriptive versioning approach.

`lecture_two.pdf`

11. Summary of the workflow

1. Create a Gitea issue.
2. Define the issue's acceptance criteria.
3. Create a branch linked to the issue number.
4. Make small, atomic commits.
5. Reference the issue in commits where appropriate.
6. Push the branch to Gitea.
7. Open a Pull Request linked to the issue.

8. Have another team member review it.
9. Resolve feedback and ensure checks pass.
10. Merge the Pull Request into main.
11. Close the issue.
12. Delete the completed branch.
13. Tag main when a sprint milestone is completed.

The most important principle is that the group must follow this methodology consistently throughout the project.

AI Declaration

The preceding document was planned and generated with the assistance of ChatGPT-Web[GPT-5.6 Thinking].

The course AI policy requires documents created with AI assistance to identify the tool, model and nature of the assistance.

ai_policy_20_jul.pdf

8/4/2026, 4:03:26 PM



1. mak it formal for submission
2. i feel like tis a bit vague?

8/4/2026, 4:05:58 PM



Thinking

- **Structuring the methodology**

I'm tightening the submission by replacing broad intentions with measurable rules. I'll specify commit timing, message format, branch naming, pull-request requirements, issue references, review expectations, and the AI-use declaration.

- **Structured the methodology**

✓ Worked for 34s

You are correct. The previous version explained the general idea, but several rules were not specific enough to prove that the methodology is being followed.

I will strengthen it by making every rule measurable. For example, instead of saying "commit regularly," the methodology will state exactly when a commit should be made, what format must be used, how it links to an issue, who reviews a Pull Request, what must pass before merging, and which merge method the team will use.

Below is a more formal, submission-ready version.

Git Methodology

1. Purpose

The team will use **GitHub Flow, adapted for use on Gitea**, as its Git methodology.

GitHub Flow is a lightweight branching methodology in which development takes place on short-lived branches created from `main`. Changes are reviewed through Pull Requests before being merged back into `main`.

This methodology was selected because the project is being developed by a small team over a limited academic period. A more complex methodology, such as Git Flow, would require additional permanent branches and release processes that are not necessary for the scale of this project. The selected methodology provides sufficient structure for collaboration, review and traceability while remaining simple enough to apply consistently.

The `main` branch will represent the latest reviewed, tested and working version of the project. Direct development on `main` will not be permitted.

This document addresses the eight areas identified in the Git Methodology section of Lecture 2: committing, commit messages, branching, branch naming, merging, merge requirements, version creation and version naming. `lecture_two.pdf`

2. When to Commit

A commit will be created whenever a developer completes one small, logically complete unit of work.

A commit must:

- represent one identifiable change;
- relate to the purpose of the branch and its Gitea issue;
- leave the affected part of the project in a working state;
- not knowingly introduce failing tests or compilation errors;
- exclude unrelated file changes.

Examples of suitable individual commits include:

- creating a database model for fixtures;
- adding validation for submitted event data;
- correcting a duplicate-statistics calculation;
- adding tests for one API endpoint;
- updating the setup instructions for one service.

A developer must not wait until an entire feature or sprint is complete before committing. Similarly, a commit must not combine unrelated work such as database changes, user-interface styling and documentation updates unless those changes form one inseparable unit of work.

This follows the atomic commit approach discussed in the lecture, in which each commit represents an independent unit of work. `lecture_two.pdf`

Temporary or incomplete work may be committed locally while development is in progress. However, before a Pull Request is submitted for review, vague or unnecessary work-in-progress commits must be cleaned up so that the final commit history remains understandable.

3. How to Write Commits

The team will use **Conventional Commit-style messages** with an optional scope.

The required format is:

```
<type>(<scope>): <description>
```

The scope may be omitted when it does not add useful information:

```
<type>: <description>
```

3.1 Approved commit types

Type	Purpose
<code>feat</code>	Adds new functionality
<code>fix</code>	Corrects incorrect behaviour
<code>docs</code>	Changes documentation only
<code>test</code>	Adds or changes tests
<code>refactor</code>	Restructures code without changing behaviour
<code>chore</code>	Changes configuration, tooling or maintenance files
<code>ci</code>	Changes the continuous integration or deployment process

3.2 Commit-message rules

The description must:

- begin with a lowercase verb;
- describe what the commit accomplishes;
- be specific enough to understand without opening the changed files;
- avoid vague terms such as `changes`, `updates`, `work` or `stuff`.

Acceptable examples include:

```
feat(api): add fixture event submission endpoint
fix(stats): prevent duplicate events from affecting totals
test(api): add invalid event submission cases
docs(setup): document local database configuration
ci: run backend tests on pull requests
```

Unacceptable examples include:

```
updated code
fixed things
more work
final changes
stuff
```

3.3 Linking commits to Gitea issues

Every commit must relate to the Gitea issue assigned to the branch.

The issue reference will be added to the commit body:

```
feat(api): validate submitted event data

Refs #12
```

Where one commit resolves a small issue completely, the commit body may state:

```
Closes #12
```

However, the normal team practice will be to close the issue through the Pull Request rather than through an individual commit. This prevents an issue from being closed before the entire branch has been reviewed and merged.

Where AI-generated code is included in a commit, the commit must also contain the required attribution:

```
feat(api): validate submitted event data

Refs #12
Assisted-by: ChatGPT-Web[GPT-5.6 Thinking]
```

The course AI policy requires code-generation assistance to be identified in the relevant commit and broader AI use to be declared in the repository README. [ai_policy_20_jul.pdf](#)

4. When to Branch

A new branch will be created when work begins on a Gitea issue that requires code, testing, configuration or substantial documentation changes.

Before creating the branch, the developer must ensure that:

1. a Gitea issue exists;
2. the issue has a clear description;
3. the issue contains acceptance criteria;
4. the issue has been assigned to a team member;
5. the issue has been allocated to the relevant sprint milestone.

The branch must be created from the latest version of `main`.

For example:

```
git switch main
git pull origin main
git switch -c feat/12-event-submission
```

A branch should normally address one issue only. Where two issues are inseparable, both issue references must be recorded in the branch's Pull Request.

Branches must remain short-lived. The intended duration of a normal branch is no longer than one week. If a branch remains open for longer, the team must review whether the issue should be divided into smaller tasks.

This reduces the likelihood of large merge conflicts and ensures that work is integrated regularly.

5. How to Name Branches

Branches will use the following naming convention:

```
<type>/<issue-number>-<short-description>
```

The branch type must match the main type of work being completed.

Examples include:

```
feat/12-event-submission
fix/18-duplicate-statistics
docs/21-api-documentation
test/24-event-validation
refactor/31-statistics-service
chore/7-project-configuration
ci/9-backend-pipeline
```

Branch names must:

- use lowercase letters;
- separate words with hyphens;
- include the related Gitea issue number;
- use a short but meaningful description;
- contain no spaces.

The issue number creates a direct relationship between the development branch and the work recorded in Gitea.

6. Linking Issues, Branches, Commits and Pull Requests

Every significant change will follow a consistent traceability chain:

```
Gitea issue → branch → commits → Pull Request → merge
```

For Gitea issue **#12**, the links would be recorded as follows.

Gitea issue

```
#12 Validate submitted event data
```

The issue contains the requirement, acceptance criteria, assignee and sprint milestone.

Branch

```
feat/12-event-validation
```

Commit

```
feat(api): validate submitted event structure
```

```
Refs #12
```

Pull Request title

```
#12 Validate submitted event data
```

Pull Request description

```
Closes #12
```

The Pull Request will also link any relevant commits, tests, documentation and supporting design decisions.

This structure ensures that a completed feature can be traced back to:

- the original requirement;
- the developer responsible for the work;
- the individual changes made;
- the review discussion;
- the tests used to verify the work;
- the final merge into `main`.

Traceability is particularly important for the Sport Analytics Tool because submitted events, derived statistics and corrections must remain traceable to their source. `sport_analytics.pdf`

7. When to Open a Pull Request

A Pull Request will be opened when the branch is ready for review.

Before opening the Pull Request, the author must:

1. update the branch with the latest changes from `main`;
2. resolve any merge conflicts;
3. review their own changes;
4. run the relevant tests locally;
5. remove debugging code and temporary files;
6. confirm that the issue's acceptance criteria have been addressed;
7. update relevant documentation;
8. confirm that AI assistance has been declared where required.

A draft or work-in-progress Pull Request may be opened earlier when the developer requires feedback before the feature is complete. It must be clearly labelled as a draft and may not be merged.

8. When to Merge

A branch will be merged only after its Pull Request has completed the required review process.

The team will use a **merge commit** when merging Pull Requests into `main`.

This merge method was selected because it preserves:

- the individual atomic commits;
- the identity of the feature branch;
- the Pull Request as a distinct unit of work;
- the relationship between the issue and the completed branch.

The merge commit will therefore provide a clear boundary showing when the work associated with a specific issue entered `main`.

After a Pull Request has been merged:

1. the related Gitea issue must be closed;
2. the branch must be deleted from Gitea;
3. the Gitea Project board item must be moved to `Done`;
4. any resulting follow-up work must be recorded as a new issue.

Developers may not merge their own Pull Requests without another team member's approval, except where the group has recorded a justified emergency exception.

9. Requirements for Merging

A Pull Request may be merged only when all of the following requirements have been met.

9.1 Issue and scope

- The Pull Request is linked to at least one Gitea issue.
- The work satisfies the issue's acceptance criteria.
- The Pull Request does not contain unrelated changes.
- Any change in scope has been recorded in the issue.

9.2 Description

The Pull Request description must explain:

- the purpose of the change;
- the main changes made;
- how the change was tested;
- any known limitations;
- which issue it closes.

9.3 Review

- The author has reviewed their own work.
- At least one team member other than the author has approved the Pull Request.
- All reviewer comments have been answered.
- All requested changes have been completed or formally resolved.
- The reviewer has checked whether the implementation matches the stated purpose.

9.4 Testing and quality

- Relevant automated tests pass.
- Existing tests continue to pass.
- Formatting and linting checks pass where configured.
- The application or affected service builds successfully.
- The change does not introduce unresolved merge conflicts.
- No sensitive information, credentials or unnecessary generated files have been committed.

9.5 Documentation

- Relevant technical documentation has been updated.
- Setup instructions have been updated where the development process changed.
- API documentation has been updated where an endpoint or schema changed.
- AI attribution has been included where required.

These requirements correspond with the Pull Request questions identified in the lecture: what the code does, whether it performs that function, whether it has been reviewed, whether automated tests pass and where it will be merged.

[lecture_two.pdf](#)

10. When to Create a Version

A version will be created when the team reaches a formal assessed milestone and the submitted code has been reviewed and stabilised.

Versions will be created after:

- Sprint 1;
- Sprint 2;
- Sprint 3;
- the final submission.

A version may only be created when:

1. all work intended for the milestone has been merged into `main`;
2. the milestone version builds successfully;
3. required automated checks pass;
4. the team has reviewed the milestone state;
5. known limitations have been documented;
6. the exact submitted commit has been confirmed.

The version will be created using an annotated Git tag on `main`.

The project brief identifies Sprint 1, Sprint 2, Sprint 3 and the final submission as the project's formal milestones.

[project_briefs.pdf](#)

11. How to Name Versions

The project will use **descriptive milestone versioning** during the academic development period.

The primary tags will be:

```
sprint-1
sprint-2
```

```
sprint-3  
final-submission
```

If a permitted correction is made after a milestone tag has been created, the correction will receive a numbered revision:

```
sprint-1.1  
sprint-1.2
```

An existing tag must not be moved or overwritten. A new tag must be created so that the previous submitted state remains available.

Descriptive milestone versioning was selected because the most significant versions of the project correspond directly with the assessed sprint milestones. It is therefore clearer for the team and assessors than applying semantic version numbers during the early academic development period.

The lecture identifies names such as `sprint-2` as a valid descriptive versioning method and emphasises that the chosen method must be applied consistently. `lecture_two.pdf`

12. Standard Development Workflow

All significant project work will follow the process below:

1. A Gitea issue is created.
2. The issue receives acceptance criteria, an assignee and a sprint milestone.
3. A branch is created from the latest version of `main`.
4. The branch name includes the issue number.
5. Development is completed through small, atomic commits.
6. Commits reference the related issue.
7. The developer tests and reviews their own work.
8. A Pull Request is opened and linked to the issue.
9. Another team member reviews the Pull Request.
10. Automated checks and relevant tests must pass.
11. Review feedback is addressed.
12. The Pull Request is approved and merged using a merge commit.
13. The issue is closed and the branch is deleted.
14. The milestone version is tagged when the sprint is complete.

13. Methodology Compliance

All team members are required to follow this methodology.

Compliance will be demonstrated through evidence in Gitea, including:

- issues with acceptance criteria;
- consistently named branches;
- issue references in commits;
- linked Pull Requests;
- review approvals;
- successful automated checks;
- milestone tags;
- contributions from all team members.

The methodology will be reviewed at the end of each sprint. Any change to the process must be agreed upon by the team, justified in writing and applied consistently from that point onward.

This is important because the Sprint 1 rubric distinguishes between merely documenting a Git methodology and actually demonstrating its use. `project_briefs.pdf`

AI Declaration

The preceding document was planned and generated with the assistance of ChatGPT-Web[GPT-5.6 Thinking].

8/4/2026, 4:06:26 PM



now give it to me in markdown as a submission ready doc (no ref lecture 2 etc)
also, add that we will not change methodology unless it is causing issues etc, also,
is this the best but easiest methodology to follow? make it submission ready

8/4/2026, 4:15:18 PM



Thinking

- **Evaluating methodology fit**

I'm treating this as a lightweight academic project, where consistency and evidence matter more than adopting a complex framework. A small adapted GitHub Flow is likely the best balance, since it stays simple while supporting documented issue work, branches, reviews, and releases. [...]

- **Evaluated methodology fit**

- **Refining branch rules**

I'm making the policy specific without adding unnecessary process: branches should usually stay within five working days, avoid crossing a sprint, and require a documented reason if delayed.

- **Refined branch rules**

- **Outlined an adapted GitHub Flow methodology with branching, reviews, testing, and versioning**

✓ Worked for 1m 8s

For your project, **GitHub Flow adapted for Gitea is the strongest balance between simple and academically defensible**. Git Flow introduces unnecessary permanent branches, while direct commits to `main` provide too little review and traceability. Your selected method uses only issues, short-lived branches, commits, Pull Requests and `main`, while still producing clear evidence that the methodology is being followed. This is important because the rubric rewards actual, consistent use rather than documentation alone. `project_briefs.pdf`

Copy the following directly into a `.md` file:

Git Methodology

```
| Document Information | Details |
|---|---|
| Project | Sport Analytics Tool |
| Platform | Gitea |
| Methodology | GitHub Flow adapted for Gitea |
| Document Version | 1.0 |
| Date | 4 August 2026 |
| Status | Approved for Use |
| Team | [Insert Group Name or Number] |
```

1. Purpose

This document defines the Git methodology that will be followed throughout the development of the Sport Analytics Tool.

The methodology establishes:

- when commits must be created;
- how commit messages must be written;
- when branches must be created;

- how branches must be named;
- when branches may be merged;
- the requirements that must be met before merging;
- when project versions must be created; and
- how project versions must be named.

The purpose of the methodology is to maintain an organised repository, support effective collaboration, reduce merge conflicts, enable code review and provide traceability between requirements and their implementation.

All team members are required to apply this methodology consistently across all project repositories.

2. Selected Methodology

The team will use ****GitHub Flow adapted for Gitea****.

Under this methodology:

1. `main` contains the latest reviewed and working version of the project.
2. Each significant unit of work begins as a Gitea issue.
3. A short-lived branch is created for the issue.
4. Development is completed through small, meaningful commits.
5. The branch, commits and Pull Request reference the issue.
6. A Pull Request is reviewed before it is merged.
7. Approved work is merged into `main`.
8. Important milestone states are identified using Git tags.

This methodology was selected because it provides sufficient control, traceability and peer review without introducing unnecessary permanent branches or administrative processes.

A more complex methodology, such as Git Flow, would require additional `develop`, `release` and `hotfix` branches. These branches are not necessary for the size, duration or release structure of this project.

The selected methodology is therefore intended to remain lightweight enough to be followed consistently by all team members.

3. The `main` Branch

The `main` branch represents the latest integrated, reviewed and working version of the project.

The following rules apply to `main`:

- Direct development on `main` is prohibited.
- Direct pushes to `main` are prohibited.
- Changes may only enter `main` through an approved Gitea Pull Request.
- `main` must remain in a buildable and working state.
- Force pushes to `main` are prohibited.
- Required automated checks must pass before a Pull Request may be merged.
- Milestone version tags may only be created from `main`.

The `main` branch will be protected using Gitea's branch-protection settings where these settings are available.

4. Gitea Issues and Traceability

Every significant development task must begin as a Gitea issue.

An issue must contain:

- a clear title;
- a description of the required work;
- acceptance criteria;
- an assigned team member;
- the relevant sprint milestone; and
- appropriate labels.

Examples of work that requires an issue include:

- implementing a feature;
- fixing a bug;
- creating or changing an API endpoint;
- changing the database schema;
- adding automated tests;
- configuring infrastructure or CI/CD;
- completing substantial documentation; and
- performing a technical investigation that will influence development.

Very small corrections, such as correcting a spelling error, may be included in an existing relevant issue.

Each significant change will follow the traceability chain below:

```
```text
```

Gitea issue → branch → commits → Pull Request → merge

For example, work related to issue `#12` would be recorded as follows:

```
Issue: #12 Validate event submissions
Branch: feat/12-validate-event-submissions
Commit body: Refs #12
Pull Request: #12 Validate event submissions
PR body: Closes #12
```

This process allows the team to trace completed work back to its original requirement, implementation, review and final integration.

---

## 5. When to Create a Branch

A branch must be created when a team member begins work on a Gitea issue involving code, tests, configuration or substantial documentation.

Before creating a branch:

1. The related Gitea issue must exist.
2. The issue must have a clear description.
3. The issue must contain acceptance criteria.
4. The issue must have an assignee.
5. The issue must be assigned to the relevant sprint milestone.
6. The developer must update their local copy of `main`.

The branch must be created from the latest version of `main`.

```
git switch main
git pull origin main
```

```
git switch -c feat/12-validate-event-submissions
```

A branch should normally address one Gitea issue.

Where two issues cannot reasonably be separated, both issue numbers must be identified in the Pull Request description.

Branches must be short-lived. A normal branch should be completed within five working days. A branch may not remain open beyond the sprint in which it was created unless the delay and reason are recorded in the related issue.

If a branch becomes too large to review effectively, the work must be divided into smaller issues and branches.

## 6. How to Name Branches

Branch names must follow this format:

```
<type>/<issue-number>-<short-description>
```

The approved branch types are:

Type	Purpose
feat	New functionality
fix	Correction of incorrect behaviour
docs	Documentation changes
test	Test creation or modification
refactor	Code restructuring without a change in behaviour
chore	Maintenance, setup or configuration
ci	Continuous integration or deployment changes

Examples include:

```
feat/12-validate-event-submissions
fix/18-prevent-duplicate-statistics
docs/21-add-api-documentation
test/24-test-event-validation
refactor/31-extract-statistics-service
chore/7-configure-project
ci/9-add-backend-pipeline
```

Branch names must:

- use lowercase letters;
- contain the related issue number;
- separate words using hyphens;
- contain no spaces;
- use a concise but meaningful description; and
- reflect the main purpose of the branch.

## 7. When to Commit

A commit must be created whenever one small, logically complete unit of work has been completed.

Each commit must:

- represent one identifiable change;
- relate to the issue assigned to the branch;
- leave the affected section of the project in a working state;
- not knowingly introduce compilation or build failures;
- not knowingly cause existing tests to fail;
- exclude unrelated changes; and
- be understandable independently of later commits.

Examples of suitable commits include:

- creating the fixture database model;
- adding event-schema validation;
- implementing one API endpoint;
- correcting one calculation error;
- adding tests for one endpoint;
- updating the setup instructions for one service.

A commit must not combine unrelated work. For example, a single commit should not contain an unrelated API feature, user-interface redesign and documentation rewrite.

Developers must commit regularly and must not wait until an entire sprint or large feature has been completed before creating a commit.

Temporary commits may be used while work is in progress. However, commits with unclear messages such as `WIP`, `fix stuff` or `more changes` must not remain in a branch submitted for final review.

## 8. How to Write Commit Messages

The team will use a simplified Conventional Commit format.

The required format is:

```
<type>(<scope>): <description>
```

The scope may be omitted when it does not provide useful additional information:

```
<type>: <description>
```

### 8.1 Approved Commit Types

Type	Purpose
<code>feat</code>	Adds new functionality
<code>fix</code>	Corrects incorrect behaviour
<code>docs</code>	Changes documentation only
<code>test</code>	Adds or modifies tests
<code>refactor</code>	Restructures code without changing behaviour
<code>chore</code>	Changes maintenance, setup or configuration files
<code>ci</code>	Changes the CI/CD process

## 8.2 Commit Message Requirements

The commit description must:

- begin with a lowercase verb;
- state what the commit accomplishes;
- be concise but specific;
- use the imperative form; and
- avoid vague descriptions.

Acceptable commit messages include:

```
feat(api): add event submission endpoint
fix(stats): prevent duplicate events from affecting totals
test(api): add invalid event submission cases
docs(setup): document local database configuration
ci: run backend tests on pull requests
```

Unacceptable commit messages include:

```
updated files
fixed things
more work
changes
final version
stuff
```

## 8.3 Linking Commits to Issues

Every normal development commit must reference the related Gitea issue in its commit body.

```
feat(api): validate submitted event structure
```

```
Refs #12
```

The issue will normally be closed by the Pull Request rather than by an individual commit. This ensures that the issue remains open until all associated work has been reviewed and merged.

Merge commits, automated dependency commits and automatically generated revert commits are exempt from the normal issue-reference requirement where a manual reference would not be appropriate.

## 8.4 AI-Assisted Code

Where AI-generated code is included in a commit, the commit body must include the required tool and model attribution.

```
feat(api): validate submitted event structure
```

```
Refs #12
```

```
Assisted-by: ChatGPT-Web[GPT-5.6 Thinking]
```

Only tools and models genuinely used in producing the code contained in that commit may be listed.

## 9. When to Open a Pull Request

A Pull Request must be opened when the work on a branch is ready for review.

Before opening a Pull Request, the author must:

1. review the changes personally;
2. update the branch with the latest version of `main`;
3. resolve all merge conflicts;
4. run the relevant tests locally;
5. remove debugging code and unnecessary temporary files;
6. confirm that the issue's acceptance criteria have been addressed;
7. update relevant documentation;
8. confirm that no credentials or sensitive information have been committed; and
9. include the required AI attribution where applicable.

A draft Pull Request may be opened earlier when feedback is required before the work is complete. A draft Pull Request must be clearly identified and may not be merged.

The Pull Request title must contain the related issue number:

```
#12 Validate event submissions
```

The Pull Request description must include:

#### ## Purpose

Explain why the change is required.

#### ## Changes

Summarise the main changes made.

#### ## Testing

Explain how the change was tested.

#### ## Known Limitations

List any remaining limitations, or state that none are known.

#### ## Related Issue

Closes #12

## 10. When to Merge

A branch may only be merged after its Pull Request has completed the required review and verification process.

A developer may not merge their own Pull Request without approval from at least one other team member.

The team will use **merge commits** when merging Pull Requests into `main`.

Merge commits were selected because they preserve:

- the individual development commits;
- the identity of the feature branch;
- the Pull Request as a distinct unit of work; and
- the relationship between the issue and its implementation.

After a Pull Request has been merged:

1. The related Gitea issue must be closed.
  2. The completed branch must be deleted from Gitea.
  3. The associated Project board item must be moved to `Done`.
  4. Any incomplete or follow-up work must be recorded as a new issue.
- 

## 11. Requirements for Merging

A Pull Request may be merged only when all the following requirements have been satisfied.

### 11.1 Issue and Scope

- The Pull Request is linked to at least one Gitea issue.
- The issue's acceptance criteria have been met.
- The Pull Request contains no unrelated changes.
- Any approved change in scope has been recorded in the issue.

### 11.2 Description

- The purpose of the change is explained.
- The main implementation changes are summarised.
- The testing process is described.
- Known limitations are documented.
- The related issue is identified using `Closes #<issue-number>`.

### 11.3 Review

- The author has reviewed their own changes.
- At least one team member other than the author has reviewed the Pull Request.
- At least one approval has been recorded.
- All reviewer comments have been answered.
- All requested changes have been implemented or formally resolved.
- The reviewer has confirmed that the implementation matches its stated purpose.

### 11.4 Testing and Quality

- All relevant automated tests pass.
- Existing tests continue to pass.
- All required CI checks pass.
- Formatting, linting and type-checking checks pass where configured.
- The affected application or service builds successfully.
- Deployment checks pass where the change affects deployment.
- No unresolved merge conflicts remain.
- No credentials, secrets or sensitive information are included.
- No unnecessary generated or temporary files are included.

### 11.5 Documentation

- Relevant technical documentation has been updated.

- Setup instructions have been updated where necessary.
- API documentation has been updated where an endpoint or schema changed.
- Database documentation has been updated where the schema changed.
- AI assistance has been declared where required.

A Pull Request that does not meet every applicable requirement must not be merged.

---

## 12. When to Create a Version

A project version will be created when the team reaches a formal assessed milestone and the corresponding code has been reviewed and stabilised.

Versions will be created for:

- Sprint 1;
- Sprint 2;
- Sprint 3; and
- the final submission.

A version may only be created when:

1. All work included in the milestone has been merged into `main`.
2. The milestone version builds successfully.
3. All required automated checks pass.
4. The team has reviewed the milestone state.
5. Known defects and limitations have been documented.
6. The exact commit being submitted has been confirmed.
7. The repository documentation accurately reflects the submitted state.

The version will be recorded using an annotated Git tag created from `main`.

---

## 13. How to Name Versions

The project will use descriptive milestone versioning.

The primary version tags will be:

```
sprint-1
sprint-2
sprint-3
final-submission
```

If an authorised correction is required after a milestone version has been created, the corrected version will use a numbered revision:

```
sprint-1.1
sprint-1.2
```

An existing tag may not be moved, renamed or overwritten. A new tag must be created so that the original submitted version remains available.

Descriptive milestone versioning was selected because the project's most significant development versions correspond directly with its assessed milestones. This makes versions clear to both the development team and project assessors.

---

## 14. Standard Development Workflow

All significant work must follow the process below:

1. Create a Gitea issue.
2. Add a description and acceptance criteria.
3. Assign the issue to a team member and sprint milestone.
4. Update the local `main` branch.
5. Create a branch containing the issue number.
6. Complete the work using small, atomic commits.
7. Reference the issue in each applicable commit.
8. Test and review the work locally.
9. Open a Pull Request linked to the issue.
10. Obtain approval from another team member.
11. Resolve all review comments.
12. Ensure all required checks pass.
13. Merge the Pull Request into `main` using a merge commit.
14. Close the issue.
15. Delete the completed branch.
16. Move the Project board item to `Done`.
17. Create a version tag when the relevant milestone is complete.

---

## 15. Methodology Review and Change Control

This methodology is intended to remain in use for the duration of the project.

The team will not change the methodology merely because another process appears more convenient or because individual team members prefer a different workflow. Consistent use is necessary to maintain an understandable repository history and provide reliable evidence of the team's development process.

The methodology may only be changed where there is clear evidence that the current process is causing significant or repeated problems. Examples include:

- repeated merge conflicts caused by the workflow;
- substantial delays in reviewing or integrating work;
- a Gitea limitation that prevents the methodology from being followed;
- a process requirement that creates unnecessary work without providing value;
- difficulty tracing issues to their implementation; or
- a project change that makes the existing methodology unsuitable.

Before changing the methodology:

1. The problem must be discussed by the full team.
2. Evidence of the problem must be identified.
3. The proposed change and its expected benefit must be explained.
4. The change must be approved by a majority of the team.
5. The decision must be recorded in the project documentation.
6. This document must be updated with a new version number and effective date.

7. The revised process must be applied consistently from that date onward.

Changes will not be applied retrospectively. Work completed before the effective date of a methodology change will remain governed by the version of the methodology in effect at that time.

Minor wording corrections that do not alter the actual development process may be made without following the full change procedure.

---

## 16. Evidence of Compliance

The team will demonstrate compliance with this methodology through evidence available in Gitea.

This evidence will include:

- Gitea issues containing acceptance criteria;
- issues assigned to team members and sprint milestones;
- consistently named branches;
- atomic commits with standardised messages;
- issue references in branches and commits;
- Pull Requests linked to issues;
- peer-review comments and approvals;
- successful CI checks;
- merged Pull Requests;
- closed issues;
- contributions from all team members; and
- milestone version tags.

The existence of this document alone will not be treated as evidence that the methodology has been followed. The repository history and Gitea activity must demonstrate its consistent use.

---

## AI Declaration

The preceding document was planned, generated, reviewed and edited with the assistance of ChatGPT-Web[GPT-5.6 Thinking].

```

8/4/2026, 4:15:31 PM



why did u stop markdown?? give entire thing in markdown, or a .md to download?

8/4/2026, 4:18:14 PM



You're right. Here is the complete Markdown file:

[Download the submission-ready Git Methodology](#)

8/4/2026, 4:18:21 PM



ok, now do same for project methodology, first on the chat so we can discuss and then a .md

8/4/2026, 4:25:40 PM



Thinking

- **Structuring the workflow**

I'm organizing the project brief into Basic, Intermediate, and Advanced tiers, prioritizing foundational work first. I'm adding lightweight WIP limits, readiness criteria, weekly stakeholder feedback, controlled changes, and clear evidence records.

- **Structured the workflow**

- **Refining the methodology**

I'm recommending three brief standups weekly, one stakeholder check-in per sprint, and issues for every meaningful task. Minor fixes can be bundled. I'll keep the process lightweight while preserving evidence for milestones, reviews, and retrospectives.

- **Refined the methodology**

✓ Worked for 45s

For the **Project Methodology**, I recommend a **lightweight Scrumban methodology**.

The supplied project documents do not prescribe Scrum, Kanban or another specific methodology. They require the team to select, document, motivate and consistently use a methodology, with visible evidence of its use. The Sprint 1 rubric gives the highest range where the methodology is motivated and supported by evidence. [project_briefs.pdf](#)

Why Scrumban is the best fit

Your project already has fixed milestone periods called Sprint 1, Sprint 2, Sprint 3 and Final Submission. However, your requirements and priorities are likely to change after stakeholder meetings, technical investigation and testing.

[project_briefs.pdf](#)

Scrumban combines:

- **Scrum's sprint structure:** planning, standups, reviews and retrospectives.
- **Kanban's visual workflow:** tasks move across a Gitea Project board.
- **Lightweight administration:** no complicated Scrum roles, story points, velocity calculations or excessive meeting minutes.

I would not recommend full Scrum because it can create unnecessary administration for a student team. I would also not recommend pure Kanban because the project has formal sprint deadlines and needs evidence of sprint planning and review.

Proposed Project Methodology

1. Work-management platform

Gitea will be the team's **single source of truth** for project work.

The team will use:

- Gitea Issues for tasks, features, bugs and investigations;
- Gitea Projects for the work board;
- Gitea Milestones for each assessed sprint;
- Pull Requests for reviewing implementation;
- repository documentation for decisions and meeting records.

Work discussed in WhatsApp, meetings or private messages must be transferred to Gitea where it affects the project.

2. Project board

The Gitea Project board will use these columns:

Backlog
Ready
In Progress
In Review
Blocked
Done

Backlog

Identified work that has not yet been selected or fully prepared.

Ready

Work that has clear acceptance criteria and can be started.

In Progress

Work currently being completed.

In Review

Work that has been completed by its assignee and is awaiting review, testing or approval.

Blocked

Work that cannot continue because of a dependency, technical problem or unanswered stakeholder question.

Done

Work that satisfies the team's Definition of Done.

I recommend limiting each team member to **one main issue in progress at a time**. This should prevent members from starting several tasks but completing none of them.

3. Sprint structure

Each formal milestone period will be treated as one sprint:

Sprint 1
Sprint 2
Sprint 3
Final Submission

At the start of each sprint, the team will identify a clear sprint goal and select the issues that should be completed.

For example, the Sprint 1 goal could be:

Establish the project foundations, remove major technical blockers and produce sufficient design, documentation and implementation to support later development.

This aligns with the project brief's description of Sprint 1 as the period for repository setup, removing major blockers and establishing a clear direction. [project_briefs.pdf](#)

4. Sprint planning

The team will hold one planning meeting at the start of each sprint.

The meeting should take approximately **30 to 45 minutes**.

During planning, the team will:

1. agree on the sprint goal;
2. review the project requirements;
3. review stakeholder feedback;
4. select issues from the backlog;
5. confirm acceptance criteria;
6. assign initial owners;
7. identify dependencies and risks;
8. place selected work in the sprint milestone.

The meeting does not need a full transcript. A short planning record should include:

- the sprint goal;
 - the major work selected;
 - important risks;
 - decisions made;
 - unresolved questions.
-

5. Standups

I recommend **three short standups per week**, rather than daily standups.

They can take place on Monday, Wednesday and Friday and should last no longer than **10 minutes**.

Each member answers:

1. What have I completed since the previous standup?
2. What will I work on next?
3. Is anything blocking my progress?

The standup is for coordination, not detailed problem-solving. Any technical discussion involving only some members should happen immediately afterwards.

You do not need to write formal minutes for every standup. Instead, the team can keep one brief weekly summary recording:

- important progress;
- current blockers;
- changed responsibilities;
- decisions;
- actions for the next week.

Near a milestone deadline, the team may temporarily move to daily standups.

6. Gitea issues

Every meaningful unit of work must have a Gitea issue.

Each issue should contain:

Description

Explain what must be completed.

Motivation

Explain why the work is required.

Acceptance Criteria

- [] Specific observable result
- [] Relevant tests pass
- [] Documentation is updated where applicable

Dependencies

List related or blocking issues.

Evidence

Link relevant designs, discussions, Pull Requests or test results.

Each issue must also have:

- an assignee;
- a sprint milestone;
- a work-type label;
- an area label where appropriate.

Suggested labels:

```
type: feature
type: bug
type: documentation
type: testing
type: research

area: frontend
area: api
area: database
area: infrastructure

priority: high
status: blocked
```

There is no need to use story points. To keep the process simple, an issue should ideally be small enough to complete within one to three working days. Larger issues should be divided.

7. Prioritisation

Work will be prioritised according to:

1. core project requirements;
2. work required for the current sprint rubric;
3. dependencies that block other work;
4. stakeholder priorities;

5. technical risk;
6. optional enhancements.

The team should first secure the **Basic** requirements before committing significant time to Intermediate or Advanced features. The project brief requires the Basic tier to meet all key and basic requirements at minimum viable product level. `project_briefs.pdf`

For the Sport Analytics Tool, early priority should be given to event data, submission validation, traceability, the API and statistic derivation because these form the central purpose of the product. `sport_analytics.pdf`

8. Definition of Ready

An issue may move to `Ready` only when:

- its purpose is understood;
- it has acceptance criteria;
- major dependencies are known;
- required stakeholder clarification has been obtained;
- it is small enough to complete within the sprint;
- the team understands how it will be tested or verified.

Research issues may use a defined output, such as a documented recommendation or prototype, instead of software acceptance criteria.

9. Definition of Done

An issue may move to `Done` only when:

- all acceptance criteria have been satisfied;
- the work has been reviewed;
- relevant tests pass;
- required CI checks pass;
- relevant documentation has been updated;
- no unresolved review comments remain;
- the work has been merged into `main`;
- the related Gitea issue has been closed.

For user-facing work, the team should also confirm that the change is responsive, accessible and usable.

10. Stakeholder interaction

The team should have **one stakeholder interaction each week where reasonably possible**. This may be a short meeting, demonstration or structured asynchronous discussion.

Stakeholder interaction should be used to:

- clarify requirements;
- confirm priorities;
- demonstrate current progress;
- request feedback;
- verify whether features meet expectations;
- discuss proposed scope changes.

A short stakeholder record should include:

```
# Stakeholder Interaction

**Date:**
**Attendees:**
**Purpose:**

## Requirements Discussed

## Feedback Received

## Decisions

## Actions

## Related Gitea Issues

- #...
```

Feedback that changes the product must update an existing issue or create a new one. Requirements must not be changed silently.

This will provide evidence for the stakeholder-interaction criterion as well as evidence that the project methodology is actively being used. `project_briefs.pdf`

11. Sprint review

At the end of each sprint, the team will review and demonstrate the completed work.

The review should record:

- the sprint goal;
- work completed;
- features demonstrated;
- incomplete work;
- stakeholder feedback;
- accepted changes;
- new requirements;
- known defects and limitations.

Only issues in `Done` should be described as completed.

Incomplete work must either:

- return to the backlog;
- be moved to the next sprint with justification; or
- be removed from scope with a recorded reason.

12. Sprint retrospective

After the sprint review, the team will hold a short retrospective of approximately **15 to 20 minutes**.

The team will discuss:

1. What worked well?
2. What caused difficulty?

3. What should the team continue doing?
4. What should change in the next sprint?

The retrospective should produce no more than two or three practical improvement actions. Each action should have an owner.

This prevents the retrospective from becoming a long theoretical discussion that produces no actual change.

13. Roles and responsibilities

To keep the methodology lightweight, the team will not use strict Scrum roles.

Instead:

- all members are responsible for development and project quality;
- issue owners are responsible for progressing and updating their assigned work;
- reviewers are responsible for checking work before integration;
- one member acts as **Sprint Coordinator** for each sprint;
- the Sprint Coordinator may rotate between sprints.

The Sprint Coordinator will:

- organise planning and review meetings;
- ensure the board remains updated;
- identify neglected or blocked work;
- ensure stakeholder notes and sprint records are stored.

The Sprint Coordinator does not act as the manager of the other team members and is not solely responsible for the sprint's success.

14. Methodology changes

The team will use this methodology consistently for the duration of the project.

It will not be changed merely because another approach appears more convenient or because one member prefers a different process.

The methodology may only be changed where there is clear evidence that it is causing repeated or significant problems, such as:

- excessive meeting or documentation overhead;
- repeated delays caused by the process;
- work becoming difficult to track;
- important tasks regularly being blocked;
- Gitea limitations preventing the process from being followed;
- team members repeatedly misunderstanding responsibilities;
- the project changing in a way that makes the methodology unsuitable.

Before changing the methodology:

1. the problem must be discussed by the full team;
2. evidence of the problem must be identified;
3. the proposed change must be explained;

4. the team must approve the change;
5. the decision and reason must be documented;
6. the methodology document must be updated;
7. the new process must be followed consistently from its effective date.

Changes will not be applied retrospectively.

15. Evidence of use

The team will demonstrate that the methodology has been followed through:

- a populated Gitea Project board;
- issues with acceptance criteria;
- sprint milestones;
- issue assignments;
- movement of work across board columns;
- blocked issues with explanations;
- stakeholder records;
- sprint planning records;
- sprint review records;
- retrospective actions;
- Pull Requests and reviews;
- completed and closed issues.

The documentation alone will not be considered sufficient evidence. The Gitea activity and repository history must show that the methodology was actually followed.

Points I recommend confirming before I turn this into the final submission document

My recommended decisions are:

- **Methodology:** lightweight Scrumban.
- **Standups:** Monday, Wednesday and Friday for 10 minutes.
- **Stakeholder contact:** once per week where possible.
- **Sprint Coordinator:** one rotating member per sprint.
- **Work-in-progress limit:** one main active issue per member.
- **Estimating:** no story points; issues should normally take one to three days.
- **Methodology changes:** only when the current process is causing evidenced problems.

The main point to decide is whether you want standups **three times per week** or **daily**. Three times per week is easier to sustain and should still provide strong evidence that the methodology is being used.

8/4/2026, 4:26:14 PM



standups once a week?
meet stakeholders on tuesday, standups on thursday? or something like that? do we need a sprint coordinator

8/4/2026, 4:31:00 PM



Thinking

- **Designing a lightweight cadence**

I'm recommending a weekly Thursday stand-up, with Tuesday stakeholder sessions feeding into it. It's enough if updates, blockers, and actions are documented. A permanent sprint coordinator isn't necessary; rotate a facilitator each sprint.

- **Designed a lightweight cadence**

- **Structuring weekly coordination**

I'm recommending Tuesday stakeholder check-ins and a short Thursday stand-up. The rubric requires evidence, not a specific frequency, so weekly is sufficient if issues stay updated. A formal coordinator isn't needed; rotate a facilitator and note-taker each week.

- **Structured weekly coordination**

- **Established lightweight weekly standups with rotating facilitation and documentation**

✓ Worked for 34s

Yes, **one standup per week can work**, provided the Gitea board is updated during the week rather than only during meetings.

The rubric does not prescribe daily standups or require a Scrum Master. It rewards a methodology that is motivated, consistently followed and supported by evidence. It also values regular stakeholder interaction and evidence that feedback was evaluated and integrated. [project_briefs.pdf](#) [project_briefs.pdf](#)

Recommended weekly structure

Tuesday: Stakeholder meeting

A short meeting with the assigned tutor or stakeholder, ideally **20-30 minutes**.

The team should:

- demonstrate recent progress;
- clarify requirements;
- ask questions;
- receive feedback;
- confirm priorities;
- record any decisions;
- create or update the relevant Gitea issues.

Tuesday works well because there is still enough time during the week to respond to the stakeholder's feedback.

Thursday: Team standup

A **15-minute internal team standup**.

Each member answers:

1. What have I completed since the previous standup?

2. What am I currently working on?
3. What will I complete next?
4. Is anything blocking me?

The team should then briefly:

- review the Gitea board;
- check progress towards the sprint goal;
- address actions arising from Tuesday's stakeholder meeting;
- reassign work where necessary;
- identify tasks that are blocked or falling behind.

Detailed technical discussions should happen after the standup with only the relevant team members.

Is one standup enough?

Yes, because this is a student project with a lightweight methodology rather than strict Scrum.

However, the following rule should be included:

Team members must update their assigned Gitea issues whenever the status of their work changes. The weekly standup is used to coordinate work and identify blockers; it is not the only time at which the Project board is updated.

Without this rule, one weekly standup could result in the board becoming outdated. The rubric specifically distinguishes between merely having a work tracker and actually using it. [project_briefs.pdf](#)

During the final few days before a milestone, the team may arrange additional short standups when needed. These should be treated as an exception rather than a permanent requirement.

Do you need a Sprint Coordinator?

No formal Sprint Coordinator is required.

Adding a permanent coordinator could make the methodology more complicated than necessary and may incorrectly suggest that one person is responsible for managing everyone else.

I recommend using a **rotating meeting facilitator** instead.

For each week, one team member will:

- arrange or open the Tuesday stakeholder meeting;
- facilitate the Thursday standup;
- ensure that brief meeting notes are saved;
- check that decisions have been converted into Gitea issues.

This role should rotate weekly and does not give the facilitator authority over the other team members. Every team member remains responsible for updating their own issues and completing their assigned work.

Final recommended choice

Your methodology should therefore state:

- **Project methodology:** Lightweight Scrumban.
- **Stakeholder meeting:** Every Tuesday, where reasonably possible.
- **Team standup:** Every Thursday for approximately 15 minutes.
- **Additional meetings:** Only when necessary, particularly before milestones.
- **Coordination:** Rotating weekly meeting facilitator, not a formal Sprint Coordinator.

- **Work tracking:** Gitea issues and the Project board must be updated continuously.
- **Sprint review and retrospective:** Conducted at the end of each formal sprint.

This is simpler than full Scrum but still gives you strong evidence of planning, stakeholder interaction, tracking, reflection and consistent methodology use.

8/4/2026, 4:31:14 PM



ok, give me the .md doc

8/4/2026, 4:33:43 PM



Download the submission-ready Project Methodology.

8/4/2026, 4:33:54 PM



Pasted markdown.md

do we need the sprint reviews and retrospectives etc??

8/4/2026, 4:42:12 PM



Thinking

- **Clarifying sprint practices**

I'm recommending a lightweight approach: use the existing Tuesday stakeholder meeting as the sprint review, and hold a brief internal retrospective. I'll frame both in one "Sprint close-out" section without adding unnecessary ceremonies.

- **Clarified sprint practices**

- **Evaluated lightweight sprint ceremonies and close-out practices**

✓ Worked for 31s

No, **separate formal sprint reviews and retrospectives are not explicitly required.**

The rubric requires you to:

- document and motivate a project methodology;
- provide evidence that you actually followed it;
- use the work tracker;
- interact regularly with the stakeholder;
- evaluate and integrate stakeholder feedback. `project_briefs.pdf` `project_briefs.pdf`

The project brief also says later milestones should involve reviews and qualitative evaluation, and the final report requires reflection on decisions and possible improvements. `project_briefs.pdf`

Easiest effective approach

Do **not** schedule two additional formal meetings.

Sprint review

Use the **Tuesday stakeholder meeting closest to the end of the sprint** as the sprint review.

During that meeting, briefly:

- demonstrate what was completed;
- confirm what the stakeholder accepts;
- identify unfinished work;
- record feedback and changed requirements.

This already supports the stakeholder-interaction and stakeholder-review criteria.

Retrospective

Add a **five- to ten-minute reflection** to the Thursday standup at the end of each sprint.

Only answer:



1. What worked well?
2. What caused problems?
3. What is one change we will make next sprint?

Record these answers in one short note. You do not need a separate formal retrospective meeting, action table or long template.

Recommended change to the methodology

Replace the separate **Sprint Review** and **Sprint Retrospective** sections with one section:

Sprint Close-Out

At the end of each sprint, the team will complete a brief sprint close-out.

The Tuesday stakeholder meeting closest to the sprint deadline will be used to demonstrate completed work, collect feedback and confirm how incomplete work will be handled.

During the final Thursday standup of the sprint, the team will briefly reflect on:

1. what worked well;
2. what caused difficulty; and
3. one improvement to apply during the next sprint.

A short sprint close-out record will document:

- completed work;
- incomplete work;
- stakeholder feedback;
- known limitations; and
- the agreed improvement for the next sprint.

The sprint close-out is intended to provide useful evidence of review and reflection without creating unnecessary additional meetings or documentation.

Also change the sprint structure to:

Every sprint will include:

1. sprint planning;
2. continuous work tracking;
3. weekly stakeholder interaction;
4. a weekly team standup;
5. development and review; and
6. a brief sprint close-out.

This is more consistent with your stated goal of an **effective, easy and lightweight methodology**. Your current document commits you to more administration than the rubric actually requires. `Project_Methodology.md`

8/4/2026, 4:42:15 PM



can u give me the whole .md then (keeping my changes)

8/4/2026, 4:43:39 PM



Download the revised Project Methodology.

I kept your changes, replaced the separate sprint review and retrospective with one lightweight **Sprint Close-Out**, and removed the remaining reference to a rotating facilitator so the document is internally consistent.

8/4/2026, 4:43:51 PM



ok, now hoqw to push these?? the repo is completley empty, and want to follow all the methodology we spoke about

8/4/2026, 4:59:00 PM



Thinking



Searching the web

docs.gitea.com dictionary.cambridge.org pullapart.com byn.skin cdc.gov github.com

- **Planning repository setup**

I'm outlining a traceable workflow: create separate issues for Git and project methodology, use one branch per issue, commit the documents, open pull requests, review, merge, and keep the default branch clean.

- **Planned repository setup**

- **Preparing the documentation workflow**

I'm turning the plan into concrete steps: create an issue, branch from `main`, add both Markdown files and README links, commit with the issue reference, open a pull request, review, merge, and clean up.

- **Prepared the documentation workflow**



Searching the web

- **Searched 6 websites**

✓ Worked for 1m 42s

Here is the cleanest way to initialise the empty repository **without immediately breaking the methodology you just documented**.

Because the repository has no commits or branches yet, you need **one small bootstrap commit directly to `main`**. This is a one-time technical exception: a Pull Request cannot exist until the repository has an initial branch. After that first commit, protect `main` and use issues, branches and Pull Requests for everything else.

1. Download the two documents

Download them somewhere easy to find:

- [Git Methodology](#)
- [Project Methodology](#)

Later, rename them to:

```
git-methodology.md
project-methodology.md
```

Part A: Set up Gitea first

2. Create the Sprint 1 milestone

In your Gitea repository:

```
Issues → Milestones → New Milestone
```

Use:

Title: Sprint 1
Due date: 25 August 2026

3. Create the Project board

Create these columns:

Backlog
Ready
In Progress
In Review
Blocked
Done

4. Create the first issue

Create this issue before pushing anything.

Issue title

Initial repository setup and methodology documentation

Issue body

Description

Bootstrap the repository and add the approved Git Methodology and Project Methodology documents.

Motivation

The repository requires an initial structure and documented development processes before implementation begins.

Acceptance Criteria

- [] The `main` branch exists
- [] The `main` branch is protected
- [] A root `README.md` file exists
- [] `docs/git-methodology.md` exists
- [] `docs/project-methodology.md` exists
- [] The README links to both methodology documents
- [] The documentation is reviewed through a Pull Request
- [] The Pull Request is merged into `main`

Dependencies

None.

Evidence

The merged Pull Request will provide evidence of completion.

Set:

Assignee: one group member
Milestone: Sprint 1
Label: type: documentation
Area: area: documentation

This will probably become issue `#1` . Use the actual issue number Gitea gives you.

Move the issue to:

Ready

Part B: Make the one-time bootstrap commit

5. Copy the repository URL

On the main Gitea repository page, copy the HTTPS clone URL. It will look something like:

```
https://your-gitea-server/group-name/sport-analytics.git
```

6. Open a terminal

On Windows, use:

- Git Bash;
- the VS Code terminal; or
- PowerShell with Git installed.

Move to the folder where you want to store the project.

For example:

```
cd Documents
mkdir sport-analytics
cd sport-analytics
```

7. Initialise Git

Replace the URL below with your actual Gitea clone URL:

```
git init -b main
git remote add origin <YOUR-GITEA-REPOSITORY-URL>
```

Example:

```
git remote add origin https://gitea.example.com/group/sport-analytics.git
```

Check that the remote was added:

```
git remote -v
```

8. Create the initial README

Create a file called:

```
README.md
```

Paste this inside:

```
# Sport Analytics Tool

A sport analytics platform developed for COMS3011A.

## Documentation
```

Project documentation is maintained in the [``docs``]([docs/](#)) directory.

Development Methodologies

- [Git Methodology]([docs/git-methodology.md](#))
- [Project Methodology]([docs/project-methodology.md](#))

AI Usage

This repository does not currently use AI code generation.

This repository does not currently use AI inline-editing tools.

This repository does not currently use AI code review.

Update the AI declarations as soon as the team begins using AI for code generation, inline editing or code review.

9. Make the bootstrap commit

Run:

```
git add README.md
git status
```

You should see only `README.md` staged.

Commit it:

```
git commit -m "chore: bootstrap repository" -m "Refs #1"
```

Replace `#1` if Gitea assigned a different issue number.

Push the first commit:

```
git push -u origin main
```

This is the **only direct push to** `main`. Do not add the methodology documents in this bootstrap commit.

Gitea recognises issue references such as `#1` in commit messages and Pull Requests. A Pull Request description containing `Closes #1` can close that issue when the Pull Request is merged. ([Gitea Documentation](#))

Part C: Protect `main`

10. Add branch protection

In Gitea, navigate approximately to:

Repository Settings → Branches → Protected Branches

Create a protection rule for:

`main`

Configure it to:

- prevent normal direct pushes;
- prevent force pushes;
- require changes to enter through Pull Requests;
- require at least one approval;

- prevent authors from approving their own Pull Requests, where available.

Do not require CI status checks yet unless you already have a working CI pipeline. Add that requirement once CI has been configured.

Gitea protected branches can enforce push and merge restrictions and can require a specified number of approvals before merging. ([Gitea Documentation](#))

Part D: Add the documents properly

11. Move the issue to In Progress

On the Gitea Project board, move issue #1 from:

Ready → In Progress

12. Create the issue branch

Return to your terminal:

```
git switch main
git pull origin main
git switch -c docs/1-add-methodology-documents
```

Replace 1 with the actual issue number where necessary.

Your branch now follows the methodology:

```
docs/<issue-number>-<description>
```

13. Create the documentation folder

Run:

```
mkdir docs
```

Using File Explorer, copy the downloaded files into the new docs folder.

Rename them as follows:

```
Git_Methodology.md
↓
git-methodology.md
```

```
Project_Methodology_v1.1.md
↓
project-methodology.md
```

Your repository should now look like this:

```
sport-analytics/
├─ README.md
└─ docs/
    ├─ git-methodology.md
    └─ project-methodology.md
```

14. Check your changes

Run:

```
git status
```

You should see:

```
docs/git-methodology.md
docs/project-methodology.md
```

The README links already point to these locations.

15. Commit the documents

Stage the files:

```
git add docs/git-methodology.md docs/project-methodology.md
```

Check again:

```
git status
```

Commit:

```
git commit -m "docs: add project methodologies" -m "Refs #1"
```

Push the branch:

```
git push -u origin docs/1-add-methodology-documents
```

Part E: Open the Pull Request

16. Create the Pull Request in Gitea

Go to:

```
Pull Requests → New Pull Request
```

Select:

```
Source branch: docs/1-add-methodology-documents
Target branch: main
```

A Gitea Pull Request proposes merging one branch into another and provides a place to describe and review the change. ([Gitea Documentation](#))

Pull Request title

```
#1 Add project methodology documentation
```

Pull Request description

Purpose

Add the approved Git Methodology and Project Methodology documents to the repository.

Changes

- Added `docs/git-methodology.md`
- Added `docs/project-methodology.md`

```
- Confirmed that both documents are linked from the repository README
```

Testing

```
- Confirmed that both Markdown files render correctly in Gitea  
- Confirmed that the README links open the correct documents  
- Confirmed that no unrelated files are included
```

Known Limitations

None.

Related Issue

Closes #1

Create the Pull Request and move issue #1 to:

In Review

Part F: Review and merge

17. Ask another team member to review it

The reviewer should:

- open both documents;
- check that the README links work;
- confirm that the files render correctly;
- confirm that no unrelated files were added;
- leave an approval in Gitea.

They should not merely send “looks good” through WhatsApp. The approval should be recorded on the Pull Request so that you have evidence of following the methodology.

18. Merge the Pull Request

Once approved:

Merge method: Create a merge commit

Then:

- merge the Pull Request;
- delete the remote branch;
- confirm that issue #1 closes automatically;
- move the issue to Done if Gitea does not do so automatically.

19. Update your local repository

Return to your terminal:

```
git switch main  
git pull origin main  
git branch -d docs/1-add-methodology-documents
```

Your local main should now contain:

```
README.md
docs/git-methodology.md
docs/project-methodology.md
```

Do not create a version tag yet

Do not tag this as `sprint-1` now.

The `sprint-1` tag should only be created at the end of Sprint 1, once all Sprint 1 work has been reviewed, merged and confirmed as the submitted version.

Your first complete workflow will therefore be:

```
Issue #1
  ↓
One-time bootstrap of main
  ↓
Protect main
  ↓
docs/1-add-methodology-documents
  ↓
docs: add project methodologies
  ↓
Pull Request
  ↓
Peer approval
  ↓
Merge into main
  ↓
Close Issue #1
```

Paste the Gitea clone URL here and I'll replace the placeholder with your exact commands.

Sources

[Automatically Linked References | Gitea Documentation](#) docs.gitea.com

8/4/2026, 4:59:02 PM



377aeea9-5cf1-4c08-81e6-0ad...

how to create the project board columns?

8/4/2026, 8:32:41 PM



Thinking



Searching the web

docs.gitea.com

- Searched 1 website

✓ Worked for 18s

You are currently on the **Sprint 1 milestone page**, not the Project board.

Create the columns

1. Click **Projects** in the top navigation bar. It has a small **1** beside it in your screenshot.
2. Click the existing project board.
3. Inside the board, look on the far right for one of these buttons:
 - **New Column**
 - **Add Column**
 - a **+** button
 - a **three-dot menu** containing **New Column**
4. Add these columns individually, in this order:

Backlog
Ready
In Progress
In Review
Blocked
Done

5. Drag the column headings left or right to put them in the correct order, if necessary.

Gitea Projects are designed to organise issues using board columns. Some installations also offer a basic Kanban template with default columns such as **To Do**, **In Progress** and **Done**, but you can rename or replace those with your chosen workflow. ([Gitea Documentation](#))

Important distinction

- **Sprint 1 milestone:** groups issues according to the submission deadline.
- **Project board:** shows the workflow status of those issues.
- **Columns:** show whether an issue is waiting, active, under review, blocked or completed.



Therefore, the same issue can be:

Milestone: Sprint 1
Project column: In Progress

Click **Projects**, open the board, and send the next screenshot if the column button is not visible.

Sources

[Configuration Cheat Sheet](#) | [Gitea Documentation](#) docs.gitea.com

8/4/2026, 8:32:44 PM



3e67ad77-a6da-4cc0-9c81-ad...

uhm, now im out of sprint 1?

8/4/2026, 8:34:16 PM



Yes, you are now on the **Projects page**, which is separate from the **Sprint 1 milestone page**. You have not deleted or left Sprint 1 permanently.

Click the project name:

Sport-Analytics-Tool-Proj

That should open the actual board. Inside it, you should then see the board columns or an option such as **New Column / Add Column**.

Create:

Backlog
Ready
In Progress
In Review
Blocked
Done

The two tools work together:

- **Milestone: Sprint 1** = which sprint the issue belongs to
- **Project board column** = the issue's current status

So one issue can simultaneously be:

Milestone: Sprint 1
Project: Sport-Analytics-Tool-Proj
Column: In Progress

After creating an issue, assign it to both **Sprint 1** and the **Project board**.

8/4/2026, 8:34:25 PM



160113d3-0538-48ba-aa2a-c8...

repo has this, ive added the board.. now give ,me instructions from here to do whjat we were doing above keeping in mind this stat with step 4, create the first issue

8/4/2026, 8:37:25 PM



Thinking



Searching the web

docs.gitea.com

- **Planning the next steps**

I'm starting with issue creation because the repository already has its initial commit. I'll outline exact Gitea actions, then branch naming, documentation commits, a pull request, protection, and Sprint 1 tracking.

- **Planned the next steps**

- **Searched 1 website**

✓ Worked for 18s

Your repository is **already initialised**: it has `main`, a README, `.gitignore`, and one initial commit. Therefore, do **not** make another bootstrap commit directly to `main`. From this point onward, use an issue, branch, commits and a Pull Request.

Step 4: Create the first issue

4.1 Open the issue form

From the repository page shown:

1. Click **Issues** in the top navigation.
2. Click **New Issue**.

4.2 Use this issue title

Add Git and project methodology documentation

4.3 Paste this issue description

Description

Add the approved Git Methodology and Project Methodology documents to the repository and link them from the main README.

Motivation

The project requires documented and consistently applied Git and project methodologies. Storing these documents in the repository provides a version-controlled record of the team's agreed development processes.

Acceptance Criteria

- [] A ``docs`` directory exists
- [] ``docs/git-methodology.md`` exists



- [] `docs/project-methodology.md` exists
- [] The main `README.md` links to both methodology documents
- [] Both Markdown documents render correctly in Gitea
- [] The changes are reviewed through a Pull Request
- [] The Pull Request is merged into `main`

Dependencies

None.

Evidence

The linked branch, commits and merged Pull Request will provide evidence of completion.

4.4 Assign the issue

Before creating it, use the options on the right-hand side where available:

- **Assignee:** assign the group member who will add the documents
- **Milestone:** Sprint 1
- **Project:** Sport-Analytics-Tool-Proj

Do not worry about the Project board column yet. We will move the issue to **Ready** after creating it.

4.5 Create the issue

Click **Create Issue**.

Gitea will give it a number, probably #1. Write down the actual number because it will be used in the branch name, commits and Pull Request.

Step 5: Add the issue to the Project board

After creating the issue:

1. Look at the issue's right-hand sidebar.
2. Find **Projects**.
3. Select Sport-Analytics-Tool-Proj if it was not already selected.
4. Return to **Projects**.
5. Open Sport-Analytics-Tool-Proj.
6. Find the new issue card.
7. Move it into the **Ready** column.

At this point, the issue should simultaneously have:

Milestone: Sprint 1
Project: Sport-Analytics-Tool-Proj
Column: Ready

The milestone indicates **when** the work is due; the Project column indicates its current workflow status.

Step 6: Create any missing labels

Your methodology says that issues should have a work-type label and an area label.

Go to:

Issues → Labels

Create these two labels for this issue:

```
type: documentation
area: documentation
```

The label colours are not important. Choose any readable colours.

Return to the issue and assign both labels.

The completed issue should now show:

```
Assignee: [team member]
Milestone: Sprint 1
Project: Sport-Analytics-Tool-Proj
Labels:
- type: documentation
- area: documentation
```

Once all this information is present, the issue meets your Definition of Ready.

Step 7: Move the issue to In Progress

When you are ready to begin adding the files:

1. Open the Project board.
2. Drag the issue from **Ready** to **In Progress**.

Do this before starting the branch.

Step 8: Protect main

Because the initial commit already exists, you can protect main now.

You need repository administration permission.

Go to:

Repository Settings → Branches

Create a protected-branch rule for:

main

Configure the rule to:

- block direct pushes;
- block force pushes;
- require Pull Requests before merging;
- require at least one approval, if your Gitea installation supports this;
- prevent the author from approving their own Pull Request, if available.

Do **not** require successful CI checks yet unless a CI workflow already exists. You can enable that later when CI has been configured.

Gitea branch-protection rules can control direct pushes, merging and review requirements on main. ([Gitea Documentation](#))

Step 9: Download the methodology files

Download these two files:

- [Git Methodology](#)
- [Project Methodology](#)

Save them somewhere accessible, such as your Downloads folder.

You will rename them to:

```
git-methodology.md
project-methodology.md
```

Step 10: Copy the repository clone URL

On the repository's **Code** page:

1. Click the blue **Code** button.
2. Select either **HTTPS** or **SSH**.
3. Copy the clone URL.

It will resemble:

```
https://<gitea-server>/git-push-pray/Sport-Analytics-Tool.git
```

Use the actual URL displayed by your Gitea installation.

Step 11: Clone the repository locally

Open the VS Code terminal, Git Bash or PowerShell.

Move to the folder where you want the project stored:

```
cd Documents
```

Clone the repository:

```
git clone <PASTE-YOUR-GITEA-URL-HERE>
```

Then enter the repository:

```
cd Sport-Analytics-Tool
```

Check the current branch and files:

```
git status
git branch
```

You should be on:

```
main
```

You should see:

```
README.md
.gitignore
```

Step 12: Create the issue branch

Replace `1` below with the actual issue number.

First update `main`:

```
git switch main
git pull origin main
```

Create the branch:

```
git switch -c docs/1-add-methodology-documents
```

The branch follows your agreed format:

```
<type>/<issue-number>-<short-description>
```

Confirm the branch:

```
git branch
```

You should see an asterisk beside:

```
docs/1-add-methodology-documents
```

Step 13: Add the methodology files

Create a `docs` directory:

```
mkdir docs
```

Move or copy the downloaded documents into it.

Rename them so the repository contains:

```
Sport-Analytics-Tool/
├─ .gitignore
├─ README.md
└─ docs/
    ├─ git-methodology.md
    └─ project-methodology.md
```

On Windows PowerShell, the commands may look like:

```
Copy-Item "$HOME\Downloads\Git_Methodology.md" "docs\git-methodology.md"
Copy-Item "$HOME\Downloads\Project_Methodology_v1.1.md" "docs\project-methodology.md"
```

Using File Explorer to copy and rename the files is also fine.

Step 14: Update the existing README

Do not replace the existing project description. Add this section beneath it:

```
## Documentation

Project documentation is stored in the [docs`](docs/) directory.

### Development Methodologies
```

```
- [Git Methodology](docs/git-methodology.md)
- [Project Methodology](docs/project-methodology.md)
```

Also add the required AI declaration section if it is not already present:

```
## AI Usage

This repository does not currently use AI code generation.

This repository does not currently use AI inline-editing tools.

This repository does not currently use AI code review.
```

Those statements must be updated when your team begins using any of those tools.

Step 15: Review the changes before committing

Run:

```
git status
```

You should see:

```
modified: README.md
untracked: docs/git-methodology.md
untracked: docs/project-methodology.md
```

Review the README changes:

```
git diff README.md
```

Check that the methodology files exist:

```
ls docs
```

On Windows Command Prompt, use:

```
dir docs
```

Step 16: Create the commit

Stage only the relevant files:

```
git add README.md docs/git-methodology.md docs/project-methodology.md
```

Check the staged files:

```
git status
```

Commit using your agreed format:

```
git commit -m "docs: add development methodologies" -m "Refs #1"
```

Replace `#1` with the actual issue number.

Because these documents were created with AI assistance, the document declarations already record that usage. The `Assisted-by` commit footer is required when AI-generated **code** is included; this commit contains documentation rather than generated code.

Step 17: Push the branch

Push the new branch to Gitea:

```
git push -u origin docs/1-add-methodology-documents
```

This does not push directly to `main`. It creates the remote issue branch.

Step 18: Open the Pull Request

In Gitea:

1. Click **Pull Requests**.
2. Click **New Pull Request**.
3. Set:

```
Target branch: main
Source branch: docs/1-add-methodology-documents
```

Gitea Pull Requests are used to propose, review and merge changes from one branch into another. ([Gitea Documentation](#))

Use this title:

```
#1 Add development methodology documentation
```

Paste this description:

Purpose

Add the approved Git Methodology and Project Methodology documents to the repository.

Changes

- Added `docs/git-methodology.md`
- Added `docs/project-methodology.md`
- Added links to both documents in `README.md`
- Added the current repository AI-usage declaration

Testing

- Confirmed that both Markdown files render correctly
- Confirmed that the README links point to the correct files
- Confirmed that no unrelated files are included

Known Limitations

None.

Related Issue

Closes #1

Replace `#1` with the real issue number.

Create the Pull Request.

Step 19: Move the issue to `In Review`

Return to the Project board and move the issue:

In Progress → In Review

Do not move it to **Done** yet.

Step 20: Have another team member review it

The reviewer should:

1. Open the Pull Request.
2. View **Files Changed**.
3. Open both methodology files.
4. test the links in `README.md` ;
5. confirm there are no unrelated changes;
6. leave any comments;
7. approve the Pull Request.

If changes are requested:

1. make them on the same local branch;
2. commit them with another atomic commit;
3. push the branch again.

The existing Pull Request updates automatically when new commits are pushed to its source branch. ([Gitea Documentation](#))

Step 21: Merge it

After approval:

1. Ensure the target is `main`.
2. Select **Create a merge commit**.
3. Merge the Pull Request.
4. Delete the source branch when Gitea offers the option.

Because the Pull Request body contains:

Closes #1

Gitea should close the related issue after the Pull Request is merged, provided that closing keywords are enabled on your installation.

Move the issue to:

Done

Step 22: Update your local copy

Return to the terminal:

```
git switch main
git pull origin main
```

Delete the completed local branch:

```
git branch -d docs/1-add-methodology-documents
```

Check the final state:

```
git status
```

It should show:

```
On branch main
Your branch is up to date with 'origin/main'.
nothing to commit, working tree clean
```

Do **not** create the `sprint-1` tag yet. Create that only when the entire Sprint 1 submission has been completed, reviewed and merged.

Sources

[Protected branches](#) | [Gitea Documentation](#) docs.gitea.com

8/4/2026, 8:37:28 PM



in step 4, dont i need to say AI?

8/4/2026, 8:39:26 PM



Thinking

- **Clarifying AI attribution**

I'm interpreting the policy conservatively: if AI helped draft the issue, add a brief disclosure in the issue body. The commit footer applies specifically to commits containing AI-generated code, while repository documentation needs its own declaration.

- **Clarified AI attribution**

✓ Worked for 15s

Yes - because the **issue description was drafted with AI assistance**, the safest and most transparent approach is to add an AI declaration at the bottom of the issue.

Add this after the **Evidence** section:

AI Declaration

The preceding issue was planned and generated with the assistance of ChatGPT-Web[GPT-5.6 Thinking].

The course policy requires AI-assisted writing to identify the tool, model and purpose, but it does not require attribution on every individual sentence. [ai_policy_20_jul.pdf](#)

So your complete first issue should end like this:

Evidence

The linked branch, commits and merged Pull Request will provide evidence of completion.

AI Declaration

The preceding issue was planned and generated with the assistance of ChatGPT-Web[GPT-5.6 Thinking].

For this documentation task:

- **Issue:** include the AI declaration above.
- **Methodology documents:** they already include AI declarations.
- **Commit:** no `Assisted-by` footer is required unless AI-generated **code** is included.
- **README:** keep the repository AI-use or non-use statements updated. [ai_policy_20_jul.pdf](#)

8/4/2026, 8:39:39 PM



dbce711e-9657-4b28-a191-87...

step 8, wheres repo settings?

8/4/2026, 8:45:08 PM



Thinking



Searching the web

docs.gitea.com

• Searched 1 website

✓ Worked for 19s

The **Settings** tab is missing from your screenshot. It would normally appear after **Activity**. This most likely means your account has **Write** access but not **Admin** access to the repository.

Because the repository belongs to the organisation `git-push-pray`, an organisation owner or repository administrator needs to do one of these:

1. Give you **Admin** permission for `Sport-Analytics-Tool`; or
2. Configure the branch protection for you.

Only repository owners and administrators can manage protected-branch rules. With Admin permission, the path is:

Repository → Settings → Branches → Add New Rule

([Gitea Documentation](#))

Ask the repository owner:

Please grant me Admin access to the Sport-Analytics-Tool repository so that I can configure branch protection and the required Pull Request rules.

You can continue creating the issue, cloning the repository and creating the documentation branch while waiting. Just **do not push directly to** `main`. Once you have Admin access, refresh the repository page and the **Settings** tab should appear.

Sources

[Protected branches](#) | [Gitea Documentation](#) docs.gitea.com

8/4/2026, 8:45:11 PM





cffe7f1f-962f-4fd2-a85e-20e92...

are these namings etc correct?

8/4/2026, 8:52:38 PM



Almost correct. One filename does **not** match the link in your README.

Correct

```
docs/git-methodology.md
```

This matches:

```
[Git Methodology](docs/git-methodology.md)
```

Needs to be renamed

Your file is currently:

```
docs/Project_Methodology.md
```

But your README links to:

```
docs/project-methodology.md
```

Gitea normally treats filenames as case-sensitive, so that link may not work. Rename the file to:

```
docs/project-methodology.md
```

In the VS Code terminal:

```
git mv docs/Project_Methodology.md docs/project-methodology.md
```

Then check:

```
git status
```

Your final structure should be:

```
Sport-Analytics-Tool/  
├─ .gitignore  
├─ README.md  
└─ docs/  
    ├─ git-methodology.md  
    └─ project-methodology.md
```

Your README links are then correct:

```
## Documentation
```

```
Project documentation is stored in the [docs`](docs/) directory.
```



Development Methodologies

- [Git Methodology](docs/git-methodology.md)
- [Project Methodology](docs/project-methodology.md)

The **u** beside the two documents means they are **untracked**, and **M** beside `README.md` means it has been **modified**. That is expected before you stage and commit them.

8/4/2026, 8:52:49 PM



Purpose

Add the approved Git Methodology and Project Methodology documents to the repository.

Changes

- Added `docs/git-methodology.md`
- Added `docs/project-methodology.md`
- Added links to both documents in `README.md`
- Added the current repository AI-usage declaration

Testing

- Confirmed that both Markdown files render correctly
- Confirmed that the README links point to the correct files
- Confirmed that no unrelated files are included

Known Limitations

None.

Related Issue

Closes #1

do we need to add this message and the docs were made by ai??

8/4/2026, 8:57:10 PM



Thinking

- **Clarifying disclosure needs**

I'm distinguishing documentation disclosures from the pull-request description itself. Since this PR text is AI-assisted, I'd add a brief declaration there, while keeping the existing document notices and README code-usage statement unchanged.

- **Clarified disclosure needs**

- **Clarified AI assistance declarations for code and pull requests**

✓ Worked for 26s

Yes. Use that text as the **Pull Request description** because your Git methodology requires the PR to explain its purpose, changes, testing, limitations and related issue.

Because I drafted the PR description, add an AI declaration at the bottom. The two methodology documents already have their own AI declarations, so you do not need to repeat that they were AI-generated inside the **Changes** list. The course policy requires AI-assisted writing to identify the tool, model and purpose, while AI-generated **code** is attributed separately in the commit. `ai_policy_20_jul.pdf`

Use this:

Purpose

Add the approved Git Methodology and Project Methodology documents to the repository.

Changes



- Added `docs/git-methodology.md`
- Added `docs/project-methodology.md`
- Added links to both documents in `README.md`
- Confirmed that both methodology documents contain the required AI declarations

Testing

- Confirmed that both Markdown files render correctly in Gitea
- Confirmed that the README links open the correct files
- Confirmed that no unrelated files are included

Known Limitations

None.

Related Issue

Closes #1

AI Declaration

The preceding Pull Request description was planned and generated with the assistance of ChatGPT-Web[GPT-5.6 Thinking].

Only claim the Markdown files render and the links work **after you have pushed the branch and checked them in Gitea**.

Also, do not include:

- Added the current repository AI-usage declaration

unless you actually added an `## AI Usage` section to the README. AI-assisted documentation does **not** mean you used AI code generation. Your README may still correctly say:

This repository does not currently use AI code generation.

8/4/2026, 8:57:28 PM



can u assign "any 1 reviewer" or do u need to chose a specific person?

8/4/2026, 8:59:31 PM



Thinking



Searching the web

docs.gitea.com

- **Searched 1 website**

✓ Worked for 14s

You can configure the protected `main` branch to require **one approval from any eligible team member**. You do not have to permanently name one specific reviewer. In Gitea, reviews from users with write access count toward the required approval unless the repository restricts approvals to selected users or teams. ([Gitea Documentation](#))

For each Pull Request:

- **Branch protection:** set **Required approvals = 1**.
- **Reviewer field:** optional. Select a specific teammate only when you want to notify that person and ask them to review.
- The PR author should not approve their own work.

So your methodology wording can remain:

At least one team member other than the author must review and approve the Pull Request.

That means **any one eligible teammate**, not a permanently assigned person. Gitea supports maintainers approving, commenting on, or requesting changes to a Pull Request. ([Gitea Documentation](#))

Sources

[Protected branches | Gitea Documentation](#) docs.gitea.com

8/4/2026, 8:59:33 PM



but i cant do that if im not admin right?

8/4/2026, 9:00:14 PM



Thinking



Searching the web

docs.gitea.com

• Searched 1 website

✓ Worked for 7s

Correct. Without **Admin** permission, you cannot configure the rule that requires one approval before merging. Only repository owners or administrators can manage protected-branch rules. ([Gitea Documentation](https://docs.gitea.com))

You can still:

- create the issue and branch;
- push your branch;
- open the Pull Request;
- manually request a specific teammate to review it;
- wait for their approval before merging.

However, Gitea will not automatically block an unapproved merge unless an admin sets:

Protected branch: main

Required approvals: 1

Ask an admin to configure it, or continue following the rule manually until they do. Your methodology can still require one approval even before Gitea technically enforces it.

Sources

[Protected branches | Gitea Documentation](https://docs.gitea.com) docs.gitea.com

8/4/2026, 9:00:17 PM





bed4715d-1ec9-416a-9da3-40...

uhmm, why does it say 0 commits?

8/4/2026, 9:02:37 PM



Thinking

- **Diagnosing the pull request**

The pull request shows no commits because your branch likely has no new commit compared with `main`. I'm checking whether the files are committed locally and whether that branch has been pushed.

Run:

```
```bash
git branch --show-current
git status
git add README.md docs/git-methodology.md docs/project-methodology.md
git commit -m "docs: add methodology documents"
git push -u origin docs/2-add-methodology-documents
```
```

Then refresh the pull request. If Git says nothing is committed, check that the branch name matches exactly.

- **Diagnosed the pull request**

✓ Worked for 10s

It says **0 commits** because the branch `docs/2-add-methodology-documents` currently has no committed changes that differ from `main`. Most likely, the files are still only modified/untracked in VS Code and have not yet been committed and pushed.

In the VS Code terminal, run:

```
git branch --show-current
git status
```

Confirm that the current branch is:

```
docs/2-add-methodology-documents
```

Then stage the files:

```
git add README.md docs/git-methodology.md docs/project-methodology.md
```

Commit them, using the **actual related issue number**:

```
git commit -m "docs: add development methodologies" -m "Refs #2"
```

Then push the branch:

```
git push -u origin docs/2-add-methodology-documents
```

Refresh the Pull Request page. It should then show:

- **1 commit**
- the three changed files under **Files Changed**



Before committing, make sure the filename is exactly:

```
docs/project-methodology.md
```

Your earlier screenshot showed `Project_Methodology.md` , which would not match the README link.

Also, the Pull Request heading appears to contain an extra `#4` at the end:

```
#2 Add development methodology documentation #4
```

The first `#2` is the Pull Request number added automatically by Gitea. Edit the title and remove the trailing `#4` unless it was intentionally referencing issue `#4` .

8/4/2026, 9:02:50 PM



ohh, someone merged it before i had the change to write it.. s how should i close this then?

8/4/2026, 9:03:34 PM



Thinking

- **Handling the duplicate pull request**

I'm inferring the documentation is already merged elsewhere. I'd verify it appears on `main`, then close this pull request as superseded, leave a brief explanatory comment, and delete the unused branch if Gitea offers that option.

✓ Worked for 9s

If the methodology files are already on `main`, this Pull Request is now redundant.

First, check **Code** → **main** and confirm that these exist:

```
docs/git-methodology.md
docs/project-methodology.md
```

Then, on the empty Pull Request:

1. Add this comment:

```
Closing this Pull Request because the methodology documentation was merged into `main` through another Pull Request before this branch contained any additional commits. No further changes are required.
```

2. Click **Close Pull Request** — do not merge it.

3. Delete the branch `docs/2-add-methodology-documents` if Gitea offers that option.

4. Check the related issue:

- If the methodology work is complete, ensure the issue is closed and moved to **Done**.
- If the earlier merge did not link to the issue, add a comment linking the merged Pull Request as evidence.

This is not a failure in the workflow. It simply records that the PR was superseded rather than leaving an empty PR open.

8/4/2026, 9:03:46 PM





oyyy, the git methodology was empty, how to follow rules and reupload that doc?

8/4/2026, 9:05:28 PM



Do **not** edit `main` directly and do not reopen the old empty Pull Request. Because the earlier change was already merged, treat the missing content as a new documentation defect and follow the full workflow again.

1. Create a new issue

Title

Restore missing Git methodology content

Description

Description

Replace the empty ``docs/git-methodology.md`` file with the approved Git Methodology document.

Motivation

The Git Methodology file was merged into ``main`` without its intended content. The repository must contain the complete methodology so that the team can follow and demonstrate its agreed Git process.

Acceptance Criteria

- [] ``docs/git-methodology.md`` contains the complete approved methodology
- [] The document renders correctly in Gitea
- [] The document contains its AI declaration
- [] The README link opens the correct document
- [] The change is reviewed through a Pull Request
- [] The Pull Request is merged into ``main``

Dependencies

None.

Evidence

The linked branch, commit and merged Pull Request will provide evidence of completion.

AI Declaration

The preceding issue was planned and generated with the assistance of ChatGPT-Web[GPT-5.6 Thinking].

Assign it to:

Milestone: Sprint 1

Project: Sport-Analytics-Tool-Proj

Labels:

- type: documentation
- area: documentation

Move it to **Ready**, then **In Progress** when you begin.

Write down its issue number. I will use `#3` below only as an example.

2. Download the complete document

Download the complete [Git Methodology](#).

You need to replace the empty repository file with this document.

3. Update your local `main`

In the VS Code terminal:

```
git switch main
git pull origin main
```

This ensures your new branch includes the version that someone already merged.

4. Create a new branch

Replace `3` with the real issue number:

```
git switch -c docs/3-restore-git-methodology
```

Do not reuse the old branch.

5. Replace the empty file

Copy the downloaded document into:

```
docs/git-methodology.md
```

It must replace the existing empty file.

Your structure should remain:

```
docs/
├─ git-methodology.md
└─ project-methodology.md
```

Be careful that the filename is exactly lowercase and uses hyphens:

```
git-methodology.md
```

6. Verify the file before committing

Open `docs/git-methodology.md` in VS Code and confirm that it begins with:

```
# Git Methodology
```

Then run:

```
git status
git diff -- docs/git-methodology.md
```

The diff should show a large amount of added content. If it shows no difference, the replacement did not work.

Also check that the README link remains:

```
[Git Methodology](docs/git-methodology.md)
```

7. Commit the correction

Replace `#3` with the real issue number:

```
git add docs/git-methodology.md
git status
git commit -m "docs: restore Git methodology content" -m "Refs #3"
```

The `git status` check should show only the intended methodology file staged.

8. Push the new branch

```
git push -u origin docs/3-restore-git-methodology
```

9. Open a new Pull Request

Target

main

Source

docs/3-restore-git-methodology

Title

#3 Restore Git methodology content

Description

Purpose

Restore the approved Git Methodology content after the previous version was merged as an empty file.

Changes

- Replaced the empty `docs/git-methodology.md` file with the complete approved document
- Confirmed that the document contains the required AI declaration
- Confirmed that the existing README link points to the correct file

Testing

- Confirmed that the Markdown file contains the complete methodology
- Confirmed that the document renders correctly in Gitea
- Confirmed that the README link opens the correct document
- Confirmed that no unrelated files are included

Known Limitations

None.

Related Issue

Closes #3

AI Declaration

The preceding Pull Request description was planned and generated with the assistance of ChatGPT-Web[GPT-5.6 Thinking].

Only say that it renders correctly and the link works after checking the pushed branch in Gitea.

Move the issue to **In Review**.

10. Review and merge

Ask another team member to:

1. Open **Files Changed**.
2. Confirm the file is no longer empty.
3. Read enough of the document to confirm it is complete.
4. Test the README link.
5. Approve the Pull Request.

Then merge using a **merge commit**, delete the branch and move the issue to **Done**.

Afterwards, update locally:

```
git switch main
git pull origin main
git branch -d docs/3-restore-git-methodology
```

This creates a clear record that the first merge contained a defect and that the defect was corrected properly through a new issue, branch and Pull Request.

8/4/2026, 9:05:42 PM



ok, now dean made this layout, can u access from the link for gitea or not? b4 i ask my q
<https://sdp.ms.wits.ac.za/git-push-pray/Sport-Analytics-Tool>

8/5/2026, 11:21:47 AM



Thinking



Searching the web

- Searching the web

✓ Worked for 9s

I tried the link, but I cannot access the repository from here; it returns a 404, likely because it is private or requires Wits/Gitea login. ([Invalid URL](#))

Send a screenshot of Dean's layout, paste the folder structure, or upload a ZIP of the repo, and I can review it properly.

Sources

[Internal Error](#)

8/5/2026, 11:21:49 AM



Sport-Analytics-Tool.zip

1. are there clashes with the things we pushed and this?
2. using spruint 1 rubric etc, what are 6 (1 for each person) issues we can create now that ppl can do for setup etc so we ready to start after that

8/5/2026, 11:24:21 AM



Thinking



Remembering

- **Reviewed repository conventions and aligned Sprint 1 setup issues**
- **Reviewing repository consistency**
I'm comparing the commit and working tree for meaningful conflicts. The main issue is that the issue templates don't fully match the methodology's required fields, while the repository structure and documentation naming appear broadly consistent.
- **Reviewed repository consistency**
- **Planning issue assignments**
I'm balancing six setup issues across governance, CI, documentation, onboarding, requirements, architecture, and the domain data foundation. Each task should produce a concrete deliverable and leave implementation ready to begin.
- **Planned issue assignments**

✓ Worked for 4m 49s

I inspected the ZIP, including the current files and Git history. The structure Dean created is generally strong and does **not** conflict with the methodologies. It already gives you separate frontend and backend applications, CI, testing, MkDocs, architecture documentation and evidence folders, which match the project-wide requirements. `project_briefs.pdf`

1. Clashes and inconsistencies to fix

No major structural clash

The monorepo structure is compatible with your methodology:

```
apps/frontend
apps/backend
packages/contracts
database
docs
evidence
infra
tests
```

The frontend and backend are separate applications communicating through HTTP, so the repository is not monolithic merely because both applications are stored in one repository.

Items requiring correction

| Item | Finding | Recommended action |
|-----------------------------|---|--|
| Project methodology version | <code>docs/project_methodology.md</code> contains the final Sprint Close-Out process, but its header says Version 1.0 , not 1.1 | Correct the document version to <code>1.1</code> |



| Item | Finding | Recommended action |
|---------------------------|--|--|
| Filename convention | The repository uses <code>project_methodology.md</code> , whereas we originally discussed <code>project-methodology.md</code> | Keep the underscore version because all current links consistently use it |
| Issue templates | The feature and bug templates do not fully contain the fields required by your Project Methodology, particularly Motivation, Dependencies and Evidence | Update both templates |
| Initial scaffold commit | Dean's scaffold appears as a direct commit to <code>main</code> , rather than an issue branch and Pull Request, and it does not reference an issue | Do not rewrite history; record it as an initial setup deviation and follow the methodology from now on |
| Branch protection | The ZIP cannot show whether <code>main</code> is protected | An admin must enable Pull Request merging and one required approval |
| Reproducible installation | There is no <code>package-lock.json</code> , and CI currently uses <code>npm install</code> | Run a verified installation, commit the lockfile and change CI to <code>npm ci</code> |
| Line endings | In the uploaded copy, Git reports almost every file as modified, but the differences are only line endings | Add <code>.gitattributes</code> and do not commit mass line-ending changes |
| AI usage | Dean's commit contains an <code>Assisted-by</code> footer, the README declares AI code generation and an AI register exists | This is aligned, but the generated scaffold still needs to be tested and reviewed |

The repository is therefore a **good foundation**, but it should not yet be treated as fully verified or complete.

Sprint 1 specifically focuses on project setup, eliminating major blockers and establishing a strong direction. The rubric then rewards version control, a public documentation site, development guides, an actively used tracker, methodologies, a motivated technology stack, stakeholder interaction, initial design and supporting implementation. `project_briefs.pdf`

Six Sprint 1 issues

Assign each issue to one person. Add all six to:

```
Milestone: Sprint 1
Project: Sport-Analytics-Tool-Proj
Column: Ready
```

Each person should create their own branch, commits and Pull Request so that all six members have visible contributions.

Issue 1: Align repository governance with the methodologies

Suggested labels

```
type: infrastructure
area: infrastructure
priority: high
```

Suggested branch

```
chore/<issue-number>-align-repository-governance
```

```
## Description
```

Align the repository configuration, Gitea templates and methodology documents with the approved Git and Project Methodologies.

Motivation

The initial repository scaffold is generally compatible with the approved methodologies, but several inconsistencies must be corrected before the team begins normal development.

Acceptance Criteria

- [] `docs/project\_methodology.md` identifies the document as Version 1.1
- [] The feature issue template includes Description or Scope, Motivation, Acceptance Criteria, Dependencies and Evidence
- [] The bug issue template includes Motivation or Impact, Acceptance Criteria, Dependencies and Evidence
- [] A `.gitattributes` file enforces consistent LF line endings
- [] No mass line-ending-only changes are included in the Pull Request
- [] The required Gitea labels exist
- [] The Project board contains Backlog, Ready, In Progress, In Review, Blocked and Done
- [] An administrator has been asked to protect `main` and require one approval
- [] The initial scaffold workflow deviation is documented without changing existing Git history
- [] Relevant documentation is updated

Dependencies

None.

Evidence

- Updated templates and methodology document
- `.gitattributes`
- Screenshot or confirmation of branch-protection settings
- Linked Pull Request

The deviation record can briefly state that the initial scaffold was committed before the workflow was fully enforced and that all future work will follow the documented methodology.

Issue 2: Establish a reproducible installation and green CI pipeline

Suggested labels

```
type: infrastructure
area: infrastructure
priority: high
```

Suggested branch

```
ci/<issue-number>-verify-install-and-ci
```

Description

Verify the generated project scaffold from a clean installation and establish a reproducible CI process.

Motivation

The repository contains frontend, backend, shared-contract and CI scaffolding, but it has not yet been fully verified. The repository also does not contain a package lockfile.

Acceptance Criteria

- [] The repository installs successfully from a clean clone
- [] A root `package-lock.json` is generated and committed
- [] `npm run check` passes locally
- [] Frontend tests pass
- [] Backend health-endpoint tests pass
- [] Shared-contract checks pass
- [] Frontend and backend production builds succeed
- [] CI uses `npm ci` instead of `npm install`
- [] The Gitea CI workflow completes successfully on the Pull Request
- [] Any generated-code errors discovered during verification are corrected
- [] Verification results are recorded in the Pull Request

Dependencies

Issue 1 should preferably be completed first, but work may begin independently.

Evidence

- Committed `package-lock.json`
- Successful local-check output
- Successful Gitea Actions run
- Linked Pull Request

This issue is particularly important because Dean's AI register currently states that verification was pending.

Issue 3: Deploy the public documentation website

Suggested labels

```
type: documentation
area: documentation
priority: high
```

Suggested branch

```
docs/<issue-number>-deploy-documentation-site
```

Description

Build, verify and deploy the MkDocs documentation website to a publicly accessible location.

Motivation

The repository contains an MkDocs configuration and substantial documentation, but the Sprint 1 rubric requires an accessible, well-designed documentation website rather than only local Markdown files.

Acceptance Criteria

- [] `mkdocs build --strict` completes successfully
- [] All navigation links work
- [] The Git and Project Methodology pages are accessible
- [] Architecture, API, database, setup, testing, security and AI-use pages are accessible
- [] The documentation website is deployed
- [] The deployed site is accessible without a Gitea or Wits login
- [] The public documentation URL is added to the root README
- [] Deployment instructions are documented
- [] The site clearly distinguishes planned functionality from implemented functionality
- [] No secrets or private information are published

Dependencies

Issue 1 should correct the methodology version before final deployment.

Evidence

- Public documentation URL
- Successful MkDocs build
- Screenshot or deployment log
- Linked Pull Request

The brief expressly requires a publicly available documentation website. `project_briefs.pdf`

Issue 4: Validate developer onboarding and motivate the technology stack

Suggested labels

type: documentation
area: documentation

Suggested branch

docs/<issue-number>-complete-onboarding-and-tech-stack

Description

Validate the development setup from a clean clone and complete the developer guides and technology-stack motivation.

Motivation

New team members must be able to install, configure, run, test and understand the project without relying on undocumented assistance. The Sprint 1 rubric also requires the technology stack to be fully listed and motivated.

Acceptance Criteria

- [] The setup guide is tested from a clean clone
- [] All prerequisite versions are accurate
- [] Frontend, backend and documentation startup commands are verified
- [] Every environment variable is documented
- [] Common setup failures and solutions are documented
- [] `apps/frontend/README.md` exists
- [] `apps/backend/README.md` exists
- [] The shared-contract package documentation is verified
- [] A technology-stack document lists every major technology
- [] Each major technology has a purpose and motivation
- [] Important alternatives considered are recorded where relevant
- [] The new documentation is linked from MkDocs and the root README

Dependencies

Issue 2 should confirm the final installation commands before this issue is closed.

Evidence

- Updated development guide
- Component READMEs
- Technology-stack document
- Clean-clone verification notes
- Linked Pull Request

Issue 5: Confirm stakeholder requirements and create the product backlog

Suggested labels

```
type: research
area: documentation
priority: high
```

Suggested branch

```
docs/<issue-number>-define-requirements-and-backlog
```

Description

Prepare for and conduct the initial stakeholder requirements meeting, then convert the confirmed requirements into documented requirements and Gitea backlog issues.

Motivation

The sport, competition context, event vocabulary, important statistics, users and submission responsibilities must be confirmed before the team can finalise the data model or core features.

Acceptance Criteria

- [] The chosen sport and professional league or tour are confirmed
- [] Primary user roles are identified
- [] Approved submitters and their submission scope are defined
- [] Important fixture events are identified
- [] Required derived statistics are identified
- [] Basic, Intermediate and Advanced requirements are separated
- [] Project-wide requirements are included
- [] Unresolved stakeholder questions are recorded
- [] The stakeholder meeting is documented under `evidence/stakeholder-meetings/`
- [] A requirements document is added to the documentation site
- [] Each confirmed requirement is linked to one or more Gitea issues
- [] Initial backlog issues are prioritised and assigned to suitable milestones
- [] Decisions not to implement requested work are motivated

Dependencies

None.

Evidence

- Stakeholder meeting record
- Requirements document
- Requirement-to-issue traceability
- Populated Gitea backlog
- Linked Pull Request

This is essential because the event types and statistics depend on the selected sport. The project brief makes event-derived statistics, validated submissions, traceability and the API the core of the product. `sport_analytics.pdf`

Issue 6: Design and prove the core event-data flow

Suggested labels

```
type: feature
area: data
```

priority: high

Suggested branch

feat/<issue-number>-prototype-event-data-flow

Description

Create the initial product design package and implement a small proof of concept for the core event-data flow.

Motivation

The largest technical risk is whether ordered event data can be validated and used to derive trustworthy statistics. This must be proved early before the team builds authentication, submission workflows and larger APIs around the design.

Acceptance Criteria

- [] A system-context or container diagram is completed
- [] An initial database ERD is completed
- [] The main API resources and endpoint groups are documented
- [] Low-fidelity wireframes are created for the main user journeys
- [] A development roadmap maps major work to the remaining sprints
- [] The chosen sport's first event schema is implemented in `packages/contracts`
- [] At least two representative event types are supported
- [] A backend endpoint validates a sample fixture-event payload
- [] Invalid submissions return actionable validation errors
- [] At least one statistic is derived from sample event records rather than entered as a total
- [] Automated tests cover valid and invalid event payloads
- [] The design and API documentation are updated
- [] Any assumptions awaiting stakeholder approval are clearly identified

Dependencies

The sport and event vocabulary must be confirmed through Issue 5 before the sport-specific implementation is finalised.

Evidence

- Diagrams and wireframes
- Roadmap
- Event schema
- Validation endpoint
- Derived-statistic proof of concept
- Automated tests
- Linked Pull Request

Keep the proof of concept deliberately small: one fixture, a few ordered event types and one derived statistic. Its purpose is to remove the biggest technical blocker, not to implement the complete submission system.

AI declaration for all six issues

Add this at the bottom of each issue description:

AI Declaration

The preceding issue was planned and generated with the assistance of ChatGPT-Web[GPT-5.6 Thinking].

These six issues collectively cover the Sprint 1 rubric without assigning six people to duplicate Dean's scaffold:

| Issue | Main rubric coverage |
|-------|--|
| 1 | Version Control, Work Tracker, Git and Project Methodology |
| 2 | Code-quality tools, CI and supporting Implementation |
| 3 | Documentation Site |
| 4 | Getting Started Guides and Tech Stack |
| 5 | Stakeholder Interaction and Project Methodology evidence |
| 6 | Initial Design, Development Plan and Implementation |

Do not commit the line-ending changes currently visible in the uploaded ZIP as though they were real edits. Issue 1 should normalise that first.

8/5/2026, 11:24:23 AM



we havent discussed actual project flow yet, so prefer not that, rather things like this? we can start
OAuth
Azure
Cloudflare
supabase
and the correction to the base ofc, maybe 1-4 of what u spoke about and then stuff like this? i can do the fixing to fit methodology now and then create 6 iissues after for everyone to do maybe?
i dont have admin access yet, waiting for that
maybe
make an issue now for me to:
doc initial commit, fix tempelates or whatever dean did that doesnt align, and then after we have fixed all that, we can create 6 issues, 1 for everyone?

8/5/2026, 11:34:38 AM



Thinking

- 🌐 **Inspecting Uploaded Repository Files**
 - **Aligning repository governance**

I'm drafting one cleanup issue first: reconcile templates and process documentation with the approved methodology, document any intentional exceptions, and verify the result. I'll keep environment setup as separate follow-up work after stakeholder review.
 - **Aligned repository governance**
 - 🌐 **Reviewing Initial Commit and Issue Alignment**
 - **Planning repository work**

I'm agreeing with that sequence: I'll create one assigned governance-alignment issue now, verify it against the approved brief and repository state, then wait for project-flow discussion before drafting six environment/setup issues.
 - **Planned repository work**
 - **Inspected repository files, templates, history, and issue alignment**
 - **Planned repository governance and infrastructure setup**
- ✓ Worked for 1m 4s

8/5/2026, 11:34:40 AM



we havent discussed actual project flow yet, so prefer not that, rather things like this? we can start
OAuth
Azure
Cloudflare
supabase
and the correction to the base ofc, maybe 1-4 of what u spoke about and then stuff like this? i can do the fixing to fit methodology now and then create 6 iusses after for everyone to do maybe?
i dont have admin access yet, waiting for that
maybe
make an issue now for me to:
doc initial commit, fix tempelates or whatever dean did that doesnt align, and then after we have fixed all that, we can create 6 issues, 1 for everyone?
also note dean didnt create an issue, so we must document that tooo i guess

8/5/2026, 11:35:43 AM



Thinking



Inspecting sport analytics repository files

- Inspected sport analytics repository files, templates, history, and documentation
- Planned six research-ready setup and development issues



Inspecting ADR template and documentation navigation

- Inspected ADR template and documentation navigation

✓ Worked for 2m 25s

Yes - **fix the repository foundation first**, then create the six parallel setup issues.

That order is cleaner because everyone's later issues will then use the corrected templates, naming rules and evidence structure. Your methodology requires meaningful work to begin with an issue containing a description, motivation, acceptance criteria, dependencies and evidence, so it makes sense to align the templates before six people start working.

Project_Methodology_v1.1.md

Dean's initial scaffold should **not be deleted, reverted or rewritten**. Record it transparently as a one-time **pre-methodology setup deviation**:

- the scaffold was committed before the issue and Pull Request workflow was fully active;
- it was committed directly to `main`;
- no linked issue was created;
- branch protection had not yet been configured;
- from this correction onward, all work will follow the approved methodology.

Create this issue now

Issue title

Align repository governance and document initial setup deviation

Issue description

Description

Align the repository governance files with the approved Git Methodology and Project Methodology.

Document the initial repository scaffold as a one-time setup deviation because it was committed directly to `main` before the issue, branch and Pull Request workflow was fully established.

Motivation

The repository scaffold provides a useful project foundation. However, the initial scaffold commit was created without a linked Gitea issue or Pull Request, and branch protection had not yet been configured.

The existing Gitea issue and Pull Request templates also do not fully reflect the fields and processes required by the approved methodologies.

The existing Git history will not be rewritten. Instead, the deviation will be documented transparently and the repository governance files will be corrected so that all future work follows the agreed process consistently.

Scope

This issue covers repository governance and methodology alignment only.

It does not include:

- product features;
- final authentication decisions;
- database implementation;
- deployment implementation;
- event modelling; or
- decisions about the detailed user and project flow.

Acceptance Criteria

- [] A decision record is added at `evidence/decisions/ADR-001-initial-repository-setup.md`
- [] The decision record identifies the initial scaffold commit and explains that it was created without a linked issue or Pull Request
- [] The decision record explains that the existing Git history will not be rewritten
- [] The decision record identifies this as a one-time pre-methodology setup deviation
- [] The decision record states that all future significant work must follow the issue, branch, commit, Pull Request and review process
- [] The pending branch-protection requirement is recorded
- [] `docs/project\_methodology.md` displays Document Version `1.1`
- [] The feature issue template includes Description or Scope, Motivation, Acceptance Criteria, Dependencies, Evidence and an AI Declaration section
- [] The bug issue template includes Summary, Motivation or Impact, Reproduction Information, Acceptance Criteria, Dependencies, Evidence and an AI Declaration section
- [] A general task template is added for documentation, infrastructure and research work
- [] The Pull Request template includes Purpose, Changes, Testing or Verification, Known Limitations, Related Issue and an AI Declaration section
- [] The Pull Request template retains an appropriate review checklist
- [] A `.gitattributes` file is added to enforce consistent line endings
- [] No mass line-ending-only changes are included in the Pull Request
- [] All links to the Git and Project Methodology documents use the correct existing filenames
- [] A separate follow-up issue is created for protecting `main` once administrator access is available
- [] The follow-up branch-protection issue is placed in `Blocked` while administrator access is pending
- [] No unrelated product or feature changes are included
- [] The changes are reviewed through a Pull Request
- [] The Pull Request is merged into `main`

Dependencies

None.

Administrator access is required for the separate branch-protection issue, but it is not required to complete the documentation and template changes in this issue.

Evidence

Evidence will include:

- `evidence/decisions/ADR-001-initial-repository-setup.md`
- the updated Gitea issue templates
- the updated Pull Request template
- the corrected methodology document
- the new `.gitattributes` file
- the linked branch and commits
- the reviewed and merged Pull Request
- the separate blocked branch-protection issue

AI Declaration

The preceding issue was planned and generated with the assistance of ChatGPT-Web[GPT-5.6 Thinking].

Assign it like this

Assignee: You
Milestone: Sprint 1
Project: Sport-Analytics-Tool-Proj
Column: Ready
Labels:
- type: infrastructure
- area: documentation
- priority: high

When you begin, move it to **In Progress**.

Branch name

Replace the number with the actual issue number:

```
chore/<issue-number>-align-repository-governance
```

Example:

```
chore/7-align-repository-governance
```

Recommended commits

Keep the work understandable with two or three atomic commits:

```
docs: record initial repository setup deviation
```

```
Refs #7
```

```
chore(gitea): align issue and pull request templates
```

```
Refs #7
```

chore: enforce consistent line endings

Refs #7

Use the real issue number.

Separate blocked admin issue

Create this as part of the acceptance criteria, but do not attempt it until you receive Admin access.

Title

Protect main and require Pull Request approval

Status

Project column: Blocked
Milestone: Sprint 1
Blocked by: Pending repository Admin access

That keeps the missing permission visible rather than allowing it to disappear informally.

Six setup issues after this correction is merged

Your proposed direction is sensible. These six do not require you to have finalised the detailed product flow:

1. **Verify the workspace and establish a green CI pipeline**
2. **Deploy the public MkDocs site using Cloudflare Pages**
3. **Validate developer onboarding and document the motivated technology stack**
4. **Investigate and establish the OAuth authentication foundation**
5. **Investigate and establish the Supabase PostgreSQL foundation**
6. **Establish the Azure application deployment foundation**

For OAuth, Supabase and Azure, initially frame the work as **technical investigation plus safe foundation setup**, not full product implementation. That allows each person to remove a technical blocker without deciding the final user journeys prematurely.

One important boundary: Supabase may be investigated as the PostgreSQL database service, but your frontend must not use Supabase-generated data endpoints as the project API. Your hand-written backend API must remain between the frontend and database. The project brief specifically prohibits relying on generated API endpoints and requires the team to design and implement its own API. [project_briefs.pdf](#)

A sensible split would be:

| Person | Setup responsibility |
|--------|--|
| 1 | CI and reproducible installation |
| 2 | Cloudflare public documentation |
| 3 | Developer onboarding and technology stack |
| 4 | OAuth investigation and authentication foundation |
| 5 | Supabase PostgreSQL investigation and backend connection |
| 6 | Azure deployment foundation |

Complete and merge the governance issue first. Then the six new issues can all be created using the corrected general task template.

8/5/2026, 11:35:44 AM



ok, its issue #7, ive made the issue, now give me step by stepp to make it happen.
note: im not changing version to 1.1

8/5/2026, 11:45:23 AM



Thinking

Inspecting repository for issue implementation

- **Planning repository instructions**

I'm preparing a concise, file-specific workflow: create a branch, verify the working tree, add line-ending rules, update templates, and address issue #7 without changing the project version.

- **Planned repository instructions**

- **Documenting governance changes**

I'm treating the scaffold commit as the main exception to record, while keeping the ADR neutral. I'll also add issue and pull-request templates with acceptance criteria, validation steps, and an AI-declaration section.

- **Documented governance changes**

- **Inspected repository structure, templates, history, governance, and line endings**

✓ Worked for 2m 4s

Yes. Keep **Project Methodology Version 1.0**. Do not edit or rename `docs/project_methodology.md`.

Follow these steps for **Issue #7**.

Part 1: Correct Issue #7

Open Issue #7 and remove this acceptance criterion if it is still present:

```
- [ ] `docs/project_methodology.md` displays Document Version `1.1`
```

Replace it with:

```
- [ ] The existing methodology documents and filenames remain unchanged
```

Assign Issue #7:

```
Milestone: Sprint 1
Project: Sport-Analytics-Tool-Proj
Column: In Progress
Labels:
- type: infrastructure
- area: documentation
- priority: high
```

Part 2: Create the blocked branch-protection issue

Create this now because Issue #7 requires the pending administration work to be formally tracked.

Issue title

```
Protect main and require Pull Request approval
```

Issue description

Description

Configure branch protection for `main` so that project changes must be reviewed through Pull Requests.

Motivation

The approved Git Methodology prohibits direct development on `main` and requires at least one team member other than the author to approve a Pull Request before it is merged.

The current user does not yet have repository administrator access, so the configuration cannot be completed immediately.

Acceptance Criteria

- [] `main` is configured as a protected branch
- [] Direct pushes to `main` are restricted
- [] Force pushes to `main` are prohibited
- [] Changes must be merged through Pull Requests
- [] At least one approval is required before merging
- [] Pull Request authors cannot approve their own work, where supported
- [] Branch-protection settings are documented with screenshot evidence
- [] The configuration is checked against the approved Git Methodology

Dependencies

Repository administrator access is required.

Blocker

****Blocked by:**** Repository administrator access has not yet been granted.

****Required action:**** An organisation or repository administrator must grant access or configure the protected branch.

****Responsible person:**** Repository administrator

****Review date:**** At the next Thursday standup

Evidence

Evidence will include a screenshot of the protected-branch settings and the linked configuration activity.

AI Declaration

The preceding issue was planned and generated with the assistance of ChatGPT-Web[GPT-5.6 Thinking].

Assign it to:

Milestone: Sprint 1

Project: Sport-Analytics-Tool-Proj

Column: Blocked

Labels:

- type: infrastructure
- area: infrastructure
- status: blocked
- priority: high

Record the issue number. It will probably be **#8**, but use the actual number Gitea gives you.

Part 3: Prepare your local repository

Open the repository in VS Code and open the terminal.

Step 1: Check your current work

Run:

```
git status
git branch --show-current
```

Before continuing, your working tree should be clean.

You should see something similar to:

```
On branch main
nothing to commit, working tree clean
```

If many files appear as modified, **do not stage, discard or reset anything yet**. That may be the line-ending problem we are trying to prevent.

Step 2: Update `main`

```
git switch main
git pull origin main
```

Run another check:

```
git status
```

Step 3: Create the Issue #7 branch

```
git switch -c chore/7-align-repository-governance
```

Confirm it:

```
git branch --show-current
```

It should return:

```
chore/7-align-repository-governance
```

Part 4: Document the initial setup deviation

Create this file:

```
evidence/decisions/ADR-001-initial-repository-setup.md
```

Paste:

```
# ADR-001: Initial Repository Setup Before Governance Enforcement

- **Status:** Accepted
- **Date:** 2026-08-05
- **Participants:** Git Push Pray
- **Related issues:** #7, #8
```

```
## Context
```

The repository was initially created before the approved Git Methodology and Project Methodology were fully operational.

The repository was subsequently expanded through commit `449095d`, titled:

```
```text
chore(repo): establish initial project structure
```

This commit established the frontend, backend, shared contracts, database structure, documentation, CI workflow, testing folders, deployment planning and evidence structure.

The scaffold was committed directly to `main` without:

- a preceding Gitea issue;
- a dedicated issue branch;
- a Pull Request; or
- a recorded peer review.

Branch protection was also not yet configured because the required repository administrator access was unavailable.

The commit did contain an AI-assistance declaration. The scaffold provides a useful project foundation and should not be discarded.

## Decision

The team will not delete, revert or rewrite the existing Git history.

The initial repository setup and scaffold will be treated as a one-time pre-methodology setup deviation.

From Issue #7 onward, all significant project work must follow the approved process:

```
Gitea issue → branch → atomic commits → Pull Request → peer review → merge
```

Repository governance files will be aligned with the approved methodologies before the team begins parallel setup work.

Branch protection is tracked separately in Issue #8 and will remain blocked until repository administrator access becomes available.

## Alternatives Considered

### Rewrite the existing Git history

This option was rejected because rewriting shared history could remove useful evidence, disrupt other team members and create unnecessary risk.

### Revert the scaffold and recreate it through an issue and Pull Request

This option was rejected because the scaffold already provides a useful foundation. Recreating the same files would add work without improving the product.

### Document the deviation and apply the methodology prospectively

This option was selected because it preserves an honest project history while ensuring that future work follows the approved process consistently.

## Advantages

- Existing work and evidence are preserved.
- The repository history remains accurate.
- The deviation is documented transparently.
- The team can begin following the methodology immediately.
- Future work will provide clear traceability and review evidence.

## Disadvantages

- The initial scaffold does not provide complete evidence of the approved workflow.
- Branch protection remains unenforced until administrator access is available.

## Consequences

- Commit `449095d` remains part of the repository history.
- The initial scaffold will not be treated as evidence of full methodology compliance.
- All future significant work must begin with a Gitea issue.
- All future changes to `main` must be reviewed through a Pull Request.
- Issue #8 must be completed once administrator access is granted.
- Any further methodology deviation must be documented and justified.

## Verification and Review Date

This decision will be reviewed during the Sprint 1 close-out to confirm that:

- Issue #7 was completed through the approved workflow;
- later work used linked issues, branches and Pull Requests; and
- branch protection was configured or remains visibly blocked.

## AI Declaration

The preceding document was planned and generated with the assistance of ChatGPT-Web[GPT-5.6 Thinking].

```
Change `#8` if the branch-protection issue received a different number.
```

```

```

```
Part 5: Correct the feature issue template
```

```
Open:
```

```
```text
.gitea/ISSUE_TEMPLATE/feature.md
```

Replace its entire contents with:

```
---
name: Feature
about: Implement a user-visible or platform capability
---
```

```
## Description
```

```
Describe the feature and the result that must be achieved.
```

```
## Motivation
```

```
Explain why the feature is required and what project or user need it addresses.
```

```
## Scope
```

```
### Included
```

```
-
```

```
### Excluded
```

```
-  
  
## Acceptance Criteria  
  
- [ ]  
  
## Dependencies  
  
List any issues, decisions, services or stakeholder information required before this work can be completed.  
  
## Testing and Verification  
  
Explain how the feature will be tested or verified.  
  
## Documentation and Evidence  
  
List the documentation, screenshots, test results or other evidence that must be produced.  
  
## Security, Accessibility and Responsiveness  
  
Describe any relevant security, accessibility or responsive-design considerations.  
  
## AI Declaration  
  
State the AI usage or non-usage declaration for this issue.  
  
Example:  
  
> The preceding issue was planned and generated with the assistance of ChatGPT-Web[GPT-5.6 Thinking].
```

Part 6: Correct the bug issue template

Open:

```
.gitea/ISSUE_TEMPLATE/bug.md
```

Replace its entire contents with:

```
---  
name: Bug  
about: Record incorrect behaviour with reproducible evidence  
---  
  
## Summary  
  
Briefly describe the incorrect behaviour.  
  
## Motivation and Impact  
  
Explain why the defect must be corrected and who or what is affected.  
  
## Environment  
  
- **Application or service:**  
- **Environment:**  
- **Browser or client:**  
- **Version or commit:**
```

Steps to Reproduce

- 1.
- 2.
- 3.

Expected Behaviour

Describe what should happen.

Actual Behaviour

Describe what currently happens.

Severity

Select one:

- ☐ Critical
- ☐ High
- ☐ Medium
- ☐ Low

Acceptance Criteria

- ☐ The issue no longer reproduces
- ☐ A regression test has been added where practical
- ☐ Existing tests continue to pass
- ☐ Relevant documentation is updated
- ☐ No unrelated behaviour has been changed

Dependencies

List any related issues, services or decisions.

Evidence

Add screenshots, logs, test output or reproduction examples.

AI Declaration

State the AI usage or non-usage declaration for this issue.

Example:

> The preceding issue was planned and generated with the assistance of ChatGPT-Web[GPT-5.6 Thinking].

Part 7: Add a general task template

Create:

```
.gitea/ISSUE_TEMPLATE/task.md
```

Paste:

```
---
name: Task
about: Record documentation, infrastructure, research or general project work
---
```

Description

Explain what must be completed.

Motivation

Explain why the work is required.

Scope

Included

-

Excluded

-

Acceptance Criteria

- []

Dependencies

List related issues, services, decisions or required access.

Evidence

List the documents, configuration, screenshots, logs, test results or other evidence that will demonstrate completion.

AI Declaration

State the AI usage or non-usage declaration for this issue.

Example:

> The preceding issue was planned and generated with the assistance of ChatGPT-Web[GPT-5.6 Thinking].

This is the template you can later use for Azure, OAuth, Cloudflare, Supabase and other setup work.

Part 8: Correct the Pull Request template

Open:

```
.gitea/PULL_REQUEST_TEMPLATE.md
```

Replace its contents with:

Purpose

Explain why this change is required.

Changes

Summarise the changes included in this Pull Request.

-

Testing and Verification

Explain how the change was tested or verified.

- [] Relevant local checks pass
- [] New or changed behaviour has appropriate tests where applicable
- [] Existing tests continue to pass
- [] Error and failure states were considered
- [] The changed files were reviewed by the author

Known Limitations

List remaining limitations or state:

```
```text
None.
```

## Related Issue

Closes #

## Review Checklist

- ☐ All applicable issue acceptance criteria are met
- ☐ The Pull Request contains no unrelated changes
- ☐ Security implications were considered
- ☐ Accessibility and responsiveness were considered where relevant
- ☐ Documentation and evidence were updated
- ☐ No credentials, secrets or sensitive data were committed
- ☐ AI assistance was attributed where required
- ☐ All reviewer comments have been resolved

## Screenshots, API Examples or Other Evidence

Add relevant evidence or state that it is not applicable.

## AI Declaration

State the AI usage or non-usage declaration for this Pull Request description.

Example:

*The preceding Pull Request description was planned and generated with the assistance of ChatGPT-Web[GPT-5.6 Thinking].*

```

Part 9: Add `.gitattributes`

At the root of the repository, create:

```text
.gitattributes
```

Paste:

```
# Normalise text files to LF in the repository
* text=auto eol=lf
```

```
# Binary files
*.png binary
*.jpg binary
*.jpeg binary
*.gif binary
*.webp binary
*.ico binary
*.pdf binary
*.zip binary
```

Do **not** run:

```
git add --renormalize .
```

That could produce changes to nearly every existing file. This Pull Request should add the rule without rewriting the entire repository.

Part 10: Update the AI usage register

Open:

```
evidence/ai/ai-usage-register.csv
```

Add a new row at the bottom. Replace [Your name] with your actual name:

```
2026-08-05,[Your name],ChatGPT Web,GPT-5.6 Thinking,Planning; documentation; review,"Align repository governance and document the initial setup deviation","ADR-001; issue templates; Pull Request template; line-ending configuration guidance","Reviewed and adapted all content before commit","Issue #7 and linked Pull Request"
```

Do not remove Dean's existing row.

Part 11: Check the changes carefully

Run:

```
git status --short
```

You should only see these files:

```
M .gitea/ISSUE_TEMPLATE/bug.md
M .gitea/ISSUE_TEMPLATE/feature.md
?? .gitea/ISSUE_TEMPLATE/task.md
M .gitea/PULL_REQUEST_TEMPLATE.md
?? .gitattributes
M evidence/ai/ai-usage-register.csv
?? evidence/decisions/ADR-001-initial-repository-setup.md
```

You should **not** see dozens of application and documentation files.

Check for whitespace problems:

```
git diff --check
```

Review the changes:

```
git diff
```

You are intentionally **not changing**:

```
docs/project_methodology.md
docs/git-methodology.md
README.md
application code
CI configuration
```

Part 12: Make atomic commits

Commit 1: Deviation record and AI register

```
git add evidence/decisions/ADR-001-initial-repository-setup.md
git add evidence/ai/ai-usage-register.csv
git commit -m "docs: record initial repository setup deviation" -m "Refs #7"
```

Commit 2: Gitea templates

```
git add .gitea/ISSUE_TEMPLATE/bug.md
git add .gitea/ISSUE_TEMPLATE/feature.md
git add .gitea/ISSUE_TEMPLATE/task.md
git add .gitea/PULL_REQUEST_TEMPLATE.md
git commit -m "chore(gitea): align issue and pull request templates" -m "Refs #7"
```

Commit 3: Line-ending configuration

```
git add .gitattributes
git commit -m "chore: enforce consistent line endings" -m "Refs #7"
```

Check the final branch:

```
git status
git log --oneline -3
```

Your working tree should be clean.

Part 13: Push the branch

```
git push -u origin chore/7-align-repository-governance
```

Refresh Gitea and confirm the branch appears.

Part 14: Open the Pull Request

Use:

```
Target branch: main
Source branch: chore/7-align-repository-governance
```

Pull Request title

```
#7 Align repository governance and document initial setup deviation
```

Pull Request description

```
## Purpose
```

Align the repository governance files with the approved Git and Project Methodologies and document the initial repository setup as a one-time pre-methodology deviation.

Changes

- Added `ADR-001` documenting the initial direct-to-`main` scaffold commit
- Recorded the decision not to rewrite existing Git history
- Updated the feature issue template
- Updated the bug issue template
- Added a general task issue template
- Updated the Pull Request template
- Added `.gitattributes` to enforce consistent LF line endings
- Updated the AI usage register
- Created a separate blocked issue for branch protection pending administrator access
- Preserved the existing Project Methodology version and filename

Testing and Verification

- [X] Reviewed all changed Markdown files
- [X] Confirmed that `git diff --check` passes
- [X] Confirmed that no product code was changed
- [X] Confirmed that no mass line-ending-only changes were included
- [X] Confirmed that the existing methodology filenames and links were not changed
- [X] Confirmed that the new templates appear in the Gitea template directories

Known Limitations

Branch protection has not yet been configured because repository administrator access is still pending. This work is tracked in Issue #8.

Related Issue

Closes #7

AI Declaration

The preceding Pull Request description was planned and generated with the assistance of ChatGPT-Web[GPT-5.6 Thinking].

Change Issue #8 if Gitea assigned another number.

Part 15: Review and merge

Move Issue #7:

In Progress → In Review

Because branch protection is not yet active:

1. Manually request a teammate to review the Pull Request.
2. Wait for that person to leave an approval in Gitea.
3. Resolve any comments.
4. Merge using a **merge commit**.
5. Delete the remote branch.
6. Confirm Issue #7 closes.
7. Move Issue #7 to **Done**.

Update locally:

```
git switch main  
git pull origin main  
git branch -d chore/7-align-repository-governance
```

Issue #8 remains in **Blocked** until you receive administrator access.

8/5/2026, 11:45:25 AM



wait, part 5, give me the whole markdown to paste, itgot split up before

8/5/2026, 11:51:37 AM



Replace the entire contents of:

```
.gitea/ISSUE_TEMPLATE/feature.md
```

with this:

```
---
name: Feature
about: Implement a user-visible or platform capability
title: ""
labels: "type: feature"
assignees: ""
---

## Description

Describe the feature and the result that must be achieved.

## Motivation

Explain why this feature is required and what project, stakeholder or user need it addresses.

## Scope

### Included

-

### Excluded

-

## Acceptance Criteria

- [ ] The required feature behaviour is implemented
- [ ] Relevant tests have been added or updated
- [ ] Existing tests continue to pass
- [ ] Relevant documentation has been updated
- [ ] The implementation has been reviewed through a Pull Request

Add or replace the criteria above with specific, observable requirements for this feature.

## Dependencies

List any related issues, services, technical decisions, stakeholder information or access requirements.

- None currently identified.

## Testing and Verification

Explain how the completed feature will be tested or verified.

Include, where applicable:
```

- automated tests;
- manual testing steps;
- expected results;
- error and failure cases;
- accessibility checks;
- responsive-design checks; and
- API or database verification.

Documentation and Evidence

List the evidence that must be produced to demonstrate completion.

Examples include:

- linked Pull Request;
- test output;
- screenshots;
- API examples;
- updated documentation;
- deployment logs; or
- stakeholder feedback.

Security, Accessibility and Responsiveness

Describe any relevant considerations.

Security

-

Accessibility

-

Responsiveness

-

If a section is not applicable, state why.

AI Declaration

State the AI usage or non-usage declaration for this issue.

Example:

> The preceding issue was planned and generated with the assistance of ChatGPT-Web[GPT-5.6 Thinking].

Or, where no AI was used:

> The preceding issue was written without the assistance of AI.

8/5/2026, 11:51:43 AM



i meant part 4 pls

8/5/2026, 11:52:29 AM



Replace the entire contents of:

```
evidence/decisions/ADR-001-initial-repository-setup.md
```

with this:

```
# ADR-001: Initial Repository Setup Before Governance Enforcement
```

- ****Status:**** Accepted
- ****Date:**** 2026-08-05
- ****Participants:**** Git Push Pray
- ****Related Issues:**** #7, #8

Context

The repository was initially created before the approved Git Methodology and Project Methodology were fully operational.

The initial project scaffold established the repository structure, including the frontend, backend, shared contracts, database folders, documentation, testing structure, CI configuration, deployment planning and evidence directories.

However, the initial scaffold was committed directly to `main` without:

- a preceding Gitea issue;
- a dedicated issue branch;
- a Pull Request;
- a recorded peer review; or
- branch protection being configured.

No issue was created for the initial scaffold before the commit was made.

Branch protection had not yet been configured because the required repository administrator access was unavailable.

The scaffold provides a useful project foundation and should not be removed solely because the approved methodology was not yet being enforced.

Decision

The team will not delete, revert or rewrite the existing Git history.

The initial repository scaffold will be treated as a one-time pre-methodology setup deviation.

The deviation will be documented transparently rather than attempting to recreate or conceal the original work.

From Issue #7 onward, all significant project work must follow the approved workflow:

```
```text
```

```
Gitea issue → branch → atomic commits → Pull Request → peer review → merge
```

Repository governance files and templates will be aligned with the approved Git Methodology and Project Methodology before the team begins parallel setup work.

Branch protection is tracked separately in Issue #8 and will remain blocked until repository administrator access becomes available.

## **Alternatives Considered**

### **Rewrite the Existing Git History**

This option was rejected because rewriting shared history could remove useful evidence, disrupt other team members and create unnecessary technical risk.

### **Revert the Scaffold and Recreate It Through an Issue and Pull Request**

This option was rejected because the scaffold already provides a useful project foundation. Recreating the same files would add work without improving the project.

### **Leave the Deviation Undocumented**

This option was rejected because it would create an inaccurate record of how the repository was established and would weaken the team's evidence of methodology compliance.

### **Document the Deviation and Apply the Methodology Prospectively**

This option was selected because it preserves an honest repository history while ensuring that all future work follows the approved process consistently.

## **Advantages**

- Existing project work is preserved.
- The repository history remains accurate.
- The deviation is documented transparently.
- The team can begin following the approved methodology immediately.
- Future work will provide clear traceability between issues, branches, commits and Pull Requests.
- The team avoids unnecessary disruption caused by rewriting shared history.

## **Disadvantages**

- The initial scaffold does not provide complete evidence of the approved workflow.
- The initial scaffold cannot be linked retrospectively to a preceding issue.
- Branch protection remains unenforced until administrator access is available.

## **Consequences**

- The initial scaffold remains part of the repository history.
- The initial scaffold will not be treated as evidence of full methodology compliance.
- Issue #7 will be used to align repository governance and templates.
- All future significant work must begin with a Gitea issue.
- All future development work must use a branch linked to the relevant issue.
- Commits must reference the relevant issue where applicable.
- Changes to `main` must be reviewed through a Pull Request.
- At least one team member other than the author must review and approve each Pull Request.
- Issue #8 must be completed once repository administrator access is granted.
- Any future methodology deviation must be documented and justified.

## **Follow-Up Actions**

- ☐ Align the Gitea issue templates with the approved Project Methodology.
- ☐ Align the Pull Request template with the approved Git Methodology.
- ☐ Add a general task issue template.
- ☐ Add a `.gitattributes` file to enforce consistent line endings.
- ☐ Create and track a separate issue for protecting `main`.
- ☐ Configure branch protection once administrator access is available.
- ☐ Require the team to follow the approved workflow for all future significant work.
- ☐ Review compliance during the Sprint 1 close-out.

## Verification

This decision will be considered successfully implemented when:

- Issue #7 has been completed through the approved issue, branch, commit, Pull Request and review workflow;
- the updated issue and Pull Request templates have been merged into `main`;
- later work uses linked Gitea issues, branches and Pull Requests;
- the initial setup deviation remains documented in the repository; and
- branch protection has either been configured or remains visibly tracked as blocked.

## Review Date

This decision will be reviewed during the Sprint 1 close-out.

The review will confirm whether:

- Issue #7 followed the approved methodology;
- later team members used linked issues, branches and Pull Requests;
- the repository templates were used consistently; and
- branch protection was configured or remains visibly blocked.

## AI Declaration

The preceding document was planned and generated with the assistance of ChatGPT-Web[GPT-5.6 Thinking].

Change `#8` wherever it appears if the branch-protection issue received a different number.

8/5/2026, 11:52:35 AM



again. from alternatives considered isnt markdown, should it be>?

8/5/2026, 11:53:40 AM



Yes, it should **all** be Markdown. The previous block broke because the document itself contained a triple-backtick code block, which accidentally closed the surrounding Markdown block.

Copy everything inside this **four-backtick block** into:

evidence/decisions/ADR-001-initial-repository-setup.md

```
ADR-001: Initial Repository Setup Before Governance Enforcement
```

- **\*\*Status:\*\*** Accepted
- **\*\*Date:\*\*** 2026-08-05
- **\*\*Participants:\*\*** Git Push Pray
- **\*\*Related Issues:\*\*** #7, #8

## ## Context

The repository was initially created before the approved Git Methodology and Project Methodology were fully operational.

The initial project scaffold established the repository structure, including the frontend, backend, shared contracts, database folders, documentation, testing structure, CI configuration, deployment planning and evidence directories.

However, the initial scaffold was committed directly to `main` without:

- a preceding Gitea issue;
- a dedicated issue branch;
- a Pull Request;
- a recorded peer review; or
- branch protection being configured.

No issue was created for the initial scaffold before the commit was made.

Branch protection had not yet been configured because the required repository administrator access was unavailable.

The scaffold provides a useful project foundation and should not be removed solely because the approved methodology was not yet being enforced.

## ## Decision

The team will not delete, revert or rewrite the existing Git history.

The initial repository scaffold will be treated as a one-time pre-methodology setup deviation.

The deviation will be documented transparently rather than attempting to recreate or conceal the original work.

From Issue #7 onward, all significant project work must follow the approved workflow:

```
```text
```

```
Gitea issue → branch → atomic commits → Pull Request → peer review → merge
```

```
```
```

Repository governance files and templates will be aligned with the approved Git Methodology and Project Methodology before the team begins parallel setup work.

Branch protection is tracked separately in Issue #8 and will remain blocked until repository administrator access becomes available.

## ## Alternatives Considered

### ### Rewrite the Existing Git History

This option was rejected because rewriting shared history could remove useful evidence, disrupt other team members and create unnecessary technical risk.

### ### Revert the Scaffold and Recreate It Through an Issue and Pull Request

This option was rejected because the scaffold already provides a useful project foundation. Recreating the same files would add work without improving the project.

### ### Leave the Deviation Undocumented

This option was rejected because it would create an inaccurate record of how the repository was established and would weaken the team's evidence of methodology compliance.

### ### Document the Deviation and Apply the Methodology Prospectively

This option was selected because it preserves an honest repository history while ensuring that all future work follows the approved process consistently.

## ## Advantages

- Existing project work is preserved.
- The repository history remains accurate.
- The deviation is documented transparently.
- The team can begin following the approved methodology immediately.
- Future work will provide clear traceability between issues, branches, commits and Pull Requests.
- The team avoids unnecessary disruption caused by rewriting shared history.

## ## Disadvantages

- The initial scaffold does not provide complete evidence of the approved workflow.
- The initial scaffold cannot be linked retrospectively to a preceding issue.
- Branch protection remains unenforced until administrator access is available.

## ## Consequences

- The initial scaffold remains part of the repository history.
- The initial scaffold will not be treated as evidence of full methodology compliance.
- Issue #7 will be used to align repository governance and templates.
- All future significant work must begin with a Gitea issue.
- All future development work must use a branch linked to the relevant issue.
- Commits must reference the relevant issue where applicable.
- Changes to `main` must be reviewed through a Pull Request.
- At least one team member other than the author must review and approve each Pull Request.
- Issue #8 must be completed once repository administrator access is granted.
- Any future methodology deviation must be documented and justified.

## ## Follow-Up Actions

- [ ] Align the Gitea issue templates with the approved Project Methodology.
- [ ] Align the Pull Request template with the approved Git Methodology.
- [ ] Add a general task issue template.
- [ ] Add a `.gitattributes` file to enforce consistent line endings.
- [ ] Create and track a separate issue for protecting `main`.

- [ ] Configure branch protection once administrator access is available.
- [ ] Require the team to follow the approved workflow for all future significant work.
- [ ] Review compliance during the Sprint 1 close-out.

### ## Verification

This decision will be considered successfully implemented when:

- Issue #7 has been completed through the approved issue, branch, commit, Pull Request and review workflow;
- the updated issue and Pull Request templates have been merged into `main`;
- later work uses linked Gitea issues, branches and Pull Requests;
- the initial setup deviation remains documented in the repository; and
- branch protection has either been configured or remains visibly tracked as blocked.

### ## Review Date

This decision will be reviewed during the Sprint 1 close-out.

The review will confirm whether:

- Issue #7 followed the approved methodology;
- later team members used linked issues, branches and Pull Requests;
- the repository templates were used consistently; and
- branch protection was configured or remains visibly blocked.

### ## AI Declaration

The preceding document was planned and generated with the assistance of ChatGPT-Web[GPT-5.6 Thinking].

Change #8 if the branch-protection issue received a different number.

8/5/2026, 11:53:49 AM



pls regive me from part 8-end, endsure markdowns are all correct!

8/5/2026, 11:57:01 AM



### Thinking

- **Continuing the workflow**

I'm providing Parts 8 through the end, keeping Markdown fencing valid and preserving the existing methodology. I'm using issue #8 as the working reference; if numbering differs, I'll substitute the correct issue number.

- **Continued the workflow**

✓ Worked for 12s

Below is **Part 8 through Part 15**, with all Markdown blocks correctly contained.

Use Issue **#7** throughout. Where Issue **#8** is mentioned, replace it only if your branch-protection issue received a different number.

## Part 8: Update the Pull Request template

Open:

```
.gitea/PULL_REQUEST_TEMPLATE.md
```

Replace the entire file with the following:

### ## Purpose

Explain why this change is required.

### ## Changes

Summarise the changes included in this Pull Request.

-

### ## Testing and Verification

Explain how the change was tested or verified.

- [ ] Relevant local checks pass
- [ ] New or changed behaviour has appropriate tests where applicable
- [ ] Existing tests continue to pass
- [ ] Error and failure states were considered
- [ ] The changed files were reviewed by the author

### ## Known Limitations

List any remaining limitations.

If there are no known limitations, write:

**None.**

### ## Related Issue

Closes #

### ## Review Checklist

- [ ] All applicable issue acceptance criteria are met
- [ ] The Pull Request contains no unrelated changes
- [ ] Security implications were considered where relevant
- [ ] Accessibility and responsiveness were considered where relevant
- [ ] Documentation and evidence were updated
- [ ] No credentials, secrets or sensitive information were committed
- [ ] AI assistance was attributed where required
- [ ] All reviewer comments have been resolved

### ## Screenshots, API Examples or Other Evidence

Add relevant evidence.

If this section is not applicable, state why.

### ## AI Declaration

State the AI usage or non-usage declaration for this Pull Request description.

Example:

*> The preceding Pull Request description was planned and generated with the assistance of ChatGPT-Web[GPT-5.6 Thinking].*

Where no AI was used:

*> The preceding Pull Request description was written without the assistance of AI.*

Save the file.

## Part 9: Add `.gitattributes`

At the root of the repository, create a new file named:

```
.gitattributes
```

It must sit alongside files such as:

```
README.md
.gitignore
package.json
```

Paste the following into `.gitattributes`:

```
Normalise text files to LF in the repository
* text=auto eol=lf

Windows scripts should retain CRLF where required
*.bat text eol=crlf
*.cmd text eol=crlf
*.ps1 text eol=crlf

Binary files
*.png binary
*.jpg binary
*.jpeg binary
```

```
*.gif binary
*.webp binary
*.ico binary
*.pdf binary
*.zip binary
```

Do **not** run this command:

```
git add --renormalize .
```

That may cause line-ending changes across most of the existing repository.

For Issue #7, you are adding the future line-ending rule without rewriting every existing file.

---

## Part 10: Update the AI usage register

Open:

```
evidence/ai/ai-usage-register.csv
```

Do not delete or change Dean's existing entry.

Add a new row at the bottom.

Replace [Your name] with your actual name:

```
2026-08-05,[Your name],ChatGPT Web,GPT-5.6 Thinking,Planning; documentation; review,"Align repository governance and document the initial setup deviation","ADR-001; issue templates; Pull Request template; line-ending configuration guidance","Reviewed and adapted all content before commit","Issue #7 and linked Pull Request"
```

Be careful that the row remains on one line in the CSV file.

Save the file.

---

## Part 11: Check all changes before committing

In the VS Code terminal, confirm that you are on the correct branch:

```
git branch --show-current
```

It should return:

```
chore/7-align-repository-governance
```

Now run:

```
git status --short
```

You should see only files related to Issue #7.

The list should look similar to:

```
M .gitea/ISSUE_TEMPLATE/bug.md
M .gitea/ISSUE_TEMPLATE/feature.md
?? .gitea/ISSUE_TEMPLATE/task.md
M .gitea/PULL_REQUEST_TEMPLATE.md
?? .gitattributes
M evidence/ai/ai-usage-register.csv
?? evidence/decisions/ADR-001-initial-repository-setup.md
```

The spacing may differ slightly.

You should **not** see:

- frontend source files;
- backend source files;
- package files;
- dozens of documentation files;
- mass line-ending changes;
- `docs/project_methodology.md`.

You are **not changing the methodology version to 1.1**.

## Check for whitespace errors

Run:

```
git diff --check
```

Ideally, this returns no output.

If it reports trailing whitespace in Markdown templates, remove the unnecessary spaces and run it again.

## Review the changes

Run:

```
git diff
```

Check that:

- the ADR correctly records the initial deviation;
- the feature template is correct;
- the bug template is correct;
- the task template is correct;
- the Pull Request template is correct;
- `.gitattributes` contains only the intended rules;
- the AI register contains your new row;
- there are no unrelated changes.

You can also inspect one file at a time:

```
git diff -- .gitea/PULL_REQUEST_TEMPLATE.md
```

```
git diff -- evidence/ai/ai-usage-register.csv
```

For new untracked files, open them directly in VS Code and review them.

---

## Part 12: Create atomic commits

You will make three separate commits.

### Commit 1: Record the setup deviation

Stage the ADR and AI register:

```
git add evidence/decisions/ADR-001-initial-repository-setup.md
git add evidence/ai/ai-usage-register.csv
```

Check what is staged:

```
git status
```

Commit:

```
git commit -m "docs: record initial repository setup deviation" -m "Refs #7"
```

## Commit 2: Align the Gitea templates

Stage the issue templates and Pull Request template:

```
git add .gitea/ISSUE_TEMPLATE/bug.md
git add .gitea/ISSUE_TEMPLATE/feature.md
git add .gitea/ISSUE_TEMPLATE/task.md
git add .gitea/PULL_REQUEST_TEMPLATE.md
```

Check:

```
git status
```

Commit:

```
git commit -m "chore(gitea): align issue and pull request templates" -m "Refs #7"
```

## Commit 3: Add line-ending configuration

Stage `.gitattributes`:

```
git add .gitattributes
```

Check:

```
git status
```

Commit:

```
git commit -m "chore: enforce consistent line endings" -m "Refs #7"
```

## Confirm the three commits

Run:

```
git log --oneline -3
```

You should see three commits similar to:

```
abc1234 chore: enforce consistent line endings
def5678 chore(gitea): align issue and pull request templates
ghi9012 docs: record initial repository setup deviation
```

Then run:

```
git status
```

It should say:

```
nothing to commit, working tree clean
```

## Part 13: Push the branch

Push the branch to Gitea:

```
git push -u origin chore/7-align-repository-governance
```

After it finishes, refresh Gitea.

You should now see the remote branch:

```
chore/7-align-repository-governance
```

The branch should show three commits ahead of `main`.

## Part 14: Open the Pull Request

In Gitea:

1. Open **Pull Requests**.
2. Click **New Pull Request**.
3. Set the branches as follows:

```
Target branch: main
Source branch: chore/7-align-repository-governance
```

### Pull Request title

Use:

```
#7 Align repository governance and document initial setup deviation
```

### Pull Request description

Paste the following entire Markdown block:

#### ## Purpose

Align the repository governance files with the approved Git Methodology and Project Methodology, and document the initial repository setup as a one-time pre-methodology deviation.

#### ## Changes

- Added `ADR-001` documenting the initial repository scaffold
- Recorded that the initial scaffold was committed without a preceding issue or Pull Request
- Recorded the decision not to rewrite or remove the existing Git history
- Updated the feature issue template
- Updated the bug issue template
- Added a general task issue template
- Updated the Pull Request template
- Added `.gitattributes` to enforce consistent future line endings
- Updated the AI usage register
- Created a separate blocked issue for branch protection pending administrator access
- Preserved the existing Project Methodology version and filename

### ## Testing and Verification

- [X] Reviewed all changed Markdown files
- [X] Confirmed that `git diff --check` passes
- [X] Confirmed that no product code was changed
- [X] Confirmed that no mass line-ending-only changes were included
- [X] Confirmed that the existing methodology documents were not altered
- [X] Confirmed that the issue templates are stored in `.gitea/ISSUE\_TEMPLATE/`
- [X] Confirmed that the Pull Request template is stored at `.gitea/PULL\_REQUEST\_TEMPLATE.md`
- [X] Confirmed that the AI usage register contains the new entry

### ## Known Limitations

Branch protection has not yet been configured because repository administrator access is still pending.

This work is tracked separately in Issue #8.

### ## Related Issue

Closes #7

### ## Screenshots, API Examples or Other Evidence

Not applicable. The changes consist of repository-governance documentation and templates.

### ## AI Declaration

The preceding Pull Request description was planned and generated with the assistance of ChatGPT-Web[GPT-5.6 Thinking].

Change #8 only if the branch-protection issue has a different number.

Before submitting the Pull Request, verify that the **Files Changed** section contains only the expected files.

Then create the Pull Request.

## Part 15: Review, merge and close Issue #7

### Move Issue #7 to In Review

Open the Project board and move Issue #7:

In Progress → In Review

### Request a review

Because you do not yet have Admin access and branch protection may not enforce approval automatically:

1. Ask one teammate to open the Pull Request.
2. They should review the **Files Changed** section.
3. They should confirm that the templates and ADR are correct.
4. They should check that no unrelated files were changed.
5. They should leave an approval or clear review comment in Gitea.

The reviewer should be someone other than the Pull Request author.

### Respond to any requested changes

If the reviewer requests a correction, make it on the same branch.

For example:

```
git branch --show-current
```

Confirm that you remain on:

```
chore/7-align-repository-governance
```

Edit the required file, then commit the correction:

```
git add <changed-file>
git commit -m "docs: address governance review feedback" -m "Refs #7"
git push
```

The existing Pull Request will update automatically.

## Merge the Pull Request

Once the review is complete:

1. Confirm the target branch is `main`.
2. Choose **Create a merge commit**.
3. Merge the Pull Request.
4. Delete the remote branch if Gitea offers that option.
5. Confirm that Issue #7 closes automatically because the PR says `Closes #7`.
6. Move Issue #7 to `Done`.

Issue #8 should remain in:

```
Blocked
```

until Admin access is available.

## Update your local repository

Return to the VS Code terminal:

```
git switch main
git pull origin main
```

Delete the completed local branch:

```
git branch -d chore/7-align-repository-governance
```

Check the final state:

```
git status
```

It should show:

```
On branch main
Your branch is up to date with 'origin/main'.
nothing to commit, working tree clean
```

Issue #7 is then complete.

8/5/2026, 11:57:12 AM



ok wait, itll be 1 ai chat for everything from methodology to today, sho yesturday and today. how should i go about that step then?

8/5/2026, 12:01:53 PM



### Thinking

- **Clarifying register entries**

I'm checking whether one continuous chat can cover multiple work sessions. Since the policy requires a new session per assessment, I'd record the chat once with its date range, tool, model, and purpose, then note the transcript location.

- **Clarified register entries**

-  **Inspecting the AI usage register template**

- **Logging one ChatGPT session**

I'd keep one unedited transcript covering both dates, because it supports one assessment. In the register, use one date-range row covering planning, documentation, and governance outputs, and link the saved transcript plus related issue or pull request.

- **Logged one ChatGPT session**

- **Inspected the AI usage register template**

- **Reviewed AI policy and documented continuous chat session requirements**

✓ Worked for 1m 30s

Yes - because this has been **one continuous ChatGPT session for the same project assessment**, covering 4-5 August 2026, you do not need a separate transcript for each day or each document.

The policy requires a new session for each **assessment**, not each day, and it requires you to record the tool, model and purpose. It also says AI-assisted writing does not need attribution for every individual chunk, although each submitted document still needs its own declaration. [ai\\_policy\\_20\\_jul.pdf](#)

## For Part 10, do this

### 1. Keep one unedited transcript

Save/export this entire chat once, from the start of the methodology work through completion of Issue #7.

Suggested filename:

2026-08-04-to-2026-08-05-chatgpt-methodology-and-governance-transcript.pdf

or:

2026-08-04-to-2026-08-05-chatgpt-methodology-and-governance-transcript.md

Do not shorten or rewrite it if it is being kept as the required unedited transcript.

The course policy says you may be required to provide an unedited assessment transcript, or a suitable saved substitute if the tool cannot export one. [ai\\_policy\\_20\\_jul.pdf](#)

You do not necessarily need to place the full transcript in the public repository. Keep it safely for submission unless your lecturer specifically asks that transcripts be committed.

### 2. Add one row to the AI usage register

Open:

```
evidence/ai/ai-usage-register.csv
```

Add **one row** for the complete session. Replace `[Your name]` :

```
2026-08-05,[Your name],ChatGPT Web,GPT-5.6 Thinking,Planning; documentation; review; Git and Gitea guidance,"Single project session used from 2026-08-04 to 2026-08-05 to develop the Git Methodology, Project Methodology, Gitea workflow, Issue #7 governance correction, ADR-001, issue templates, Pull Request template and related repository setup guidance","Git Methodology; Project Methodology; Issue #7 description; ADR-001; issue templates; Pull Request template; AI register wording; repository workflow instructions","Reviewed, adapted and verified each item before use; no application code was generated or used from this session","Unedited ChatGPT transcript retained for the project; Issue #7 and linked Pull Request"
```

Keep that whole row on **one line** in the CSV.

### 3. Keep the individual declarations

The one register entry does **not** replace the declarations already included in:

- the Git Methodology;
- the Project Methodology;
- ADR-001;
- Issue #7;
- the Pull Request description.

Those declarations identify which particular documents were planned or generated with AI, while the register records the broader session. The policy requires submitted documents to include either a usage or non-usage declaration.

```
ai_policy_20_jul.pdf
```

### 4. Commit wording

Because the session mainly produced documentation and repository templates, the register and document declarations provide the main attribution.

For maximum transparency, you can also add this footer to the Issue #7 commits:

```
Assisted-by: ChatGPT-Web[GPT-5.6 Thinking]
```

For example:

```
git commit -m "docs: record initial repository setup deviation" \
-m "Refs #7" \
-m "Assisted-by: ChatGPT-Web[GPT-5.6 Thinking]"
```

Use the same footer on the template and `.gitattributes` commits because AI provided the initial content for those files.

Going forward, start a fresh chat for each major issue. Multiple chats are allowed within the same assessment, and separate sessions will make the evidence much easier to organise.

8/5/2026, 12:02:03 PM



in part 12 commits, dont i need Ai declerations??

8/5/2026, 12:08:06 PM



### Thinking

- **Explained AI attribution requirements for all Part 12 commits**

✓ Worked for 20s

Yes - for these commits, add the AI attribution footer.

Strictly, the policy explicitly requires `Assisted-by` for **AI-generated code**, while AI-assisted writing must be declared in the documents themselves. However, because these commits include AI-assisted repository templates and configuration, adding the footer to all three is the safest and most transparent approach. The policy requires the tool and model to be identified in the commit when AI is used for code generation. `ai_policy_20_jul.pdf`

Use these corrected commands for **Part 12**:

### Commit 1: Deviation record and AI register

```
git add evidence/decisions/ADR-001-initial-repository-setup.md
git add evidence/ai/ai-usage-register.csv

git commit \
 -m "docs: record initial repository setup deviation" \
 -m "Refs #7" \
 -m "Assisted-by: ChatGPT-Web[GPT-5.6 Thinking]"
```

### Commit 2: Gitea templates

```
git add .gitea/ISSUE_TEMPLATE/bug.md
git add .gitea/ISSUE_TEMPLATE/feature.md
git add .gitea/ISSUE_TEMPLATE/task.md
git add .gitea/PULL_REQUEST_TEMPLATE.md

git commit \
 -m "chore(gitea): align issue and pull request templates" \
 -m "Refs #7" \
 -m "Assisted-by: ChatGPT-Web[GPT-5.6 Thinking]"
```

### Commit 3: Line-ending configuration

```
git add .gitattributes

git commit \
 -m "chore: enforce consistent line endings" \
 -m "Refs #7" \
 -m "Assisted-by: ChatGPT-Web[GPT-5.6 Thinking]"
```

Then verify:

```
git log -3 --format=full
```

Each commit should show:

Refs #7

Assisted-by: ChatGPT-Web[GPT-5.6 Thinking]

The document-level AI declarations and the AI usage register must still remain; the commit footer does not replace them.

8/5/2026, 12:08:30 PM



uhm?

8/5/2026, 12:09:49 PM



### Thinking

- Clarified AI attribution requirements

✓ Worked for 14s

8/5/2026, 12:10:06 PM



e0dca32e-50d6-4bf5-9d0c-e3f...

uhm?

8/5/2026, 12:10:05 PM



### Thinking

- **Clarifying commit declarations**

You're right to question that. I'm revising my earlier advice: don't add AI footers automatically to all three commits. Follow the assignment's exact rule—declare AI-generated content where required, and use the register for transparency.

- **Clarified commit declarations**

✓ Worked for 26s

That command format was wrong for **PowerShell**. The `\` line continuation works in Bash, not PowerShell.

Also, a correction: these commits contain documentation and templates, not generated application code. The course policy requires `Assisted-by:` in commits specifically for AI-generated code. Your document declarations and AI register are sufficient here. `ai_policy_20_jul.pdf`

Use these simpler commands:

```
git add evidence/decisions/ADR-001-initial-repository-setup.md evidence/ai/ai-usage-register.csv
git commit -m "docs: record initial repository setup deviation" -m "Refs #7"
```

```
git add .gitea/ISSUE_TEMPLATE/bug.md .gitea/ISSUE_TEMPLATE/feature.md .gitea/ISSUE_TEMPLATE/task.md
.gitea/PULL_REQUEST_TEMPLATE.md
git commit -m "chore(gitea): align issue and pull request templates" -m "Refs #7"
```

```
git add .gitattributes
git commit -m "chore: enforce consistent line endings" -m "Refs #7"
```

If PowerShell is currently showing a continuation prompt such as `>>`, press:

```
Ctrl + C
```

Then run the commands above one line at a time.

You may still add `Assisted-by: ChatGPT-Web[GPT-5.6 Thinking]` voluntarily, but it is not necessary for these documentation-only commits.

8/5/2026, 12:10:07 PM



i want to undo the commit! how to get out of where i am?

8/5/2026, 12:10:53 PM



You have **not committed yet**. You are inside Vim editing the commit message.

To cancel the commit and exit:

1. Press `Esc`
2. Type:

```
:cq
```

3. Press `Enter`

That aborts the commit safely. Your staged files will remain staged.

Then run:

```
git status
```

To also unstage the files afterward:

```
git restore --staged evidence/decisions/ADR-001-initial-repository-setup.md evidence/ai/ai-usage-register.csv
```

8/5/2026, 12:11:08 PM



now give me commits that will work with Ai declaration too

8/5/2026, 12:11:59 PM



### Thinking

- **Preparing commit commands**

I'm giving three PowerShell one-liners in dependency order. Each includes the requested subject, issue reference, and exact AI-assistance footer; the final command assumes `.gitattributes` is already created.

- **Prepared commit commands**

✓ Worked for 9s

Run each command on **one line** in the VS Code terminal. These work in PowerShell and include the AI attribution.

### Commit 1: Setup deviation and AI register

```
git add evidence/decisions/ADR-001-initial-repository-setup.md evidence/ai/ai-usage-register.csv
```

```
git commit -m "docs: record initial repository setup deviation" -m "Refs #7" -m "Assisted-by: ChatGPT-Web[GPT-5.6 Thinking]"
```

### Commit 2: Issue and Pull Request templates

```
git add .gitea/ISSUE_TEMPLATE/bug.md .gitea/ISSUE_TEMPLATE/feature.md .gitea/ISSUE_TEMPLATE/task.md .gitea/PULL_REQUEST_TEMPLATE.md
```

```
git commit -m "chore(gitea): align issue and pull request templates" -m "Refs #7" -m "Assisted-by: ChatGPT-Web[GPT-5.6 Thinking]"
```

### Commit 3: Line-ending configuration

```
git add .gitattributes
```

```
git commit -m "chore: enforce consistent line endings" -m "Refs #7" -m "Assisted-by: ChatGPT-Web[GPT-5.6 Thinking]"
```

Then verify all three commits:

```
git log -3 --format=full
```

Each commit should show:

Refs #7

Assisted-by: ChatGPT-Web[GPT-5.6 Thinking]

Finally:

```
git status
```

It should say:

```
nothing to commit, working tree clean
```

*8/5/2026, 12:12:08 PM*



a3f9a95a-ab22-4cf0-b1c4-8fd...

what does this mean?

8/5/2026, 12:20:24 PM



## Thinking

- **Interpreting the workflow status**

I'm reading the clock icons as queued runs: the workflow was triggered but hasn't started on a runner. The warning beside `ci.yml` suggests a configuration or validation issue; zero seconds means no job actually ran.

- **Interpreted the workflow status**



- **Inspecting Screenshot Workflow Status**

- **Inspected screenshot workflow status and ci.yml configuration**



- **Searching the web**

[docs.gitea.com](https://docs.gitea.com)

- **Reading the run states**

I'm seeing two queued runs and one that stopped immediately. The likely issue is that no runner is available for `ubuntu-latest`, rather than a failed test. Open the newest run: if it says "waiting for runner," an administrator must enable or register one.

- **Read the run states**

- **Searched 1 website**

✓ Worked for 41s

Your **Pull Request merged successfully**. The problem shown here is with the **CI workflow**, not the merge.

## What the symbols mean

- **Top row:** The merge of PR `#9` into `main` triggered a new CI run.
- **Second row:** The final commit on PR `#9` also triggered a CI run.
- **Yellow clock + 0s:** The workflow is **queued and has not started running**.
- **Bottom row with !:** The earlier CI run did not complete successfully or could not start.
- **Red warning triangle next to ci.yml:** The workflow currently has an unresolved problem or no successful run.

Your `ci.yml` contains:

```
runs-on: ubuntu-latest
```

Gitea therefore needs an active **Gitea Actions runner** registered with the `ubuntu-latest` label. Gitea Actions cannot execute jobs without a runner, and jobs are matched to runners using the `runs-on` label. ([Gitea Documentation](#))

## What to do now

Click the **top workflow run**. It will probably say something similar to:

Waiting for a runner to pick up this job



If it says that, the issue is not your code. The Wits Gitea administrator needs to provide or activate an Actions runner that supports:

```
ubuntu-latest
```

You can send the administrator or lecturer:

*Our Gitea Actions workflow is remaining queued at 0 seconds. The workflow uses `runs-on: ubuntu-latest`. Please confirm whether an Actions runner with the `ubuntu-latest` label is available for our repository or organisation.*

Do **not** change the workflow merely to make the warning disappear until you know which runner labels the Wits server supports.

Your governance changes have merged. The next item is simply to determine why CI has no available runner.

### Sources

[Design of Gitea Actions | Gitea Documentation](https://docs.gitea.com) docs.gitea.com

8/5/2026, 12:20:36 PM



so the merge will be succesfull regardless? then i have admin access so we can do protecti nvng main

8/5/2026, 12:27:23 PM



### Thinking

 **Searching the web**

[docs.gitea.com](https://docs.gitea.com)

- **Clarifying merge protection**  
Yes, the merge is already in `main`; a queued or failing CI run won't undo it. I'm recommending branch protection now: require one approval, dismiss stale approvals, block force-pushes, and require administrators to follow rules. I'd delay mandatory CI checks until the runner works.
- **Searched 1 website**

✓ Worked for 16s

Yes. **The merge has already succeeded and the changes are now on** `main`. The queued or failed CI run does not automatically undo or roll back that merge. It only shows that the automated workflow did not complete. Future merges can be prevented when required checks are enabled, but that protection was not active for this merge.

Now that you have Admin access, complete the branch-protection issue.

## 1. Update Issue #8

Move Issue #8 from:

Blocked → In Progress

Remove or update the blocker comment to state that Admin access has now been granted.

## 2. Open the branch settings

In the repository:

Settings → Branches

Then click:

Add new rule

Only repository owners and administrators can create these rules. ([Gitea Documentation](#))

## 3. Set the protected branch

For the branch-name pattern, enter:

main

Do not use `*`, because you currently only want to protect `main`.

## 4. Configure direct pushes

Under the **Push** section, select the option equivalent to:

Disable push

This means nobody with normal write access can push commits directly to `main` ; changes must enter through Pull Requests. ([Gitea Documentation](#))

Do not add yourself or the other team members to a push allowlist.

## 5. Disable force pushes

Set:

```
Force push: Disabled
```

This prevents anyone from rewriting the history of `main` . ([Gitea Documentation](#))

## 6. Configure Pull Request approvals

Set:

```
Required approvals: 1
```

This means any one eligible team member with write access can approve the Pull Request. It does not need to be the same reviewer each time. ([Gitea Documentation](#))

I also recommend enabling:

```
Dismiss stale approvals
```

This means that if new changes are pushed after approval, the previous approval is removed and the updated work must be reviewed again.

Enable:

```
Block merge on rejected reviews
```

This prevents merging while a reviewer has formally requested changes.

## 7. Make administrators follow the rules

Enable:

```
Administrators must follow branch protection rules
```

Without this option, an administrator may still be offered a force-merge or bypass option. Enabling it ensures that you and Dean must follow the same Pull Request and review process as everyone else. ([Gitea Documentation](#))

## 8. Do not require CI checks yet

For now, leave this disabled:

```
Enable status check
```

Your workflow is currently queued because the Actions runner does not appear to be available. Gitea Actions requires a separate runner to execute jobs. ([Gitea Documentation](#))

If you require a successful CI check now, future Pull Requests may become impossible to merge because the check cannot run.

Once the Wits runner is working and you have at least one successful Actions run, return to this rule and enable required status checks.

## 9. Save the rule

Click:

Save rule

The settings should effectively be:

```
Protected branch: main
Direct pushes: disabled
Force pushes: disabled
Required approvals: 1
Dismiss stale approvals: enabled
Block merge on rejected reviews: enabled
Administrators must follow rules: enabled
Required status checks: disabled for now
```

## 10. Capture evidence

Take a screenshot of the saved branch-protection rule.

Add it to Issue #8 or store it under an evidence folder such as:

```
evidence/repository-governance/main-branch-protection.png
```

Add a comment to Issue #8:

Branch protection has been configured for `main`.

The rule:

- blocks direct pushes;
- blocks force pushes;
- requires one Pull Request approval;
- requires administrators to follow the protection rule; and
- blocks merging where changes have been requested.

Required CI status checks have not yet been enabled because the Gitea Actions runner is currently unavailable. This will be revisited once a successful CI run is available.

Then move Issue #8:

In Progress → Done

Strictly, the CI limitation could remain as a separate issue rather than keeping branch protection blocked. Create a new issue such as:

```
Investigate unavailable Gitea Actions runner
```

That separates the completed branch-protection work from the unresolved CI infrastructure problem.

### Sources

[Protected branches | Gitea Documentation](https://docs.gitea.com) docs.gitea.com

8/5/2026, 12:27:25 PM



ok, so now give me recomendatin of 6 issues we should create and work on now (myabe ci one can be one?)

8/5/2026, 12:35:53 PM



## Thinking

### Inspecting repository structure and existing files

- Planning sprint issues

I'm mapping six independent setup issues across CI, OAuth, Azure, Cloudflare, Supabase, and onboarding. I'll keep each scoped to a clear deliverable, while noting dependencies where infrastructure choices overlap.

- Planned sprint issues

- Choosing six setup issues

I'm treating the work as infrastructure discovery, not product-flow implementation. My six recommendations are: CI baseline, Cloudflare MkDocs deployment, Supabase PostgreSQL setup, OAuth/OIDC decision spike, Azure backend deployment, and onboarding/technology documentation.

- Selected six setup issues

- Inspected repository structure, packages, CI, sources, and documentation

- Planned and designed the sport analytics project with cited requirements

### Searching the web

[learn.microsoft.com](https://learn.microsoft.com)

[supabase.com](https://supabase.com)

[developers.cloudflare.com](https://developers.cloudflare.com)

[docs.gitea.com](https://docs.gitea.com)

- Searched 4 websites

✓ Worked for 1m 34s

Yes - **CI should definitely be one of the six**. The best next step is six parallel **foundation issues**, one per team member, without deciding the sport-specific product flow yet.

These six align well with Sprint 1, which focuses on project setup, removing major blockers and establishing a strong direction. The rubric also rewards CI/CD, a public documentation site, getting-started guides, a motivated technology stack and supporting implementation. `project_briefs.pdf`

## Recommended split

| Person | Issue                                                              |
|--------|--------------------------------------------------------------------|
| 1      | Restore green CI and reproducible installation                     |
| 2      | Deploy the public documentation site with Cloudflare Pages         |
| 3      | Validate developer onboarding and document the technology stack    |
| 4      | Establish the Supabase PostgreSQL foundation                       |
| 5      | Investigate and establish the OAuth/OIDC authentication foundation |
| 6      | Establish the Azure backend deployment foundation                  |

These are all suitable for Sprint 1 and can be completed without finalising the detailed event flow.

## 1. Restore green CI and reproducible installation

### Suggested title

Restore green CI and reproducible installation

### Suggested branch

ci/<issue-number>-restore-green-ci

### Labels

type: infrastructure  
area: infrastructure  
priority: high

### Purpose

Fix the current queued Gitea Actions workflow and prove that the repository can be installed, tested and built consistently.

The current workflow uses `runs-on: ubuntu-latest`. Gitea Actions requires an active runner, and the runner must have a label matching the workflow's `runs-on` value. ([Gitea Documentation](#))

### Acceptance criteria

- [ ] The reason the current CI jobs remain queued is confirmed
- [ ] The available Gitea runner labels are confirmed with the Wits administrator
- [ ] ``.gitea/workflows/ci.yml`` uses a supported runner label
- [ ] A root ``.package-lock.json`` is generated and committed
- [ ] CI uses ``npm ci`` rather than ``npm install``
- [ ] ``npm run check`` passes locally from a clean installation
- [ ] Frontend tests pass
- [ ] Backend tests pass
- [ ] Type checking, linting and builds pass
- [ ] At least one Pull Request receives a successful CI result
- [ ] The CI process is documented
- [ ] Required status checks are enabled on ``main`` only after a successful run exists

This should be treated as the highest-priority technical issue because other Pull Requests need trustworthy automated checks.

## 2. Deploy the public MkDocs documentation site

### Suggested title

Deploy the public documentation site with Cloudflare Pages

### Suggested branch

docs/<issue-number>-deploy-public-documentation

### Labels

type: infrastructure  
area: documentation  
priority: high

### Purpose

Deploy the existing MkDocs documentation to a website that can be accessed without a Wits or Gitea login.

The course explicitly requires a publicly available documentation website and identifies static hosting such as Cloudflare Pages as an appropriate option. [project\\_briefs.pdf](#) Cloudflare Pages supports deploying prebuilt static output through direct upload as well as supported Git integrations, so the team can choose a deployment route compatible with the private Gitea repository. ([Cloudflare Docs](#))

### Acceptance criteria

- [ ] `mkdocs build --strict` completes successfully
- [ ] The generated static site is deployed to Cloudflare Pages
- [ ] The site is accessible without a Wits or Gitea login
- [ ] The Git Methodology page is accessible
- [ ] The Project Methodology page is accessible
- [ ] The setup, testing, architecture, API, database and security pages are accessible
- [ ] All navigation links work
- [ ] The public URL is added to `README.md`
- [ ] The chosen deployment process is documented
- [ ] The documentation states clearly what is implemented and what remains planned
- [ ] No secrets or private information are published

Because Gitea is private, the person should investigate whether to use a direct upload, Wrangler or a Gitea Actions deployment rather than assuming Cloudflare can connect directly to the repository.

## 3. Validate developer onboarding and document the technology stack

### Suggested title

Validate developer onboarding and motivate the technology stack

### Suggested branch

docs/<issue-number>-validate-onboarding-and-stack

### Labels

type: documentation  
area: documentation

### Purpose

Ensure that every team member can clone, install and run the repository without relying on Dean or undocumented instructions.

This issue also directly supports the Sprint 1 criteria for getting-started guides and a fully listed, motivated technology stack. [project\\_briefs.pdf](#)

### Acceptance criteria

- [ ] The setup process is tested from a clean clone
- [ ] All required software and versions are documented
- [ ] Backend environment variables are documented
- [ ] Frontend environment variables are documented
- [ ] Commands for running the frontend and backend are verified
- [ ] Commands for testing, linting, type checking and building are verified
- [ ] `apps/frontend/README.md` is created
- [ ] `apps/backend/README.md` is created
- [ ] The contracts package instructions are verified

- [ ] Common setup problems and solutions are documented
- [ ] A technology-stack page lists all major tools and services
- [ ] Each major technology includes its purpose and motivation
- [ ] Relevant alternatives are briefly recorded
- [ ] The new guides are linked from the README and MkDocs navigation

## Dependency

This person can begin immediately, but the issue should only close once Issue 1 confirms the final clean-install and CI commands.

## 4. Establish the Supabase PostgreSQL foundation

### Suggested title

Establish the Supabase PostgreSQL foundation

### Suggested branch

chore/<issue-number>-establish-supabase-database

### Labels

type: infrastructure  
area: database  
priority: high

## Purpose

Create the hosted PostgreSQL development database and prove that the hand-written Express backend can connect to it securely.

Supabase supports direct PostgreSQL connections from server applications. ([Supabase](#)) However, the project brief prohibits relying on generated Supabase endpoints as the project API. The frontend must continue to access data through your own hand-written backend HTTP API. [project\\_briefs.pdf](#)

## Acceptance criteria

- [ ] A shared Supabase development project is created
- [ ] Access is granted only to required team members
- [ ] The backend connects using `DATABASE\_URL`
- [ ] The database credentials are not committed
- [ ] `.env.example` contains safe placeholder configuration
- [ ] A minimal connection check succeeds from the backend
- [ ] The frontend does not connect directly to Supabase
- [ ] Supabase-generated data endpoints are not used as the project API
- [ ] A migration tool or migration approach is selected and documented
- [ ] An initial harmless migration is successfully applied
- [ ] Connection and security decisions are recorded in an ADR
- [ ] Local and hosted database setup instructions are documented

Do **not** create the full sports schema yet. A simple connection test or neutral table such as a migration metadata table is enough for this issue.

## 5. Investigate and establish the OAuth/OIDC authentication foundation

### Suggested title

Investigate and establish the OAuth/OIDC authentication foundation

### Suggested branch

chore/<issue-number>-establish-auth-foundation

### Labels

type: research  
area: infrastructure  
priority: high

### Purpose

Select an established authentication provider and prove that the backend can validate an authenticated identity without designing the final user roles or interface yet.

The project requires sign-up, sign-in, password reset and account deletion, and it explicitly prohibits the team from writing its own authentication system. [project\\_briefs.pdf](#)

### Acceptance criteria

- [ ] At least two suitable managed authentication options are compared
- [ ] The comparison considers sign-up, sign-in, password reset and account deletion
- [ ] OAuth 2.0 and OpenID Connect support are considered
- [ ] Frontend and backend integration approaches are compared
- [ ] Cost and student-access limitations are recorded
- [ ] Local-development support is considered
- [ ] A provider is selected and motivated in an ADR
- [ ] A development tenant or project is created
- [ ] Secrets and client credentials are not committed
- [ ] The backend can validate a test identity or token
- [ ] An unauthenticated request can be rejected by a protected proof-of-concept endpoint
- [ ] The work does not yet define final user roles or sport-specific permissions
- [ ] Authentication setup is documented

Keep this as a **foundation spike**, not the complete account-management feature.

## 6. Establish the Azure backend deployment foundation

### Suggested title

Establish the Azure backend deployment foundation

### Suggested branch

chore/<issue-number>-deploy-backend-to-azure

### Labels

type: infrastructure  
area: infrastructure  
priority: high

### Purpose

Deploy the existing Express backend health endpoint to Azure and document a repeatable deployment process.

Azure App Service supports deployment of Node.js and Express web applications, making it a reasonable first deployment target for the existing backend scaffold. ([Microsoft Learn](#))

## Acceptance criteria

- [ ] The appropriate Azure hosting option is selected and motivated
- [ ] A non-production Azure resource is created
- [ ] The Express backend builds successfully for deployment
- [ ] The application uses Azure-provided environment configuration
- [ ] No secrets are committed to the repository
- [ ] The deployed `/api/v1/health` endpoint returns HTTP 200
- [ ] Production CORS configuration is documented
- [ ] Application logs can be viewed
- [ ] The deployment process is repeatable
- [ ] The public backend URL is recorded in the deployment documentation
- [ ] Expected cost or free-tier limitations are documented
- [ ] No sport-specific endpoints are implemented as part of this issue

This issue proves the deployment path without prematurely deciding the actual product flow.

## Recommended order

Start all six, but prioritise them like this:

1. **CI and reproducible installation**
2. **Public documentation**
3. **Developer onboarding**
4. **Supabase**
5. **Authentication**
6. **Azure deployment**

The first three provide the strongest immediate Sprint 1 rubric evidence. The remaining three remove major technical blockers and establish a credible direction for later implementation.

Each issue should be assigned to:

Milestone: Sprint 1  
Project: Sport-Analytics-Tool-Proj  
Column: Ready

Each description should end with:

## AI Declaration

The preceding issue was planned and generated with the assistance of ChatGPT-Web[GPT-5.6 Thinking].

## Sources

[Quick Start | Gitea Documentation](#) docs.gitea.com

[Overview · Cloudflare Pages docs](#) developers.cloudflare.com

[Connection management | Supabase Docs](#) supabase.com

[Quickstart: Create a Node.js Web App - Azure App Service | Microsoft Learn](#) learn.microsoft.com

8/5/2026, 12:35:56 PM



do we need a gitwiki if we using a docs file? how do we display it from the repo

8/5/2026, 12:42:17 PM



## Thinking

- 🌐 **Searching the web**
    - [docs.gitea.com](https://docs.gitea.com)
  - **Searched 1 website**
  - **Clarifying documentation flow**

I'm separating the repository source from the public website: Markdown files in `docs/` render inside Gitea, while MkDocs builds them into a public static site. I'll recommend Cloudflare Pages for deployment and treating the Wiki as optional, not authoritative.
  - **Clarified documentation flow**
  - **Evaluated repository documentation and public website requirements**
  - 🌐 **Searching the web**
    - [docs.gitea.com](https://docs.gitea.com)
  - **Searched 1 website**
- ✓ Worked for 31s

No, you do **not** need to use the Gitea Wiki.

Using both the Wiki and the `docs/` directory would duplicate content and make it unclear which version is authoritative. Keep your documentation as version-controlled Markdown files in the main repository, then publish those files as an MkDocs website.

## Recommended structure

Use this as your documentation flow:

```
Repository Markdown files
 ↓
 MkDocs
 ↓
Public documentation website
```

Your repository remains the source of truth:

```
docs/
├─ index.md
├─ git-methodology.md
├─ project-methodology.md
├─ setup.md
├─ architecture.md
├─ testing.md
└─ ...
```

The project brief specifically requires a **publicly available documentation website** and suggests Markdown-based tools such as MkDocs deployed through static hosting such as Cloudflare Pages. Repository files alone are therefore useful, but they do not fully satisfy the requirement while your Gitea repository is internal. `project_briefs.pdf`

Gitea itself does not provide a built-in Pages service, so the generated MkDocs site needs to be hosted separately, such as on Cloudflare Pages. ([Gitea Documentation](#))

## How the documentation appears inside Gitea

Gitea automatically renders `.md` files when someone clicks them in the repository. ([Gitea Documentation](#))

From the repository, a user can:

1. Open **Code**.
2. Click the `docs` folder.
3. Click a Markdown document.
4. Read the rendered page.

You should also link the important pages from the root `README.md`.

For example:

```
Documentation

The project documentation is maintained in the [``docs``](docs/) directory.

- [Documentation Home](docs/index.md)
- [Git Methodology](docs/git-methodology.md)
- [Project Methodology](docs/project-methodology.md)

Public Documentation Website

The publicly accessible documentation site is available at:

[Open the Sport Analytics Tool Documentation](INSERT-CLOUDFLARE-URL-HERE)
```

## Add a documentation home page

Create:

```
docs/index.md
```

A simple version could be:

```
Sport Analytics Tool Documentation

This website contains the technical and project documentation for the Sport Analytics Tool.

Project Management

- [Git Methodology](git-methodology.md)
- [Project Methodology](project-methodology.md)

Development

- [Getting Started](getting-started.md)
- [Technology Stack](technology-stack.md)
- [Testing](testing.md)

System Design

- [Architecture](architecture.md)
- [API Design](api.md)
- [Database Design](database.md)
```

```
Deployment and Operations
```

- [Deployment](deployment.md)
- [Security](security.md)

```
Project Evidence
```

Project-development evidence is maintained in the repository's `evidence` directory.

## How MkDocs displays it

Your `mkdocs.yml` determines the website navigation. For example:

```
site_name: Sport Analytics Tool Documentation

nav:
 - Home: index.md
 - Methodologies:
 - Git Methodology: git-methodology.md
 - Project Methodology: project-methodology.md
 - Development:
 - Getting Started: getting-started.md
 - Technology Stack: technology-stack.md
 - Testing: testing.md
 - Design:
 - Architecture: architecture.md
 - API: api.md
 - Database: database.md
 - Deployment:
 - Deployment Guide: deployment.md
 - Security: security.md
```

MkDocs reads these files from `docs/` and produces a styled static website.

## What to do with the Wiki tab

You can simply leave it unused. Since you now have Admin access, you may also disable the Wiki repository unit in the repository settings to avoid confusion; Gitea treats Wiki as a separate repository feature. ([Gitea Documentation](#))

Your final arrangement should therefore be:

- **Gitea Wiki:** not used.
- `docs/` **folder:** authoritative documentation source.
- **Gitea rendering:** convenient internal viewing and review.
- **MkDocs on Cloudflare Pages:** public documentation website required for the project.
- **README:** links to both the source documentation and public website.

### Sources

[FAQ | Gitea Documentation](#) docs.gitea.com

8/5/2026, 12:42:19 PM



ok, so give me complete issues for the 6 things and how to make them etc and then each teammember will chose one, be clear etc and include ai etc

8/5/2026, 1:38:23 PM



## Thinking

### Inspecting Repository Structure for Issues

- **Organizing issue set**

I'm preparing six copy-paste-ready Git issues covering CI, OAuth, Supabase, Cloudflare, onboarding, and AI governance. Each will include scope, acceptance criteria, labels, ownership, and branch guidance.

- **Organized issue set**

- **Inspected repository structure, files, configuration, CI, and documentation**

- **Planning six Git issues**

I'm structuring six independent Sprint 1 tasks, keeping them unassigned in Ready until someone claims one. Each issue will include scope, acceptance criteria, evidence, labels, branch naming, and AI documentation guidance.

- **Planned six Git issues**

- **Created HexGrid foundation setup and Sprint 1 issues**

✓ Worked for 2m 17s

These six issues are suitable for **parallel Sprint 1 foundation work**. They deliberately avoid deciding the sport-specific event flow, statistics or user roles too early. Together they address the major setup blockers: CI/CD, public documentation, onboarding, database hosting, authentication and deployment. Sprint 1 is specifically intended for project setup, removing major blockers and establishing a strong direction. The overall brief also requires CI/CD, established authentication, a hand-written API and a publicly accessible documentation site. `project_briefs.pdf`

## How to create the six issues

Create all six now, but initially leave them **unassigned** so each team member can choose one.

For each issue:

1. Go to **Issues → New Issue**.
2. Choose the **Task** template.
3. Paste the title and Markdown body supplied below.
4. Set:
  - **Milestone:** `Sprint 1`
  - **Project:** `Sport-Analytics-Tool-Proj`
  - **Column:** `Ready`
  - the labels listed for that issue
5. Leave **Assignee** empty.
6. Once a team member chooses the issue, they assign it to themselves and move it from `Ready` to `In Progress`.

Your methodology requires issues to have a clear purpose, acceptance criteria, dependencies, evidence, an assignee and a sprint milestone. It also expects the Project board and linked Pull Requests to demonstrate that the process was actually followed. `Pasted markdown.md`

# Issue 1: Restore green CI and reproducible installation

## Title

Restore green CI and reproducible installation

## Labels

type: infrastructure  
area: infrastructure  
priority: high

## Branch after the issue is claimed

ci/<issue-number>-restore-green-ci

## Issue body

### ## Description

Investigate the current queued Gitea Actions workflow and establish a reproducible installation, test and build process for the repository.

The existing workflow must be checked against the available Wits Gitea Actions runner configuration. The local and automated processes must use the same dependency installation and repository checks where possible.

### ## Motivation

The repository currently contains a CI workflow, but the workflow remains queued and has not produced a successful result.

A reliable CI process is necessary to detect build, test, linting, formatting and type-checking problems before changes are merged into `main`. A reproducible installation is also required so that all team members and the CI runner use the same dependency versions.

### ## Scope

#### ### Included

- investigating why the current CI workflow remains queued;
- confirming which runner labels are available;
- establishing a reproducible dependency installation;
- verifying the existing repository check scripts;
- updating the CI workflow where necessary;
- documenting the CI process and any external blocker.

#### ### Excluded

- application deployment to Azure;
- Cloudflare Pages deployment;
- product-specific automated tests;
- sport-specific implementation.

### ## Acceptance Criteria

- [ ] The reason the current Gitea Actions workflow remains queued is confirmed
- [ ] The available or required runner label is documented
- [ ] Any server-side runner dependency is recorded clearly
- [ ] A root `package-lock.json` exists and is committed
- [ ] A clean installation succeeds using the committed lockfile

- [ ] CI uses `npm ci` rather than `npm install`
- [ ] `npm run check` succeeds locally from a clean installation
- [ ] The frontend tests pass
- [ ] The backend tests pass
- [ ] Linting, formatting, type-checking and production builds pass
- [ ] `.gitea/workflows/ci.yml` is updated where required
- [ ] The CI process is documented in the repository
- [ ] At least one successful CI run is recorded, unless the Wits server has no available runner
- [ ] If a server-side runner is unavailable, the issue is moved to `Blocked` with the required blocker information
- [ ] Required status checks are only enabled on `main` after a successful CI run is available
- [ ] No unrelated application features are added

### ## Dependencies

Access to, or information from, the Wits Gitea administrator may be required to confirm runner availability and supported runner labels.

### ## Evidence

Evidence must include:

- the updated CI workflow;
- the committed lockfile;
- local `npm run check` results;
- the Gitea Actions result or screenshot;
- CI documentation;
- any administrator response or documented blocker;
- the linked Pull Request.

### ## AI Declaration

The preceding issue was planned and generated with the assistance of ChatGPT-Web[GPT-5.6 Thinking].

## Issue 2: Deploy the public documentation site with Cloudflare Pages

### Title

Deploy the public documentation site with Cloudflare Pages

### Labels

type: infrastructure  
area: documentation  
priority: high

### Branch

docs/<issue-number>-deploy-public-documentation

### Issue body

#### ## Description

Build and deploy the existing MkDocs documentation as a publicly accessible website using Cloudflare Pages.

The deployment method must work with the private internal Gitea repository and must not require members of the public to sign into Wits or Gitea.

#### ## Motivation

The project brief requires a publicly available documentation website. The repository already contains Markdown documentation and an MkDocs configuration, but repository files alone do not satisfy the public-access requirement while the Gitea repository remains internal.

The deployed website will provide a readable and accessible version of the project documentation while the repository remains the authoritative source.

## ## Scope

### ### Included

- verifying the existing MkDocs configuration;
- building the documentation locally;
- selecting a deployment method compatible with private Gitea;
- deploying the generated static website to Cloudflare Pages;
- documenting the deployment and update process;
- linking the public website from the root README.

### ### Excluded

- using the Gitea Wiki;
- deploying the frontend application;
- deploying the backend application;
- publishing secrets, private evidence or internal credentials.

## ## Acceptance Criteria

- [ ] `mkdocs build --strict` completes successfully
- [ ] All configured MkDocs navigation entries resolve correctly
- [ ] The Git Methodology page is accessible
- [ ] The Project Methodology page is accessible
- [ ] The architecture, API, database, setup, testing, deployment, security and AI pages are accessible
- [ ] The site is deployed to Cloudflare Pages
- [ ] The website can be opened without a Wits or Gitea account
- [ ] The deployed website contains no credentials, secrets or private information
- [ ] The deployment method works with the private Gitea repository
- [ ] The public documentation URL is added to `README.md`
- [ ] Deployment and update instructions are documented
- [ ] The documentation distinguishes implemented functionality from planned functionality
- [ ] The Gitea Wiki is not used as a duplicate documentation source
- [ ] No unrelated product functionality is added

## ## Dependencies

A Cloudflare account or team space must be available.

The CI issue is not a strict dependency. A manual or direct static-site deployment may be used initially if Gitea Actions remains unavailable.

## ## Evidence

Evidence must include:

- the public Cloudflare Pages URL;
- a successful strict MkDocs build;
- screenshots of the deployed site;
- updated README links;
- deployment documentation;
- the linked Pull Request.

## ## AI Declaration

The preceding issue was planned and generated with the assistance of ChatGPT-Web[GPT-5.6 Thinking].

The course brief specifically suggests Markdown documentation tools such as MkDocs and static hosting such as Cloudflare Pages. `project_briefs.pdf`

# Issue 3: Validate developer onboarding and motivate the technology stack

## Title

Validate developer onboarding and motivate the technology stack

## Labels

type: documentation  
area: documentation

## Branch

docs/<issue-number>-validate-onboarding-and-stack

## Issue body

### ## Description

Test the complete developer setup process from a clean clone and improve the onboarding and technology-stack documentation.

The final documentation must allow a team member to install dependencies, configure the environment, run the frontend and backend, execute the repository checks and understand why each major technology was selected.

### ## Motivation

The project must be maintainable by the full team rather than only by the person who created the initial scaffold.

Clear onboarding instructions reduce setup delays and provide evidence for the Sprint 1 getting-started and technology-stack criteria. The technology choices must also be motivated rather than merely listed.

### ## Scope

#### ### Included

- testing the setup process from a clean clone;
- checking prerequisite versions;
- validating all documented commands;
- documenting frontend, backend, contracts and documentation setup;
- documenting required environment variables;
- documenting common setup problems;
- listing and motivating the project technology stack.

#### ### Excluded

- changing the selected frontend or backend frameworks without a separate decision;
- configuring Supabase;
- configuring authentication;
- deploying to Azure or Cloudflare;
- implementing product features.



### ## Acceptance Criteria

- [ ] The repository setup is tested from a clean clone
- [ ] Required Node.js, npm and Python versions are documented
- [ ] Dependency installation instructions are verified
- [ ] Backend environment variables are documented
- [ ] Frontend environment variables are documented
- [ ] Commands for running the frontend are verified
- [ ] Commands for running the backend are verified
- [ ] Commands for testing, linting, formatting, type-checking and building are verified
- [ ] `apps/frontend/README.md` is created or completed
- [ ] `apps/backend/README.md` is created or completed
- [ ] The `packages/contracts` setup instructions are verified
- [ ] MkDocs local setup instructions are verified
- [ ] Common setup problems and solutions are documented
- [ ] A technology-stack document lists every major framework, library, language and hosting service currently selected
- [ ] Each major technology includes its purpose and motivation
- [ ] Important alternatives considered are recorded where appropriate
- [ ] The new documentation is linked from the root README
- [ ] The new documentation is included in the MkDocs navigation
- [ ] A second team member follows the guide and records whether it works
- [ ] No unrelated product implementation is added

### ## Dependencies

The issue may begin immediately.

It may only be closed once the CI and reproducible-installation issue confirms the final dependency installation and checking commands.

### ## Evidence

Evidence must include:

- the clean-clone verification record;
- updated onboarding documentation;
- frontend and backend READMEs;
- the technology-stack document;
- second-team-member verification;
- updated MkDocs navigation;
- the linked Pull Request.

### ## AI Declaration

The preceding issue was planned and generated with the assistance of ChatGPT-Web[GPT-5.6 Thinking].

## Issue 4: Establish the Supabase PostgreSQL foundation

### Title

Establish the Supabase PostgreSQL foundation

### Labels

type: infrastructure  
area: database  
priority: high

## Branch

chore/<issue-number>-establish-supabase-database

## Issue body

### ## Description

Create and document the hosted Supabase PostgreSQL development foundation for the project.

The Express backend must be able to connect securely to the hosted PostgreSQL database. The frontend must not connect directly to Supabase, and Supabase-generated API endpoints must not be used as the project API.

### ## Motivation

The project requires a reliable database foundation before sport-specific event, fixture and statistic schemas are designed.

Supabase is being considered as the managed PostgreSQL host. However, the project must continue to use the team's hand-written backend HTTP API rather than generated database endpoints.

### ## Scope

#### ### Included

- creating a shared Supabase development project;
- configuring secure server-side database access;
- proving that the backend can connect;
- selecting and configuring a migration approach;
- documenting local and hosted database setup;
- recording the database-hosting decision.

#### ### Excluded

- designing the final sport-specific schema;
- creating event, fixture, athlete or statistic tables;
- using Supabase-generated REST or GraphQL endpoints as the application API;
- connecting the frontend directly to Supabase;
- implementing authentication through Supabase unless separately approved in the authentication issue.

### ## Acceptance Criteria

- [ ] A Supabase development project is created
- [ ] Only required team members have access
- [ ] Database credentials are stored outside the repository
- [ ] No secret or real connection string is committed
- [ ] `apps/backend/.env.example` contains safe placeholder database configuration
- [ ] The Express backend connects using `DATABASE\_URL`
- [ ] A repeatable backend connection-check command or test is provided
- [ ] The connection check succeeds against the hosted development database
- [ ] The frontend has no direct Supabase database connection
- [ ] Supabase-generated API endpoints are not used as the project API
- [ ] At least two migration approaches are briefly considered
- [ ] A migration approach is selected and motivated
- [ ] The migration approach stores migrations in version control
- [ ] A non-sport-specific baseline migration is applied successfully
- [ ] The database-hosting and migration decision is recorded in an ADR
- [ ] Local and hosted database setup instructions are documented
- [ ] Security and credential-handling guidance is documented
- [ ] No sport-specific product schema is introduced

### ## Dependencies

A shared Supabase account or project must be available.

The final sport-specific schema depends on later stakeholder and product-design decisions and is outside this issue.

### ## Evidence

Evidence must include:

- the database ADR;
- safe environment examples;
- the migration configuration;
- the version-controlled baseline migration;
- successful connection-test output;
- database setup documentation;
- confirmation that no credentials were committed;
- the linked Pull Request.

### ## AI Declaration

The preceding issue was planned and generated with the assistance of ChatGPT-Web[GPT-5.6 Thinking].

The project brief allows a managed database service, but the team must design and implement its own HTTP API rather than relying on generated Supabase endpoints. `project_briefs.pdf`

## Issue 5: Investigate and establish the OAuth/OIDC authentication foundation

### Title

Investigate and establish the OAuth/OIDC authentication foundation

### Labels

type: research  
area: infrastructure  
priority: high

### Branch

chore/<issue-number>-establish-auth-foundation

### Issue body

#### ## Description

Investigate suitable managed authentication providers and establish a minimal OAuth 2.0 and OpenID Connect authentication foundation.

The work must select an established provider, configure a development application or tenant, and prove that the backend can validate an authenticated identity. It must not define the final sport-specific roles or complete application account flow.

#### ## Motivation

The project requires users to sign up, sign in, reset passwords and delete accounts. The project team may not build its own authentication system.

Selecting and proving the authentication foundation early removes a major technical and security blocker without prematurely defining the final product roles and permissions.

## ## Scope

### ### Included

- comparing suitable established authentication providers;
- evaluating required account lifecycle capabilities;
- considering OAuth 2.0 and OpenID Connect support;
- selecting and motivating a provider;
- creating a development application or tenant;
- configuring safe environment placeholders;
- validating a development token in the backend;
- protecting one proof-of-concept endpoint;
- documenting the authentication architecture.

### ### Excluded

- defining final application roles;
- defining approved submitter permissions;
- implementing final sign-up and sign-in pages;
- implementing all account-management screens;
- building a custom password or session system;
- implementing sport-specific authorisation rules.

## ## Acceptance Criteria

- [ ] At least two suitable established authentication providers are compared
- [ ] The comparison covers sign-up, sign-in, password reset and account deletion
- [ ] OAuth 2.0 and OpenID Connect support are considered
- [ ] Frontend and backend integration approaches are considered
- [ ] Cost, free-tier and student-access limitations are recorded
- [ ] Local-development support is considered
- [ ] Security and secret-management requirements are considered
- [ ] One provider is selected and motivated in an ADR
- [ ] A development application, tenant or project is created
- [ ] Client secrets and credentials are not committed
- [ ] Safe placeholders are added to the relevant `.env.example` files
- [ ] The backend can validate a development identity or token
- [ ] One proof-of-concept protected endpoint is added
- [ ] An unauthenticated request receives an appropriate rejection response
- [ ] An authenticated test request succeeds
- [ ] Automated tests cover the protected endpoint where practical
- [ ] Authentication setup and architecture are documented
- [ ] The work does not define final product roles or sport-specific permissions

## ## Dependencies

Access to the selected authentication provider is required.

Final roles and authorisation rules depend on future stakeholder and product-flow decisions.

## ## Evidence

Evidence must include:

- the provider comparison;
- the authentication ADR;

- safe environment examples;
- the proof-of-concept backend protection;
- test results;
- setup and architecture documentation;
- confirmation that no secrets were committed;
- the linked Pull Request.

### ## AI Declaration

The preceding issue was planned and generated with the assistance of ChatGPT-Web[GPT-5.6 Thinking].

The project requires established authentication rather than a system written by the team. `project_briefs.pdf`

## Issue 6: Establish the Azure backend deployment foundation

### Title

Establish the Azure backend deployment foundation

### Labels

type: infrastructure  
area: infrastructure  
priority: high

### Branch

chore/<issue-number>-deploy-backend-to-azure

### Issue body

#### ## Description

Investigate the suitable Azure hosting options and deploy the existing Express backend foundation to a non-production Azure environment.

The deployed application must expose the current health endpoint and use environment-based configuration. This issue establishes the deployment path only and must not add sport-specific functionality.

#### ## Motivation

The project requires deployment and CI/CD processes. Establishing the backend deployment path early removes uncertainty around hosting, environment configuration, logs and public service availability.

A successful health-endpoint deployment proves that the current backend can be built and hosted before database, authentication and product-specific functionality are added.

#### ## Scope

#### ### Included

- comparing suitable Azure hosting options;
- selecting and motivating one Azure service;
- creating a non-production Azure resource;
- deploying the current Express backend;
- configuring environment variables securely;
- verifying the public health endpoint;
- documenting logs, costs and repeatable deployment steps;
- recording the deployment decision.

### ### Excluded

- deploying the frontend application;
- deploying the MkDocs site;
- configuring the final production domain;
- implementing sport-specific endpoints;
- requiring the Supabase or authentication foundations to be complete;
- creating a full production environment.

### ## Acceptance Criteria

- [ ] At least two suitable Azure hosting options are briefly compared
- [ ] One hosting option is selected and motivated in an ADR
- [ ] Expected free-tier or cost limitations are documented
- [ ] A non-production Azure resource is created
- [ ] The backend production build succeeds
- [ ] The backend is deployed successfully
- [ ] The deployed `/api/v1/health` endpoint returns HTTP 200
- [ ] The deployed service uses environment-based configuration
- [ ] No Azure credentials or secrets are committed
- [ ] Production or hosted CORS behaviour is considered and documented
- [ ] Application logs can be accessed
- [ ] The deployment can be repeated using documented steps
- [ ] The public backend URL is recorded in the deployment documentation
- [ ] A rollback or redeployment approach is documented
- [ ] The future CI/CD automation step is documented
- [ ] No sport-specific endpoint or feature is added

### ## Dependencies

An Azure subscription or approved student Azure access is required.

A manual deployment may be used for the initial foundation if the Gitea Actions runner is not yet available. Automated deployment can be completed once CI infrastructure is working.

### ## Evidence

Evidence must include:

- the Azure hosting ADR;
- the public health-endpoint result;
- deployment logs or screenshots;
- secure configuration evidence;
- repeatable deployment instructions;
- cost or free-tier notes;
- confirmation that no secrets were committed;
- the linked Pull Request.

### ## AI Declaration

The preceding issue was planned and generated with the assistance of ChatGPT-Web[GPT-5.6 Thinking].

## When a team member chooses an issue

The team member should:

1. Assign the issue to themselves.
2. Move it from `Ready` to `In Progress`.

3. Comment:

```
I am taking responsibility for this issue.
```

```
I will update the issue if I become blocked or if the scope needs clarification.
```

4. Update local `main` and create the issue branch:

```
git switch main
git pull origin main
git switch -c <branch-name>
```

Example, if the CI issue becomes Issue `#10`:

```
git switch -c ci/10-restore-green-ci
```

5. Make small, atomic commits that reference the issue:

```
git commit -m "ci: use reproducible dependency installation" -m "Refs #10"
```

6. Where AI-generated code or configuration is included in a commit, add:

```
git commit -m "ci: use reproducible dependency installation" -m "Refs #10" -m "Assisted-by: ChatGPT-Web[GPT-5.6 Thinking]"
```

7. Push the branch:

```
git push -u origin <branch-name>
```

8. Open a Pull Request:

- target: `main`
- source: the issue branch
- title: `#<issue-number> <issue title>`
- body: complete the repository Pull Request template
- related issue: `Closes #<issue-number>`

9. Move the issue to `In Review`.

10. Obtain approval from another team member.

11. Merge using a merge commit.

12. Delete the branch.

13. Confirm the issue closes.

14. Move it to `Done`.

That follows the agreed traceability chain:

```
Issue → Branch → Commits → Pull Request → Review → Merge
```

Your Git Methodology requires branches and commits to reference issues, atomic commits, peer-reviewed Pull Requests and closure through the PR. `Git_Methodology.md`

## AI requirements for all six team members

The AI declarations included in the issue bodies disclose that **these issue descriptions** were drafted using this ChatGPT session.

Each assignee must additionally:

- change the declaration if they use another tool or model;
- record their actual AI usage in `evidence/ai/ai-usage-register.csv` ;
- keep the relevant transcript or saved session;
- add `Assisted-by` to commits containing AI-generated code or configuration;
- ensure any AI writing placed in a submitted document has a document-level declaration;
- review and verify all generated work before submitting it.

The policy requires the tool, model and purpose to be recorded. It requires document declarations for AI-assisted writing and `Assisted-by` attribution in commits containing AI-generated code. `ai_policy_20_jul.pdf`

*8/5/2026, 1:38:25 PM*



for each are they features, bgs, tasks etc?

8/5/2026, 1:45:06 PM



Use the **Task** template for all six. None of them is a product feature yet, and none is strictly a bug report.

| Issue                                                 | Template    | Main label           | Area label           |
|-------------------------------------------------------|-------------|----------------------|----------------------|
| Restore green CI and reproducible installation        | <b>Task</b> | type: infrastructure | area: infrastructure |
| Deploy the public documentation site                  | <b>Task</b> | type: infrastructure | area: documentation  |
| Validate onboarding and motivate the technology stack | <b>Task</b> | type: documentation  | area: documentation  |
| Establish the Supabase PostgreSQL foundation          | <b>Task</b> | type: infrastructure | area: database       |
| Investigate OAuth/OIDC authentication                 | <b>Task</b> | type: research       | area: infrastructure |
| Establish the Azure backend deployment foundation     | <b>Task</b> | type: infrastructure | area: infrastructure |

## Why not Feature?

A **Feature** issue should implement a product capability that a user or external consumer can use, such as:

- submit fixture events;
- view player statistics;
- export a dataset;
- manage approved submitters.

These six issues are project foundations and technical setup rather than finished user-facing capabilities.

## Why not Bug?

A **Bug** issue should describe existing behaviour that is incorrect and reproducible.

The CI workflow currently being queued may eventually reveal a specific bug, but the current issue is broader: investigate the runner, create the lockfile, verify installation and establish working CI. That makes it a **Task**.

If the investigation later proves that the workflow contains a specific defect, such as an invalid runner label, you may either fix it within the existing task or create a separate Bug issue if it is substantial.

## What each member chooses in Gitea

For every one of the six:

1. Click **New Issue**.
2. Select **Task**.
3. Paste the relevant issue description.
4. Add the labels shown above.

5. Set:

- Milestone: Sprint 1
- Project: Sport-Analytics-Tool-Proj
- Column: Ready

6. Leave it unassigned until a member chooses it.

8/5/2026, 1:45:19 PM



is Ci a clever one for me to take?

8/5/2026, 1:53:01 PM



Yes, **CI is a clever issue to take**, but it is not the easiest one.

It is valuable because it:

- directly supports the Sprint 1 rubric;
- affects every future Pull Request;
- gives you visible technical evidence;
- helps prove the existing scaffold actually installs, tests and builds;
- can lead to required status checks on `main`.

However, it carries more risk than the documentation or onboarding issues because the current workflow may depend on a **Wits-hosted Gitea Actions runner**. If the runner is unavailable, part of the issue will depend on an administrator rather than on your own work.

#### **CI is a good choice for you if:**

- you are comfortable running `npm` commands and reading terminal errors;
- you are willing to edit a YAML workflow;
- you want a meaningful technical contribution;
- you do not mind contacting the Wits administrator about the runner;
- you are prepared to document a blocker if the server configuration prevents completion.

#### **It may not be the best choice if:**

- you want a straightforward issue that you can complete independently;
- you are still becoming comfortable with Git, PowerShell and repository workflows;
- you do not want your progress to depend on an external administrator.

Based on the work you have been doing so far, **developer onboarding and technology-stack documentation would be the safest fit**, because you already understand the repository methodology and can test instructions step by step. It is still a strong Sprint 1 contribution.

My recommendation:

- **Take CI** if you want to stretch yourself technically and another technical team member can assist when necessary.
- **Take onboarding and technology stack** if you want a lower-risk issue that you can own confidently from beginning to end.

CI is probably the more impressive issue, but onboarding is more likely to be completed cleanly and independently.

8/5/2026, 1:53:17 PM



k, im gonna do CI, are you ready to help me?

8/5/2026, 1:54:16 PM



Yes — ready.

We'll do it slowly, one step at a time, and verify each command before moving on.

First:

1. Assign the CI issue to yourself.
2. Move it from **Ready** to **In Progress**.
3. Send me the issue number and a screenshot of the current CI workflow page.

Then we'll create the branch and diagnose why the workflow is queued before changing anything.

8/5/2026, 1:54:25 PM



Sport-Analytics-Tool (2).zip

cant find ci?

8/5/2026, 1:57:35 PM



## Thinking



### Locating CI Workflow Files

- Inspected CI workflow files and configuration

✓ Worked for 18s

The CI file **is in the repository**, but it is hidden inside the `.gitea` folder.

From your uploaded ZIP, the path is:

```
.gitea/workflows/ci.yml
```

In VS Code, expand:

```
.gitea
├── workflows
│ └── ci.yml
```

The current workflow contains:

```
name: Continuous Integration

on:
 push:
 branches:
 - main
 pull_request:

jobs:
 quality:
 runs-on: ubuntu-latest
 timeout-minutes: 15

 steps:
 - name: Check out repository
 uses: actions/checkout@v4

 - name: Set up Node.js
 uses: actions/setup-node@v4
 with:
 node-version: 20

 - name: Install dependencies
 run: npm install

 - name: Run repository checks
 run: npm run check
```



Two likely problems are already visible:

- it uses `runs-on: ubuntu-latest`, but your Wits Gitea may not currently have a runner with that label;
- it uses `npm install`, while your issue intends to make installation reproducible with `package-lock.json` and `npm ci`.

Do not edit it yet.

First, create or find the issue titled:

Restore green CI and reproducible installation

Then assign it to yourself, move it to **In Progress**, and send me the issue number. After that, we will create the correct branch and diagnose the runner before changing `ci.yml`.

8/5/2026, 1:57:36 PM



issue #10

8/5/2026, 1:58:56 PM



### Thinking

- **Starting issue #10**

I'm treating issue #10 as the next task. First, assign it, move it to **In Progress**, add the Sprint 1 milestone, and leave a brief start comment. Then I'll have the user sync `main` and create a dedicated branch before editing.

- **Started issue #10**

✓ Worked for 12s

Great — Issue **#10** will use this branch:

```
ci/10-restore-green-ci
```

## Step 1: Start the issue in Gitea

Open Issue #10 and:

1. Assign it to yourself.
2. Set **Milestone** to `Sprint 1`.
3. Add it to `Sport-Analytics-Tool-Proj`.
4. Move it from `Ready` to `In Progress`.
5. Add this comment:

I am taking responsibility for this issue.

I will first verify the current repository state and diagnose why the Gitea Actions workflow is queued before changing the CI configuration.

## Step 2: Create your branch

In the VS Code PowerShell terminal, run these commands one at a time:

```
git switch main
```

```
git pull origin main
```

```
git status
```

Only continue if it says:

```
nothing to commit, working tree clean
```

Then create the branch:

```
git switch -c ci/10-restore-green-ci
```

Confirm it:

```
git branch --show-current
```

It should display:

```
ci/10-restore-green-ci
```

### Step 3: Inspect, but do not change, the workflow

Run:

```
Get-Content .gitea/workflows/ci.yml
```

Then run:

```
Test-Path package-lock.json
```

Send me the terminal output from those two commands. Do not edit `ci.yml` or generate the lockfile yet.

*8/5/2026, 1:59:08 PM*



```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> Get-Content
.gitea/workflows/ci.yml
name: Continuous Integration

on:
 push:
 branches:
 - main
 pull_request:

jobs:
 quality:
 runs-on: ubuntu-latest
 timeout-minutes: 15

 steps:
 - name: Check out repository
 uses: actions/checkout@v4

 - name: Set up Node.js
 uses: actions/setup-node@v4
 with:
 node-version: 20

 - name: Install dependencies
 run: npm install

 - name: Run repository checks
 run: npm run check

PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> Test-Path package-
lock.json
False
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool>
```

8/5/2026, 2:00:26 PM



Good. We have confirmed two things:

- CI currently uses `npm install`.
- There is no `package-lock.json`, so `npm ci` cannot work yet.
- The queued Gitea run may still be a runner problem, but first we need to prove the repository installs and checks locally.

Do not edit `ci.yml` yet.

## Next step: verify the local project configuration

Run these commands one at a time:

```
git branch --show-current
```

It should return:

```
ci/10-restore-green-ci
```

Then:

```
node --version
```

```
npm --version
```

Then display the root package configuration:

```
Get-Content package.json
```

Finally:

```
Test-Path .npmrc
```

Send me all four outputs. After checking them, we will install the dependencies and generate the lockfile safely.

*8/5/2026, 2:00:39 PM*



```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> node --version
v24.14.0
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> npm --version
npm : File C:\Program Files\nodejs\npm.ps1 cannot be loaded because running
scripts is disabled on this system. For more information,
see about_Execution_Policies at https://go.microsoft.com/fwlink/?LinkID=135170.
At line:1 char:1
+ npm --version
+ ~~~
+ CategoryInfo : SecurityError: (:) [], PSSecurityException
+ FullyQualifiedErrorId : UnauthorizedAccess
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> npm.cmd --version
11.9.0
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> Get-Content
package.json
{
 "name": "sport-analytics-tool",
 "version": "0.1.0",
 "private": true,
 "description": "Event-driven sports analytics platform for COMS3011A.",
 "workspaces": [
 "apps/*",
 "packages/*"
],
 "engines": {
 "node": ">=20"
 },
 "scripts": {
 "dev": "npm run dev --workspace=@sport-analytics/frontend",
 "dev:frontend": "npm run dev --workspace=@sport-analytics/frontend",
 "dev:backend": "npm run dev --workspace=@sport-analytics/backend",
 "build": "npm run build --workspace=@sport-analytics/contracts && npm run
build --workspace=@sport-analytics/backend && npm run build --
workspace=@sport-analytics/frontend",
 "typecheck": "npm run typecheck --workspaces --if-present",
 "lint": "npm run lint --workspaces --if-present",
 "test": "npm run test --workspaces --if-present",
 "format": "prettier --write .",
 "format:check": "prettier --check .",
 "structure:check": "node scripts/check-required-files.mjs",
 "check": "npm run structure:check && npm run format:check && npm run lint
&& npm run typecheck && npm run test && npm run build"
 },
 "devDependencies": {
 "@typescript-eslint/eslint-plugin": "^7.18.0",
 "@typescript-eslint/parser": "^7.18.0",
 "eslint": "^8.57.1",
 "eslint-plugin-react-hooks": "^4.6.2",
 "prettier": "^3.3.3",
 "typescript": "^5.6.3"
 }
}
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> Test-Path .npmrc
False
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool>
```

8/5/2026, 2:01:56 PM



### Thinking

- **Aligning local setup**

I'm seeing two separate issues: PowerShell is blocking `npm.ps1`, and your local Node 24 differs from CI's Node 20. I'd first use `npm.cmd`, then install dependencies and generate the lockfile, ideally under Node 20 for consistency.

- **Aligned local setup**

✓ Worked for 28s

Good — this tells us:

- Node `v24.14.0` satisfies the current requirement of `>=20`.
- CI uses Node 20, so we may later pin the recommended development version to Node 20 for consistency.
- PowerShell is blocking `npm.ps1`; using `npm.cmd` is perfectly fine. Do **not** change your execution policy now.
- No `.npmrc` is not a problem.

### Next step: install dependencies and generate the lockfile

First confirm your branch:

```
git branch --show-current
```

It must show:

```
ci/10-restore-green-ci
```

Then check that you have no unrelated local changes:

```
git status
```

Now install the workspace dependencies:

```
npm.cmd install
```

This should:

- install the root and workspace dependencies;
- create `node_modules`;
- create the missing root `package-lock.json`.

Do **not** run `npm audit fix`, even if npm suggests it.

When installation finishes, run:

```
git status --short
```

Then run the complete repository check:

```
npm.cmd run check
```

Send me:

1. the final part of the `npm.cmd install` output;
2. the `git status --short` output;

3. the full result of `npm.cmd run check`.

Do not edit or commit anything yet.

*8/5/2026, 2:02:11 PM*



```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> npm.cmd install
npm warn deprecated inflight@1.0.6: This module is not supported, and leaks
memory. Do not use it. Check out lru-cache if you want a good and tested way to
coalesce async requests by a key value, which is much more comprehensive and
powerful.
npm warn deprecated glob@7.2.3: Old versions of glob are not supported, and
contain widely publicized security vulnerabilities, which have been fixed in the
current version. Please update. Support for old versions may be purchased (at
exorbitant rates) by contacting i@izs.me
npm warn deprecated whatwg-encoding@3.1.1: Use @exodus/bytes instead for a
more spec-conformant and faster implementation
npm warn deprecated @humanwhocodes/config-array@0.13.0: Use @eslint/config-
array instead
npm warn deprecated rimraf@3.0.2: Rimraf versions prior to v4 are no longer
supported
npm warn deprecated @humanwhocodes/object-schema@2.0.3: Use
@eslint/object-schema instead
npm warn deprecated eslint@8.57.1: This version is no longer supported. Please
see https://eslint.org/version-support for other options.

added 418 packages, and audited 422 packages in 35s

84 packages are looking for funding
 run `npm fund` for details

5 vulnerabilities (3 moderate, 1 high, 1 critical)

To address all issues (including breaking changes), run:
 npm audit fix --force

Run `npm audit` for details.
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> git status --short
?? package-lock.json
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> npm.cmd run check

> sport-analytics-tool@0.1.0 check
> npm run structure:check && npm run format:check && npm run lint && npm run
typecheck && npm run test && npm run build

> sport-analytics-tool@0.1.0 structure:check
> node scripts/check-required-files.mjs

Repository structure check passed (12 required files).

> sport-analytics-tool@0.1.0 format:check
> prettier --check .

Checking formatting...
[warn] .eslintrc.cjs
[warn] .gitea/ISSUE_TEMPLATE/bug.md
[warn] .gitea/ISSUE_TEMPLATE/feature.md
[warn] .gitea/PULL_REQUEST_TEMPLATE.md
[warn] .gitea/workflows/ci.yml
[warn] .prettierrc.json
[warn] apps/backend/package.json
[warn] apps/backend/src/app.ts
[warn] apps/backend/src/config/env.ts
```

[warn] apps/backend/src/index.ts  
[warn] apps/backend/src/middleware/error-handler.ts  
[warn] apps/backend/src/middleware/not-found.ts  
[warn] apps/backend/src/modules/README.md  
[warn] apps/backend/src/routes/health.routes.ts  
[warn] apps/backend/tests/health.test.ts  
[warn] apps/backend/tsconfig.json  
[warn] apps/frontend/index.html  
[warn] apps/frontend/package.json  
[warn] apps/frontend/public/README.md  
[warn] apps/frontend/src/api/client.ts  
[warn] apps/frontend/src/App.test.tsx  
[warn] apps/frontend/src/App.tsx  
[warn] apps/frontend/src/components/README.md  
[warn] apps/frontend/src/features/README.md  
[warn] apps/frontend/src/main.tsx  
[warn] apps/frontend/src/pages/README.md  
[warn] apps/frontend/src/styles.css  
[warn] apps/frontend/src/test/setup.ts  
[warn] apps/frontend/src/vite-env.d.ts  
[warn] apps/frontend/tsconfig.app.json  
[warn] apps/frontend/tsconfig.json  
[warn] apps/frontend/tsconfig.node.json  
[warn] apps/frontend/vite.config.ts  
[warn] CONTRIBUTING.md  
[warn] database/migrations/README.md  
[warn] database/README.md  
[warn] database/schema/event-model.md  
[warn] database/seeds/README.md  
[warn] docs/ai/usage.md  
[warn] docs/api/overview.md  
[warn] docs/architecture/overview.md  
[warn] docs/architecture/repository-structure.md  
[warn] docs/database/overview.md  
[warn] docs/database/schema.md  
[warn] docs/deployment/azure.md  
[warn] docs/deployment/overview.md  
[warn] docs/development/dependencies.md  
[warn] docs/development/setup.md  
[warn] docs/development/testing.md  
[warn] docs/git-methodology.md  
[warn] docs/index.md  
[warn] docs/project\_methodology.md  
[warn] docs/security/overview.md  
[warn] evidence/decisions/ADR-000-template.md  
[warn] evidence/README.md  
[warn] evidence/sprints/README.md  
[warn] evidence/stakeholder-meetings/README.md  
[warn] evidence/user-testing/README.md  
[warn] infra/azure/README.md  
[warn] MIGRATION\_GUIDE.md  
[warn] mkdocs.yml  
[warn] package.json  
[warn] packages/contracts/package.json  
[warn] packages/contracts/README.md  
[warn] packages/contracts/src/health.ts  
[warn] packages/contracts/src/index.ts  
[warn] packages/contracts/tsconfig.json  
[warn] README.md  
[warn] scripts/check-required-files.mjs



```
[warn] SECURITY.md
[warn] tests/accessibility/README.md
[warn] tests/e2e/README.md
[warn] tests/performance/README.md
[warn] tsconfig.base.json
[warn] Code style issues found in 74 files. Run Prettier with --write to fix.
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool>
```

8/5/2026, 2:04:05 PM



## Thinking

### Inspecting repository files and formatting configuration

- **Assessing formatting changes**

I'm interpreting the install as successful, with only the formatting check failing and the lockfile newly created. I'd avoid forced vulnerability fixes, then inspect formatter configuration before deciding whether a clean, isolated formatting pass is appropriate.

- **Assessed formatting changes**

- **Diagnosing line endings**

I'm narrowing the warnings to existing CRLF files rather than substantive formatting. I'd verify repository attributes and tracked line endings first, then use a clean checkout to confirm whether normalization is needed before changing many files.

- **Diagnosed line endings**

- **Inspected repository files and Prettier formatting configuration**

✓ Worked for 46s

Good news: the installation succeeded and `package-lock.json` was created.

The repository check failed at the **formatting stage**, so linting, tests and builds have not run yet. The 74 warnings may be caused mainly by Windows `CRLF` line endings in your existing checkout, especially because `.gitattributes` was added after the repository had already been cloned.

Do **not** run `npm audit fix --force` and do **not** run the formatter across all 74 files yet.

## Next: check the line endings

Run these commands:

```
git ls-files --eol package.json .gitea/workflows/ci.yml README.md
```

```
Get-Content .gitattributes
```

```
git status --short
```

The first command may show something such as:

```
i/lf w/crlf attr/text eol=lf
```

That would mean:

- `i/lf`: Git stores the file with LF;
- `w/crlf`: your local working copy currently uses CRLF;
- `attr/text eol=lf`: the repository now requires LF.

Send me those three outputs. Then we will decide whether to normalise the checkout or whether there are actual formatting problems.

8/5/2026, 2:04:07 PM



```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> git ls-files --eol
package.json .gitea/workflows/ci.yml README.md
i/lf w/crlf attr/ .gitea/workflows/ci.yml
i/lf w/crlf attr/ README.md
i/lf w/crlf attr/ package.json
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> Get-Content
.gitattributes
Get-Content : Cannot find path 'C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool\gitattributes' because it does not exist.
At line:1 char:1
+ Get-Content .gitattributes
+ ~~~~~
+ CategoryInfo : ObjectNotFound: (C:\Users\shayn\...\gitattributes:String)
[Get-Content], ItemNotFoundException
+ FullyQualifiedErrorId :
PathNotFound,Microsoft.PowerShell.Commands.GetContentCommand

PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> git status --short
?? package-lock.json
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool>
```

8/5/2026, 2:06:05 PM



## Thinking

- **Diagnosing attributes**

I'm checking whether `.gitattributes` actually exists on the current remote branch or whether line-ending settings are causing the discrepancy. I'll verify repository history before changing files, so normalization doesn't create unnecessary churn.

- **Diagnosed attributes**

✓ Worked for 12s

This confirms the problem:

- The files are stored in Git with `LF`.
- Your Windows working copy has `CRLF`.
- No `.gitattributes` rule is currently active because `.gitattributes` is missing from your checkout.
- Only `package-lock.json` is currently untracked, so you have not accidentally changed the 74 files.

Do **not** run Prettier yet.

## Check whether `.gitattributes` exists on remote `main`

Run these one at a time:

```
git branch --show-current
```

It should show:

```
ci/10-restore-green-ci
```

Then:

```
git fetch origin
```



```
git rev-parse --short HEAD
```

```
git rev-parse --short origin/main
```

Now check whether remote `main` contains `.gitattributes`:

```
git ls-tree -r --name-only origin/main | Select-String "^\.gitattributes$"
```

Then check its history:

```
git log --oneline --all -- .gitattributes
```

Also show the Prettier configuration:

```
Get-Content .prettierrc.json
```

### What the results will tell us

- If `.gitattributes` appears on `origin/main`, your branch is behind and we will update it safely.
- If it does not appear, the Issue #7 Pull Request probably merged without that file, and we will add it properly under Issue #10 or create a small correction issue.
- The Prettier configuration will confirm whether `CRLF` is the reason all 74 files fail formatting.

Keep `package-lock.json` where it is. Do not commit, format, reset or delete anything yet.

8/5/2026, 2:06:17 PM



```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> git fetch origin
remote: Enumerating objects: 1, done.
remote: Counting objects: 100% (1/1), done.
remote: Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (1/1), 339 bytes | 84.00 KiB/s, done.
From https://sdp.ms.wits.ac.za/git-push-pray/Sport-Analytics-Tool
 449095d..e84fe8d main -> origin/main
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> git rev-parse --short HEAD
449095d
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> git rev-parse --short origin/main
e84fe8d
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> git ls-tree -r --name-only origin/main | Select-String "^\.gitattributes$"

.gitattributes

PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> git log --oneline --all -- .gitattributes
14d0194 (origin/chore/7-align-repository-governance, chore/7-align-repository-governance) chore: enforce consistent line endings
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> Get-Content .prettierrc.json
{
 "singleQuote": true,
 "trailingComma": "all",
 "semi": true,
 "printWidth": 100
}
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> Get-Content .prettierrc.json
{
 "singleQuote": true,
 "trailingComma": "all",
 "semi": true,
 "printWidth": 100
}
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool>
```

8/5/2026, 2:08:11 PM



### Thinking

- **Updating the branch**

I'm confirming the branch is behind `origin/main`, not ahead. I'd first preserve the untracked lockfile, rebase onto the updated main branch, then restore it before running tests and committing the CI changes.

- **Updated the branch**

✓ Worked for 22s

This confirms the issue: your CI branch was created from the old commit `449095d`, while `main` has advanced to `e84fe8d` and now includes `.gitattributes`.

We need to update your branch **without losing** `package-lock.json`.

## Step 1: Temporarily save the lockfile

Run:

```
git stash push -u -m "temporary package lock before updating CI branch"
```

Then check:

```
git status
```

It should say the working tree is clean.

## Step 2: Update your CI branch from `main`

Confirm your branch:

```
git branch --show-current
```

It must show:

```
ci/10-restore-green-ci
```

Now rebase it onto the latest remote `main`:

```
git rebase origin/main
```

Because you have not committed anything on this branch yet, this should be straightforward.

## Step 3: Refresh the working copy using `.gitattributes`

Run:

```
git reset --hard HEAD
```

This rewrites the tracked working files according to the newly added line-ending rules.

Confirm that `.gitattributes` now exists:

```
Get-Content .gitattributes
```

## Step 4: Restore the generated lockfile

```
git stash pop
```

Then check:

```
git status --short
```

You should ideally see only:

```
?? package-lock.json
```

## Step 5: Confirm the line endings

Run:

```
git ls-files --eol package.json .gitea/workflows/ci.yml README.md
```

We want the output to show:

```
i/lf w/lf
```

rather than:

```
i/lf w/crlf
```

Then retry only the formatting check:

```
npm.cmd run format:check
```

Send me the outputs from:

```
git status --short
git ls-files --eol package.json .gitea/workflows/ci.yml README.md
npm.cmd run format:check
```

Do not commit the lockfile or change `ci.yml` yet.

8/5/2026, 2:08:24 PM



```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> git ls-files --eol
package.json .gitea/workflows/ci.yml README.md
i/lf w/crlf attr/text=auto eol=lf .gitea/workflows/ci.yml
i/lf w/crlf attr/text=auto eol=lf README.md
i/lf w/crlf attr/text=auto eol=lf package.json
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool>
```

8/5/2026, 2:10:24 PM



### Thinking

- **Fixing line endings**

I'm narrowing this to Git's checkout behavior: the attributes are correct, but existing files weren't rewritten. I'd disable automatic conversion locally, then force a worktree refresh while preserving untracked files.

- **Fixed line endings**

✓ Worked for 25s

That means `.gitattributes` is now active, but your existing Windows working copy has not yet been rewritten from `CRLF` to `LF`.

Do this carefully.

## 1. Confirm only the lockfile is untracked

```
git status --short
```

Continue only if the only output is:

```
?? package-lock.json
```

## 2. Configure this repository to keep LF line endings

```
git config core.autocrlf false
git config core.eol lf
```

These settings apply only to this repository.

## 3. Force Git to rewrite the tracked files

```
git restore --source=HEAD --worktree -- .
```

This will not delete the untracked `package-lock.json`.

## 4. Check again

```
git ls-files --eol package.json .gitea/workflows/ci.yml README.md
```

We want:

```
i/lf w/lf attr/text=auto eol=lf
```



## 5. Retry the formatting check

```
npm.cmd run format:check
```

Do not stage or commit mass formatting changes. Send me the outputs from steps 4 and 5.

*8/5/2026, 2:10:35 PM*



```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> git config core.autocrlf false
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> git config core.eol lf
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> git restore --source=HEAD --worktree -- .
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> git ls-files --eol
package.json .gitea/workflows/ci.yml README.md
i/lf w/crlf attr/text=auto eol=lf .gitea/workflows/ci.yml
i/lf w/crlf attr/text=auto eol=lf README.md
i/lf w/crlf attr/text=auto eol=lf package.json
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool>
```

8/5/2026, 2:11:49 PM



### Thinking

- **Troubleshooting line endings**

I'm checking whether Git's attributes are being applied during checkout. Since the index already appears normalized, I'm considering a forced worktree refresh or a fresh clone, while avoiding changes to staged files.

- **Troubleshoot line endings**

✓ Worked for 11s

That means Git is still not physically rewriting the existing files, even though the new LF rule is active. We'll test a forced checkout on only three tracked files first.

Your untracked `package-lock.json` will not be affected.

## 1. Confirm there are no tracked changes

```
git status --short
```

Continue only if the only output is:

```
?? package-lock.json
```

## 2. Delete the three test files from the working folder

```
Remove-Item package.json
Remove-Item README.md
Remove-Item .gitea/workflows/ci.yml
```

This does **not** delete them from Git. We are forcing Git to recreate them from the committed versions.

## 3. Restore them from Git

```
git checkout-index --force -- package.json README.md .gitea/workflows/ci.yml
```

## 4. Check their line endings again

```
git ls-files --eol package.json .gitea/workflows/ci.yml README.md
```

We want:

```
i/lf w/lf attr/text=auto eol=lf
```

If those three now show `w/lf`, we can safely force Git to rewrite all tracked files using:

```
git checkout-index --force --all
```

Then verify:

```
git status --short
npm.cmd run format:check
```

Do not run the all-files command until the three-file test shows `w/lf`.

*8/5/2026, 2:11:59 PM*



```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> git status --short
>> npm.cmd run format:check
M .gitea/workflows/ci.yml
M README.md
M package.json
?? package-lock.json
```

```
> sport-analytics-tool@0.1.0 format:check
> prettier --check .
```

```
Checking formatting...
[warn] .eslintrc.cjs
[warn] .gitea/ISSUE_TEMPLATE/bug.md
[warn] .gitea/ISSUE_TEMPLATE/feature.md
[warn] .gitea/ISSUE_TEMPLATE/task.md
[warn] .gitea/PULL_REQUEST_TEMPLATE.md
[warn] .prettierrc.json
[warn] apps/backend/package.json
[warn] apps/backend/src/app.ts
[warn] apps/backend/src/config/env.ts
[warn] apps/backend/src/index.ts
[warn] apps/backend/src/middleware/error-handler.ts
[warn] apps/backend/src/middleware/not-found.ts
[warn] apps/backend/src/modules/README.md
[warn] apps/backend/src/routes/health.routes.ts
[warn] apps/backend/tests/health.test.ts
[warn] apps/backend/tsconfig.json
[warn] apps/frontend/index.html
[warn] apps/frontend/package.json
[warn] apps/frontend/public/README.md
[warn] apps/frontend/src/api/client.ts
[warn] apps/frontend/src/App.test.tsx
[warn] apps/frontend/src/App.tsx
[warn] apps/frontend/src/components/README.md
[warn] apps/frontend/src/features/README.md
[warn] apps/frontend/src/main.tsx
[warn] apps/frontend/src/pages/README.md
[warn] apps/frontend/src/styles.css
[warn] apps/frontend/src/test/setup.ts
[warn] apps/frontend/src/vite-env.d.ts
[warn] apps/frontend/tsconfig.app.json
[warn] apps/frontend/tsconfig.json
[warn] apps/frontend/tsconfig.node.json
[warn] apps/frontend/vite.config.ts
[warn] CONTRIBUTING.md
[warn] database/migrations/README.md
[warn] database/README.md
[warn] database/schema/event-model.md
[warn] database/seeds/README.md
[warn] docs/ai/usage.md
[warn] docs/api/overview.md
[warn] docs/architecture/overview.md
[warn] docs/architecture/repository-structure.md
[warn] docs/database/overview.md
[warn] docs/database/schema.md
[warn] docs/deployment/azure.md
[warn] docs/deployment/overview.md
[warn] docs/development/dependencies.md
```

```
[warn] docs/development/setup.md
[warn] docs/development/testing.md
[warn] docs/git-methodology.md
[warn] docs/index.md
[warn] docs/project_methodology.md
[warn] docs/security/overview.md
[warn] evidence/decisions/ADR-000-template.md
[warn] evidence/decisions/ADR-001-initial-repository-setup.md
[warn] evidence/README.md
[warn] evidence/sprints/README.md
[warn] evidence/stakeholder-meetings/README.md
[warn] evidence/user-testing/README.md
[warn] infra/azure/README.md
[warn] MIGRATION_GUIDE.md
[warn] mkdocs.yml
[warn] packages/contracts/package.json
[warn] packages/contracts/README.md
[warn] packages/contracts/src/health.ts
[warn] packages/contracts/src/index.ts
[warn] packages/contracts/tsconfig.json
[warn] scripts/check-required-files.mjs
[warn] SECURITY.md
[warn] tests/accessibility/README.md
[warn] tests/e2e/README.md
[warn] tests/performance/README.md
[warn] tsconfig.base.json
[warn] Code style issues found in 73 files. Run Prettier with --write to fix.
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool>
```

8/5/2026, 2:13:18 PM



## Thinking

- **Recovering repository state**

I'm seeing three tracked files still differ after the reset attempt. I'd avoid more destructive commands, inspect their diffs, and then restore only those files before rerunning the formatter and checking status.

- **Recovered repository state**

-  **Inspecting uploaded repository structure and formatting issues**

- **Checking file differences**

I'm narrowing this down to whether those three files contain real content changes or only line-ending differences. I'll compare their diffs first, then restore them safely if the differences are formatting-only.

- **Checked file differences**

- **Inspected repository structure, formatting issues, status, branch, and history**

✓ Worked for 1m 31s

Nothing is lost. The three `M` entries mean our line-ending test changed those three working files. The 73 Prettier warnings strongly suggest that the remaining problem is **CRLF versus LF**, not 73 separate code problems.

First, restore the three files safely while keeping `package-lock.json`.

## 1. Temporarily move the lockfile outside the repository

```
Move-Item package-lock.json ..\package-lock-ci10.json
```

## 2. Restore all tracked files exactly to the current commit

```
git reset --hard HEAD
```

This removes the unintended modifications to:

```
.gitea/workflows/ci.yml
README.md
package.json
```

## 3. Move the lockfile back

```
Move-Item ..\package-lock-ci10.json .\package-lock.json
```

## 4. Confirm the repository state

```
git status --short
```

It should now show only:

```
?? package-lock.json
```

Do not continue unless that is the only entry.

## 5. Check whether the warnings are purely line-ending related

Run Prettier while temporarily allowing the current platform's line endings:

```
npx.cmd prettier --check . --end-of-line auto
```

This does **not** change any files.

### Expected result

If it says:

```
All matched files use Prettier code style!
```

then we have confirmed that the repository is formatted correctly and the 73 warnings were caused only by Windows line endings.

If it still lists files, send me that output. We will then know there are genuine formatting differences as well.

Do not run `npm.cmd run format`, `prettier --write`, `npm audit fix`, or commit anything yet.

8/5/2026, 2:13:30 PM



```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> npx.cmd prettier --
check . --end-of-line auto
Checking formatting...
[warn] .gitea/ISSUE_TEMPLATE/bug.md
[warn] .gitea/ISSUE_TEMPLATE/feature.md
[warn] .gitea/ISSUE_TEMPLATE/task.md
[warn] .gitea/PULL_REQUEST_TEMPLATE.md
[warn] apps/frontend/src/App.tsx
[warn] apps/frontend/src/styles.css
[warn] apps/frontend/tsconfig.json
[warn] docs/development/dependencies.md
[warn] docs/development/testing.md
[warn] docs/git-methodology.md
[warn] docs/project_methodology.md
[warn] evidence/decisions/ADR-001-initial-repository-setup.md
[warn] scripts/check-required-files.mjs
[warn] Code style issues found in 13 files. Run Prettier with --write to fix.
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool>
```

8/5/2026, 2:15:44 PM



## Thinking

- **Reviewing formatting fixes**

I'm focusing on the 13 files that CI flagged, avoiding broad rewrites. I'll verify the working tree first, run the repository formatter only on those paths, then inspect the diff before committing the formatting-only change.

- **Reviewed formatting fixes**

- **Adjusting line-ending checks**

I'm recognizing that local CRLF conversion may keep triggering Prettier after the 13 files are fixed. I'll add a cross-platform line-ending setting, format only the flagged files, then verify the focused checks and review the diff.

✓ Worked for 52s

Good — this means:

- about **60 files only failed because of Windows line endings;**
- **13 files have genuine Prettier formatting differences;**
- we can fix those 13 files without formatting the entire repository.

We will also make Prettier tolerate Windows working-copy line endings while `.gitattributes` continues to store files as LF in Git.

## Step 1: Confirm the current state

Run:

```
git status --short
```

At this stage, you should ideally see only:

```
?? package-lock.json
```

Do not continue if there are other modified files.

## Step 2: Make Prettier cross-platform

Open:

```
.prettierrc.json
```

Replace its contents with:

```
{
 "singleQuote": true,
 "trailingComma": "all",
 "semi": true,
 "printWidth": 100,
 "endOfLine": "auto"
}
```

This does not remove the LF repository rule. `.gitattributes` still normalises committed text files to LF. It simply prevents Prettier from failing because your Windows working copy uses CRLF.

## Step 3: Format only the 13 genuine problem files

Run these commands one at a time.

### Gitea templates

```
npx.cmd prettier --write .gitea/ISSUE_TEMPLATE/bug.md .gitea/ISSUE_TEMPLATE/feature.md .gitea/ISSUE_TEMPLATE/task.md
.gitea/PULL_REQUEST_TEMPLATE.md
```

### Frontend files

```
npx.cmd prettier --write apps/frontend/src/App.tsx apps/frontend/src/styles.css apps/frontend/tsconfig.json
```

### Documentation files

```
npx.cmd prettier --write docs/development/dependencies.md docs/development/testing.md docs/git-methodology.md
docs/project_methodology.md evidence/decisions/ADR-001-initial-repository-setup.md
```

### Repository script

```
npx.cmd prettier --write scripts/check-required-files.mjs
```

## Step 4: Inspect what changed

Run:

```
git status --short
```

Then:

```
git diff --stat
```

And:

```
git diff --check
```

`git diff --check` should produce no output.

The changed files should consist only of:

- the 13 formatted files;
- `.prettierrc.json`;
- the new `package-lock.json`.

## Step 5: Confirm formatting now passes

First run:

```
npm.cmd run format:check
```

Then run:

```
npx.cmd prettier --check .
```

Both should now pass because Prettier will use:

```
"endOfLine": "auto"
```

## Step 6: Run the complete repository check again

```
npm.cmd run check
```

This time it should progress beyond formatting and continue into:

1. linting;
2. type checking;
3. tests;
4. production builds.

Do not commit anything yet. Send the output of:

```
git status --short
npm.cmd run check
```

The next errors, if any, will tell us what must be corrected before changing `ci.yml`.

8/5/2026, 2:16:00 PM



```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> git status --short
M .gitea/ISSUE_TEMPLATE/bug.md
M .gitea/ISSUE_TEMPLATE/feature.md
M .gitea/ISSUE_TEMPLATE/task.md
M .gitea/PULL_REQUEST_TEMPLATE.md
M .prettierrc.json
M apps/frontend/src/App.tsx
M apps/frontend/src/styles.css
M apps/frontend/tsconfig.json
M scripts/check-required-files.mjs
?? package-lock.json
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> git diff --stat
warning: in the working copy of '.prettierrc.json', CRLF will be replaced by LF the
next time Git touches it
warning: in the working copy of 'apps/frontend/src/App.tsx', CRLF will be replaced
by LF the next time Git touches it
warning: in the working copy of 'apps/frontend/src/styles.css', CRLF will be
replaced by LF the next time Git touches it
warning: in the working copy of 'apps/frontend/tsconfig.json', CRLF will be replaced
by LF the next time Git touches it
warning: in the working copy of 'scripts/check-required-files.mjs', CRLF will be
replaced by LF the next time Git touches it
.gitea/ISSUE_TEMPLATE/bug.md | 16 ++++++-----
.gitea/ISSUE_TEMPLATE/feature.md | 18 ++++++-----
.gitea/ISSUE_TEMPLATE/task.md | 6 +++--
.gitea/PULL_REQUEST_TEMPLATE.md | 4 +++-
.prettierrc.json | 5 +++-
apps/frontend/src/App.tsx | 4 +++-
apps/frontend/src/styles.css | 8 ++++++-
apps/frontend/tsconfig.json | 5 +---
scripts/check-required-files.mjs | 4 +++-
9 files changed, 38 insertions(+), 32 deletions(-)
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> git diff --check
warning: in the working copy of '.prettierrc.json', CRLF will be replaced by LF the
next time Git touches it
warning: in the working copy of 'apps/frontend/src/App.tsx', CRLF will be replaced
by LF the next time Git touches it
warning: in the working copy of 'apps/frontend/src/styles.css', CRLF will be
replaced by LF the next time Git touches it
warning: in the working copy of 'apps/frontend/tsconfig.json', CRLF will be replaced
by LF the next time Git touches it
warning: in the working copy of 'scripts/check-required-files.mjs', CRLF will be
replaced by LF the next time Git touches it
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> npm.cmd run
format:check

> sport-analytics-tool@0.1.0 format:check
> prettier --check .

Checking formatting...
[warn] .prettierrc.json
[warn] docs/development/dependencies.md
[warn] docs/development/testing.md
[warn] docs/git-methodology.md
[warn] docs/project_methodology.md
[warn] evidence/decisions/ADR-001-initial-repository-setup.md
[warn] Code style issues found in 6 files. Run Prettier with --write to fix.
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> npx.cmd prettier --
```

```
check .
Checking formatting...
[warn] .prettierrc.json
[warn] docs/development/dependencies.md
[warn] docs/development/testing.md
[warn] docs/git-methodology.md
[warn] docs/project_methodology.md
[warn] evidence/decisions/ADR-001-initial-repository-setup.md
[warn] Code style issues found in 6 files. Run Prettier with --write to fix.
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool>
```

8/5/2026, 2:19:06 PM



### Thinking

- **Fixing remaining files**

The earlier formatting command likely didn't modify those six documents, since Git still shows no changes there. I'm formatting exactly those files, then I'll rerun Prettier and the repository checks. The line-ending warnings are likely normalization notices rather than content errors.

- **Fixed remaining files**

✓ Worked for 16s

You are close. The remaining six files are still genuinely unformatted; they do not appear in `git status`, which means Prettier has not rewritten them yet.

Run this **single PowerShell command**:

```
npx.cmd prettier --write .prettierrc.json docs/development/dependencies.md docs/development/testing.md docs/git-
methodology.md docs/project_methodology.md evidence/decisions/ADR-001-initial-repository-setup.md
```

Then check formatting again:

```
npm.cmd run format:check
```

It should report:

```
All matched files use Prettier code style!
```

Next, inspect the changed files:

```
git status --short
```

```
git diff --stat
```

The CRLF warnings are not errors. Git is telling you that `.gitattributes` will store those files as LF when committed.

Once formatting passes, run the full check:

```
npm.cmd run check
```

This will continue through:

1. repository structure;
2. formatting;
3. linting;

4. type checking;
5. tests;
6. production builds.

Do not commit yet. Send the output from `npm.cmd run check`, especially the first error if it stops.

*8/5/2026, 2:19:20 PM*



```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> npm.cmd run check

> sport-analytics-tool@0.1.0 check
> npm run structure:check && npm run format:check && npm run lint && npm run
typecheck && npm run test && npm run build

> sport-analytics-tool@0.1.0 structure:check
> node scripts/check-required-files.mjs

Repository structure check passed (12 required files).

> sport-analytics-tool@0.1.0 format:check
> prettier --check .

Checking formatting...
All matched files use Prettier code style!

> sport-analytics-tool@0.1.0 lint
> npm run lint --workspaces --if-present

> @sport-analytics/backend@0.1.0 lint
> eslint src tests --ext .ts

=====

WARNING: You are currently running a version of TypeScript which is not officially
supported by @typescript-eslint/typescript-estree.

You may find that it works just fine, or you may not.

SUPPORTED TYPESCRIPT VERSIONS: >=4.7.4 <5.6.0

YOUR TYPESCRIPT VERSION: 5.9.3

Please only submit bug reports when using the officially supported version.

=====

C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-
Tool\apps\backend\src\middleware\error-handler.ts
 3:78 error '_next' is defined but never used @typescript-eslint/no-unused-vars

✖ 1 problem (1 error, 0 warnings)

npm error Lifecycle script `lint` failed with error:
npm error code 1
npm error path C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-
Tool\apps\backend
npm error workspace @sport-analytics/backend@0.1.0
npm error location C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-
Tool\apps\backend
npm error command failed
npm error command C:\Windows\system32\cmd.exe /d /s /c eslint src tests --ext .ts

> @sport-analytics/frontend@0.1.0 lint
```

```
> eslint src --ext .ts,.tsx

=====

WARNING: You are currently running a version of TypeScript which is not officially
supported by @typescript-eslint/typescript-estree.

You may find that it works just fine, or you may not.

SUPPORTED TYPESCRIPT VERSIONS: >=4.7.4 <5.6.0

YOUR TYPESCRIPT VERSION: 5.9.3

Please only submit bug reports when using the officially supported version.

=====

> @sport-analytics/contracts@0.1.0 lint
> eslint src --ext .ts

=====

WARNING: You are currently running a version of TypeScript which is not officially
supported by @typescript-eslint/typescript-estree.

You may find that it works just fine, or you may not.

SUPPORTED TYPESCRIPT VERSIONS: >=4.7.4 <5.6.0

YOUR TYPESCRIPT VERSION: 5.9.3

Please only submit bug reports when using the officially supported version.

=====
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool>
```

8/5/2026, 2:21:08 PM



Great — formatting now passes. CI has reached a genuine lint error, which is exactly what we needed to uncover.

There are two separate findings:

- **Blocking error:** `_next` is declared but unused in `error-handler.ts`.
- **Non-blocking warning:** npm installed TypeScript `5.9.3`, while the current `@typescript-eslint` version officially supports TypeScript versions below `5.6.0`.

Do not edit anything yet. Let us inspect the exact code and lint configuration first.

Run:

```
Get-Content apps/backend/src/middleware/error-handler.ts
```

```
Get-Content .eslintrc.cjs
```

Then verify the installed versions:

```
npm.cmd ls typescript @typescript-eslint/parser @typescript-eslint/eslint-plugin
```

Also show the backend package configuration:

```
Get-Content apps/backend/package.json
```

Send those outputs. We will then make the smallest correct fix rather than hiding the lint error or upgrading dependencies unnecessarily.

*8/5/2026, 2:21:25 PM*



```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> Get-Content
apps/backend/src/middleware/error-handler.ts
import type { ErrorRequestHandler } from 'express';

export const errorHandler: ErrorRequestHandler = (error, _request, response,
_next) => {
 const message = error instanceof Error ? error.message : 'Unknown error';

 response.status(500).json({
 error: {
 code: 'INTERNAL_SERVER_ERROR',
 message: 'An unexpected server error occurred.',
 },
 });

 // Replace this temporary console output with the central logger once
 observability is configured.
 console.error(message);
};
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> Get-Content .eslintrc.cjs
module.exports = {
 root: true,
 env: {
 es2022: true,
 node: true,
 },
 parser: '@typescript-eslint/parser',
 parserOptions: {
 ecmaVersion: 'latest',
 sourceType: 'module',
 },
 plugins: ['@typescript-eslint'],
 extends: ['eslint:recommended', 'plugin:@typescript-eslint/recommended'],
 ignorePatterns: ['**/dist/**', '**/coverage/**', 'site/**'],
 overrides: [
 {
 files: ['apps/frontend/**/*.ts,tsx'],
 env: {
 browser: true,
 node: false,
 },
 plugins: ['react-hooks'],
 extends: ['plugin:react-hooks/recommended'],
 parserOptions: {
 ecmaFeatures: {
 jsx: true,
 },
 },
 },
],
};
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> npm.cmd ls typescript
@typescript-eslint/parser @typescript-eslint/eslint-plugin
sport-analytics-tool@0.1.0 C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool
├─ @typescript-eslint/eslint-plugin@7.18.0
│ └─ @typescript-eslint/parser@7.18.0 deduped
│ └─ ts-api-utils@1.4.3
│ └─ typescript@5.9.3 deduped
```

```
└─ @typescript-eslint/parser@7.18.0
└─ typescript@5.9.3
```

```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> Get-Content
apps/backend/package.json
```

```
{
 "name": "@sport-analytics/backend",
 "version": "0.1.0",
 "private": true,
 "scripts": {
 "dev": "tsx watch src/index.ts",
 "build": "tsc -p tsconfig.json",
 "start": "node dist/index.js",
 "lint": "eslint src tests --ext .ts",
 "typecheck": "tsc --noEmit",
 "test": "vitest run"
 },
 "dependencies": {
 "cors": "^2.8.5",
 "express": "^4.21.1",
 "helmet": "^8.0.0",
 "pino-http": "^10.3.0",
 "zod": "^3.23.8"
 },
 "devDependencies": {
 "@types/cors": "^2.8.17",
 "@types/express": "^4.17.21",
 "@types/node": "^22.9.0",
 "@types/supertest": "^6.0.2",
 "supertest": "^7.0.0",
 "tsx": "^4.19.2",
 "vitest": "^2.1.4"
 }
}
```

```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool>
```

8/5/2026, 2:22:30 PM



## Thinking

- **Fixing lint errors**

I'm checking the failure rather than changing architecture. The smallest fix is configuring TypeScript ESLint to ignore intentionally unused parameters beginning with `\_` ; then I'll rerun the exact lint command.

- **Fixed lint errors**

- **Aligning lint and dependencies**

I'm fixing this in two focused changes: allowing the existing underscore-prefixed error-handler parameter, then pinning TypeScript to a parser-compatible version and regenerating the lockfile. Afterward, I'll rerun lint before touching formatting.

✓ Worked for 43s

We now know the correct minimal fixes:

1. The `_next` parameter is intentionally present, so ESLint should recognise underscore-prefixed arguments as deliberately unused.
2. TypeScript must be pinned below `5.6.0` to match the installed `@typescript-eslint` version.

## Step 1: Update the ESLint configuration

Open `.eslintrc.cjs` and add this `rules` section immediately after the `extends` line:

```
rules: {
 '@typescript-eslint/no-unused-vars': ['error', { argsIgnorePattern: '^_' }],
},
```

The relevant section should look like this:

```
plugins: ['@typescript-eslint'],
extends: ['eslint:recommended', 'plugin:@typescript-eslint/recommended'],
rules: {
 '@typescript-eslint/no-unused-vars': ['error', { argsIgnorePattern: '^_' }],
},
ignorePatterns: ['**/dist/**', '**/coverage/**', 'site/**'],
```

Save the file.

## Step 2: Pin TypeScript to a compatible version

Run:

```
npm.cmd install --save-dev --save-exact typescript@5.5.4
```

This should update:

- `package.json`
- `package-lock.json`
- the installed TypeScript version

Do not run `npm audit fix`.

## Step 3: Verify the installed versions

```
npm.cmd ls typescript @typescript-eslint/parser @typescript-eslint/eslint-plugin
```

We want TypeScript to show:

```
typescript@5.5.4
```

## Step 4: Run linting only

```
npm.cmd run lint
```

The unsupported TypeScript warning should disappear, and the `_next` error should be resolved.

Then run:

```
git status --short
```

Send me the version output, lint result and Git status. Do not commit yet.

8/5/2026, 2:22:44 PM



```
sport-analytics-tool@0.1.0 C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool
├─┬─ @typescript-eslint/eslint-plugin@7.18.0
│ └─ @typescript-eslint/parser@7.18.0 deduped
│ └─ ts-api-utils@1.4.3
│ └─ typescript@5.5.4 deduped
├─ @typescript-eslint/parser@7.18.0
└─ typescript@5.5.4
```

```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> npm.cmd run lint
```

```
> sport-analytics-tool@0.1.0 lint
> npm run lint --workspaces --if-present
```

```
> @sport-analytics/backend@0.1.0 lint
> eslint src tests --ext .ts
```

```
> @sport-analytics/frontend@0.1.0 lint
> eslint src --ext .ts,.tsx
```

```
> @sport-analytics/contracts@0.1.0 lint
> eslint src --ext .ts
```

```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> git status --short
```

```
M .eslintrc.cjs
M .gitea/ISSUE_TEMPLATE/bug.md
M .gitea/ISSUE_TEMPLATE/feature.md
M .gitea/ISSUE_TEMPLATE/task.md
M .gitea/PULL_REQUEST_TEMPLATE.md
M .prettierrc.json
M apps/frontend/src/App.tsx
M apps/frontend/src/styles.css
M apps/frontend/tsconfig.json
M docs/development/dependencies.md
M docs/development/testing.md
M docs/git-methodology.md
M docs/project_methodology.md
M evidence/decisions/ADR-001-initial-repository-setup.md
M package.json
M scripts/check-required-files.mjs
?? package-lock.json
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool>
```

8/5/2026, 2:24:46 PM



Excellent. The lint error is fixed, and the TypeScript compatibility warning is gone.

## Next step: run the complete repository check again

```
npm.cmd run check
```

It should now continue through:

- structure checking;

- formatting;
- linting;
- type checking;
- tests;
- builds.

Send the output from where it either fails or finishes successfully.

Do not commit yet. Once the full check passes, we still need to:

1. verify that the new `package-lock.json` supports a clean `npm ci`;
2. change the Gitea workflow from `npm install` to `npm ci`;
3. diagnose why the online workflow remains queued;
4. organise the changes into clear atomic commits.

*8/5/2026, 2:25:00 PM*



```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> npm.cmd run check

> sport-analytics-tool@0.1.0 check
> npm run structure:check && npm run format:check && npm run lint && npm run
typecheck && npm run test && npm run build

> sport-analytics-tool@0.1.0 structure:check
> node scripts/check-required-files.mjs

Repository structure check passed (12 required files).

> sport-analytics-tool@0.1.0 format:check
> prettier --check .

Checking formatting...
[warn] .eslintrc.cjs
[warn] Code style issues found in the above file. Run Prettier with --write to fix.
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool>
```

8/5/2026, 2:25:25 PM



Only `.eslintrc.cjs` remains unformatted.

Run:

```
npx.cmd prettier --write .eslintrc.cjs
```

Then verify formatting:

```
npm.cmd run format:check
```

If that passes, rerun the complete check:

```
npm.cmd run check
```

Send the output from `npm.cmd run check`, whether it succeeds or stops at the next error. Do not commit yet.

8/5/2026, 2:25:30 PM





```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> npm.cmd install --save-dev --save-exact typescript@5.5.4
```

changed 1 package, and audited 422 packages in 2s

84 packages are looking for funding  
run `npm fund` for details

5 vulnerabilities (3 moderate, 1 high, 1 critical)

To address all issues (including breaking changes), run:  
npm audit fix --force

Run `npm audit` for details.

```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> npm.cmd ls typescript
@typescript-eslint/parser @typescript-eslint/eslint-plugin
sport-analytics-tool@0.1.0 C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool
├─┬ @typescript-eslint/eslint-plugin@7.18.0
│ └─ @typescript-eslint/parser@7.18.0 deduped
│ └─ ts-api-utils@1.4.3
│ └─ typescript@5.5.4 deduped
├─ @typescript-eslint/parser@7.18.0
└─ typescript@5.5.4
```

```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> npm.cmd run lint
```

```
> sport-analytics-tool@0.1.0 lint
> npm run lint --workspaces --if-present
```

```
> @sport-analytics/backend@0.1.0 lint
> eslint src tests --ext .ts
```

```
> @sport-analytics/frontend@0.1.0 lint
> eslint src --ext .ts,.tsx
```

```
> @sport-analytics/contracts@0.1.0 lint
> eslint src --ext .ts
```

```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> git status --short
```

```
M .eslintrc.cjs
M .gitea/ISSUE_TEMPLATE/bug.md
M .gitea/ISSUE_TEMPLATE/feature.md
M .gitea/ISSUE_TEMPLATE/task.md
M .gitea/PULL_REQUEST_TEMPLATE.md
M .prettierrc.json
M apps/frontend/src/App.tsx
M apps/frontend/src/styles.css
M apps/frontend/tsconfig.json
M docs/development/dependencies.md
M docs/development/testing.md
M docs/git-methodology.md
M docs/project_methodology.md
M evidence/decisions/ADR-001-initial-repository-setup.md
M package.json
M scripts/check-required-files.mjs
```

```

?? package-lock.json
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> npm.cmd run check

> sport-analytics-tool@0.1.0 check
> npm run structure:check && npm run format:check && npm run lint && npm run
typecheck && npm run test && npm run build

> sport-analytics-tool@0.1.0 structure:check
> node scripts/check-required-files.mjs

Repository structure check passed (12 required files).

> sport-analytics-tool@0.1.0 format:check
> prettier --check .

Checking formatting...
[warn] .eslintrc.cjs
[warn] Code style issues found in the above file. Run Prettier with --write to fix.
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> npx.cmd prettier --write
.eslintrc.cjs
.eslintrc.cjs 25ms
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> npm.cmd run
format:check

> sport-analytics-tool@0.1.0 format:check
> prettier --check .

Checking formatting...
All matched files use Prettier code style!
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> npm.cmd run check

> sport-analytics-tool@0.1.0 check
> npm run structure:check && npm run format:check && npm run lint && npm run
typecheck && npm run test && npm run build

> sport-analytics-tool@0.1.0 structure:check
> node scripts/check-required-files.mjs

Repository structure check passed (12 required files).

> sport-analytics-tool@0.1.0 format:check
> prettier --check .

Checking formatting...
All matched files use Prettier code style!

> sport-analytics-tool@0.1.0 lint
> npm run lint --workspaces --if-present

> @sport-analytics/backend@0.1.0 lint
> eslint src tests --ext .ts

> @sport-analytics/frontend@0.1.0 lint
> eslint src --ext .ts,.tsx

```

```

> @sport-analytics/contracts@0.1.0 lint
> eslint src --ext .ts

> sport-analytics-tool@0.1.0 typecheck
> npm run typecheck --workspaces --if-present

> @sport-analytics/backend@0.1.0 typecheck
> tsc --noEmit

> @sport-analytics/frontend@0.1.0 typecheck
> tsc --noEmit

> @sport-analytics/contracts@0.1.0 typecheck
> tsc --noEmit

> sport-analytics-tool@0.1.0 test
> npm run test --workspaces --if-present

> @sport-analytics/backend@0.1.0 test
> vitest run

```

RUN v2.1.9 C:/Users/shayn/OneDrive/Desktop/Sport-Analytics-Tool/apps/backend

```

{"level":30,"time":1785932762722,"pid":22492,"hostname":"ShaynaU-
Laptop","req":{"id":1,"method":"GET","url":"/api/v1/health","query":{"params":
{},"headers":{"host":"127.0.0.1:53079","accept-encoding":"gzip,
deflate","connection":"close"},"remoteAddress":"::ffff:127.0.0.1","remotePort":53080}
{"statusCode":200,"headers":{"content-security-policy":"default-src 'self';base-uri
'self';font-src 'self' https: data:;form-action 'self';frame-ancestors 'self';img-src 'self'
data:;object-src 'none';script-src 'self';script-src-attr 'none';style-src 'self' https:
'unsafe-inline';upgrade-insecure-requests","cross-origin-opener-policy":"same-
origin","cross-origin-resource-policy":"same-origin","origin-agent-cluster":"?
1","referrer-policy":"no-referrer","strict-transport-security":"max-age=31536000;
includeSubDomains","x-content-type-options":"nosniff","x-dns-prefetch-
control":"off","x-download-options":"noopen","x-frame-options":"SAMEORIGIN","x-
permitted-cross-domain-policies":"none","x-xss-
protection":"0","vary":"Origin","access-control-allow-credentials":"true","content-
type":"application/json; charset=utf-8","content-length":"86","etag":"W/\"56-
GqDG+eLCoUp98dWUCP8Wh4ZHMOI\""},"responseTime":2,"msg":"request
completed"}
✓ tests/health.test.ts (1)
✓ GET /api/v1/health (1)
 ✓ returns the API health state

```

```

Test Files 1 passed (1)
 Tests 1 passed (1)
Start at 14:26:00
Duration 2.23s (transform 331ms, setup 0ms, collect 1.72s, tests 21ms,
environment 0ms, prepare 232ms)

```

```

> @sport-analytics/frontend@0.1.0 test
> vitest run

```

RUN v2.1.9 C:/Users/shayn/OneDrive/Desktop/Sport-Analytics-Tool/apps/frontend

- ✓ src/App.test.tsx (1)
- ✓ App (1)
- ✓ shows the application heading and successful API state

Test Files 1 passed (1)

Tests 1 passed (1)

Start at 14:26:03

Duration 4.43s (transform 51ms, setup 567ms, collect 546ms, tests 52ms, environment 2.88s, prepare 123ms)

> @sport-analytics/contracts@0.1.0 test  
> vitest run --passWithNoTests

RUN v2.1.9 C:/Users/shayn/OneDrive/Desktop/Sport-Analytics-Tool/packages/contracts

include: \*\*/\*.{test,spec}.?(c|m)[jt]s?(x)

exclude: \*\*/node\_modules/\*\*, \*\*/dist/\*\*, \*\*/cypress/\*\*, \*\*/.

{idea,git,cache,output,temp}/\*\*,

\*\*/{karma,rollup,webpack,vite,vitest,jest,ava,babel,nyc,cypress,tsup,build,eslint,pretti

No test files found, exiting with code 0

> sport-analytics-tool@0.1.0 build  
> npm run build --workspace=@sport-analytics/contracts && npm run build --workspace=@sport-analytics/backend && npm run build --workspace=@sport-analytics/frontend

> @sport-analytics/contracts@0.1.0 build  
> tsc -p tsconfig.json

> @sport-analytics/backend@0.1.0 build  
> tsc -p tsconfig.json

> @sport-analytics/frontend@0.1.0 build  
> tsc -b && vite build

src/api/client.ts:10:56 - error TS2769: No overload matches this call.

Overload 1 of 2, '(input: string | URL | Request, init?: RequestInit | undefined): Promise<Response>', gave the following error.

Argument of type '{ headers: { Accept: string; }; signal: AbortSignal | undefined; }' is not assignable to parameter of type 'RequestInit' with 'exactOptionalPropertyTypes: true'. Consider adding 'undefined' to the types of the target's properties.

Types of property 'signal' are incompatible.

Type 'AbortSignal | undefined' is not assignable to type 'AbortSignal | null'.

Type 'undefined' is not assignable to type 'AbortSignal | null'.

Overload 2 of 2, '(input: URL | RequestInfo, init?: RequestInit | undefined): Promise<Response>', gave the following error.

Argument of type '{ headers: { Accept: string; }; signal: AbortSignal | undefined; }' is not assignable to parameter of type 'RequestInit' with

'exactOptionalPropertyTypes: true'. Consider adding 'undefined' to the types of the target's properties.

Types of property 'signal' are incompatible.

Type 'AbortSignal | undefined' is not assignable to type 'AbortSignal | null'.

```
10 const response = await fetch(`${apiBaseUrl}/health`, {
 ~
11 headers: {
    ~~~~~~
...
14   signal,
    ~~~~~~
15 });
    ~~~
```

Found 1 error.

npm error Lifecycle script `build` failed with error:

npm error code 2

npm error path C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool\apps\frontend

npm error workspace @sport-analytics/frontend@0.1.0

npm error location C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool\apps\frontend

npm error command failed

npm error command C:\Windows\system32\cmd.exe /d /s /c tsc -b && vite build

PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool>

8/5/2026, 2:26:29 PM



Excellent progress: structure, formatting, linting, type checking and all tests now pass. Only the frontend production build is failing.

The error is caused by `exactOptionalPropertyTypes`. The code is explicitly passing:

```
signal: undefined
```

when no abort signal is provided. The Fetch API expects the property either to be absent or to contain an `AbortSignal / null`.

Do not weaken the TypeScript configuration.

## Inspect the exact function

Run:

```
Get-Content apps/frontend/src/api/client.ts
```

Also run:

```
Get-Content apps/frontend/tsconfig.app.json
```

Send both outputs. We will then make the smallest type-safe change, most likely either conditionally adding `signal` or converting the missing value to `null`.

8/5/2026, 2:26:39 PM



```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> Get-Content
apps/frontend/src/api/client.ts
export interface HealthResponse {
  status: 'ok';
  service: string;
  timestamp: string;
}

const apiBaseUrl = import.meta.env.VITE_API_BASE_URL ??
'http://localhost:3000/api/v1';

export async function getHealth(signal?: AbortSignal): Promise<HealthResponse>
{
  const response = await fetch(`${apiBaseUrl}/health`, {
    headers: {
      Accept: 'application/json',
    },
    signal,
  });

  if (!response.ok) {
    throw new Error(`Health request failed with status ${response.status}`);
  }

  return (await response.json()) as HealthResponse;
}
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> Get-Content
apps/frontend/tsconfig.app.json
{
  "extends": "../../tsconfig.base.json",
  "compilerOptions": {
    "target": "ES2022",
    "useDefineForClassFields": true,
    "lib": ["ES2022", "DOM", "DOM.Iterable"],
    "allowJs": false,
    "module": "ESNext",
    "moduleResolution": "Bundler",
    "allowImportingTsExtensions": false,
    "resolveJsonModule": true,
    "isolatedModules": true,
    "noEmit": true,
    "jsx": "react-jsx"
  },
  "include": ["src"]
}
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool>
```

8/5/2026, 2:27:17 PM



Make the smallest type-safe change in `apps/frontend/src/api/client.ts`.

Replace:

```
signal,
```

with:



```
signal: signal ?? null,
```

That section should become:

```
const response = await fetch(`${apiBaseUrl}/health`, {
  headers: {
    Accept: 'application/json',
  },
  signal: signal ?? null,
});
```

This preserves the optional function parameter while giving `fetch` either a real `AbortSignal` or `null`, never `undefined`.

Then run:

```
npx.cmd prettier --write apps/frontend/src/api/client.ts
```

```
npm.cmd run build --workspace=@sport-analytics/frontend
```

If the frontend build succeeds, run the complete check:

```
npm.cmd run check
```

Send the final output from `npm.cmd run check`. Do not commit yet.

8/5/2026, 2:27:26 PM



```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> npm.cmd run build --
workspace=@sport-analytics/frontend

> @sport-analytics/frontend@0.1.0 build
> tsc -b && vite build

vite v5.4.21 building for production...
✓ 32 modules transformed.
dist/index.html          0.62 kB | gzip: 0.37 kB
dist/assets/index-DqRmXqCP.css 1.21 kB | gzip: 0.65 kB
dist/assets/index-RwBW4OZx.js 144.02 kB | gzip: 46.51 kB
✓ built in 472ms
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> npm.cmd run check

> sport-analytics-tool@0.1.0 check
> npm run structure:check && npm run format:check && npm run lint && npm run
typecheck && npm run test && npm run build

> sport-analytics-tool@0.1.0 structure:check
> node scripts/check-required-files.mjs

Repository structure check passed (12 required files).

> sport-analytics-tool@0.1.0 format:check
> prettier --check .

Checking formatting...
All matched files use Prettier code style!

> sport-analytics-tool@0.1.0 lint
> npm run lint --workspaces --if-present

> @sport-analytics/backend@0.1.0 lint
> eslint src tests --ext .ts

> @sport-analytics/frontend@0.1.0 lint
> eslint src --ext .ts,.tsx

> @sport-analytics/contracts@0.1.0 lint
> eslint src --ext .ts

> sport-analytics-tool@0.1.0 typecheck
> npm run typecheck --workspaces --if-present

> @sport-analytics/backend@0.1.0 typecheck
> tsc --noEmit

> @sport-analytics/frontend@0.1.0 typecheck
> tsc --noEmit
```

```
> @sport-analytics/contracts@0.1.0 typecheck
> tsc --noEmit
```

```
> sport-analytics-tool@0.1.0 test
> npm run test --workspaces --if-present
```

```
> @sport-analytics/backend@0.1.0 test
> vitest run
```

RUN v2.1.9 C:/Users/shayn/OneDrive/Desktop/Sport-Analytics-Tool/apps/backend

```

> tests/health.test.ts (1)
  > GET /api/v1/health (1)
    \ returns the API health state
    {"level":30,"time":1785932921846,"pid":15480,"hostname":"ShaynaU-
    Laptop","req":{"id":1,"method":"GET","url":"/api/v1/health","query":{"params":
    {},"headers":{"host":"127.0.0.1:61903","accept-encoding":"gzip,
    deflate","connection":"close"},"remoteAddress":"::ffff:127.0.0.1","remotePort":61904}
    {"statusCode":200,"headers":{"content-security-policy":"default-src 'self';base-uri
    'self';font-src 'self' https: data:;form-action 'self';frame-ancestors 'self';img-src 'self'
    data:;object-src 'none';script-src 'self';script-src-attr 'none';style-src 'self' https:
    'unsafe-inline';upgrade-insecure-requests","cross-origin-opener-policy":"same-
    origin","cross-origin-resource-policy":"same-origin","origin-agent-cluster":"?
    1","referrer-policy":"no-referrer","strict-transport-security":"max-age=31536000;
    includeSubDomains","x-content-type-options":"nosniff","x-dns-prefetch-
    control":"off","x-download-options":"noopen","x-frame-options":"SAMEOR
    ✓ tests/health.test.ts (1)
      ✓ GET /api/v1/health (1)
        ✓ returns the API health state
```

```
Test Files 1 passed (1)
  Tests 1 passed (1)
  Start at 14:28:41
  Duration 842ms (transform 296ms, setup 0ms, collect 442ms, tests 21ms,
  environment 0ms, prepare 114ms)
```

```
> @sport-analytics/frontend@0.1.0 test
> vitest run
```

RUN v2.1.9 C:/Users/shayn/OneDrive/Desktop/Sport-Analytics-Tool/apps/frontend

```

✓ src/App.test.tsx (1)
  ✓ App (1)
    ✓ shows the application heading and successful API state
```

```
Test Files 1 passed (1)
  Tests 1 passed (1)
  Start at 14:28:42
  Duration 964ms (transform 50ms, setup 67ms, collect 85ms, tests 53ms,
  environment 376ms, prepare 110ms)
```

```
> @sport-analytics/contracts@0.1.0 test
> vitest run --passWithNoTests
```

```

RUN v2.1.9 C:/Users/shayn/OneDrive/Desktop/Sport-Analytics-
Tool/packages/contracts

include: **/*.{test,spec}.?(c|m)[jt]s?(x)
exclude: **/node_modules/**, **/dist/**, **/cypress/**, **/.
{idea,git,cache,output,temp}/**,
**/{karma,rollup,webpack,vite,vitest,jest,ava,babel,nyc,cypress,tsup,build,eslint,pretti
No test files found, exiting with code 0

> sport-analytics-tool@0.1.0 build
> npm run build --workspace=@sport-analytics/contracts && npm run build --
workspace=@sport-analytics/backend && npm run build --workspace=@sport-
analytics/frontend

> @sport-analytics/contracts@0.1.0 build
> tsc -p tsconfig.json

> @sport-analytics/backend@0.1.0 build
> tsc -p tsconfig.json

> @sport-analytics/frontend@0.1.0 build
> tsc -b && vite build

vite v5.4.21 building for production...
✓ 32 modules transformed.
dist/index.html          0.62 kB | gzip: 0.37 kB
dist/assets/index-DqRmXqCP.css 1.21 kB | gzip: 0.65 kB
dist/assets/index-RwBW4OZx.js 144.02 kB | gzip: 46.51 kB
✓ built in 453ms
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool>

```

8/5/2026, 2:29:01 PM



Excellent — the **entire repository check now passes:**

- structure check;
- formatting;
- linting;
- type checking;
- backend and frontend tests;
- all production builds.

The temporary `>` shown during the backend test was just Vitest's live progress display; the final result was `1 passed`.

## Next: prove the lockfile is reproducible

First check the current changes:

```
git status --short
```

Then perform a clean installation using only `package-lock.json`:

```
npm.cmd ci
```

This intentionally removes the existing `node_modules` directory and reinstalls the exact locked dependency versions.

Do not run `npm audit fix`.

After installation, run:

```
npm.cmd run check
```

Finally:

```
git status --short
```

A successful `npm ci` followed by a successful full check will prove that CI can safely replace:

```
run: npm install
```

with:

```
run: npm ci
```

Send the `npm.cmd ci` result, the final result of `npm.cmd run check`, and the updated Git status. Do not commit yet.

8/5/2026, 2:29:16 PM



```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> git status --short
M .eslintrc.cjs
M .gitea/ISSUE_TEMPLATE/bug.md
M .gitea/ISSUE_TEMPLATE/feature.md
M .gitea/ISSUE_TEMPLATE/task.md
M .gitea/PULL_REQUEST_TEMPLATE.md
M .prettierrc.json
M apps/frontend/src/App.tsx
M apps/frontend/src/api/client.ts
M apps/frontend/src/styles.css
M apps/frontend/tsconfig.json
M docs/development/dependencies.md
M docs/development/testing.md
M docs/git-methodology.md
M docs/project_methodology.md
M evidence/decisions/ADR-001-initial-repository-setup.md
M package.json
M scripts/check-required-files.mjs
?? package-lock.json
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> npm.cmd ci
npm warn deprecated inflight@1.0.6: This module is not supported, and leaks
memory. Do not use it. Check out lru-cache if you want a good and tested way to
coalesce async requests by a key value, which is much more comprehensive and
powerful.
npm warn deprecated @humanwhocodes/config-array@0.13.0: Use @eslint/config-
array instead
npm warn deprecated rimraf@3.0.2: Rimraf versions prior to v4 are no longer
supported
npm warn deprecated glob@7.2.3: Old versions of glob are not supported, and
contain widely publicized security vulnerabilities, which have been fixed in the
current version. Please update. Support for old versions may be purchased (at
exorbitant rates) by contacting i@izs.me
npm warn deprecated whatwg-encoding@3.1.1: Use @exodus/bytes instead for a
more spec-conformant and faster implementation
npm warn deprecated @humanwhocodes/object-schema@2.0.3: Use
@eslint/object-schema instead
npm warn deprecated eslint@8.57.1: This version is no longer supported. Please
see https://eslint.org/version-support for other options.

added 418 packages, and audited 422 packages in 8s

84 packages are looking for funding
  run `npm fund` for details

5 vulnerabilities (3 moderate, 1 high, 1 critical)

To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool>
```

8/5/2026, 2:30:09 PM



`npm.cmd ci` succeeded, so the new lockfile supports a clean, reproducible installation.

The vulnerability and deprecation messages are warnings, not the reason CI was failing. Do not run `npm audit fix --force` during this issue.

Now verify the clean installation still passes every repository check:

```
npm.cmd run check
```

When it finishes, run:

```
git status --short
```

Send the final part of the check output and the Git status. Once it passes, we will update `.gitea/workflows/ci.yml` from `npm install` to `npm ci`.

*8/5/2026, 2:30:21 PM*



```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> git status --short
M .eslintrc.cjs
M .gitea/ISSUE_TEMPLATE/bug.md
M .gitea/ISSUE_TEMPLATE/feature.md
M .gitea/ISSUE_TEMPLATE/task.md
M .gitea/PULL_REQUEST_TEMPLATE.md
M .prettierrc.json
M apps/frontend/src/App.tsx
M apps/frontend/src/api/client.ts
M apps/frontend/src/styles.css
M apps/frontend/tsconfig.json
M docs/development/dependencies.md
M docs/development/testing.md
M docs/git-methodology.md
M docs/project_methodology.md
M evidence/decisions/ADR-001-initial-repository-setup.md
M package.json
M scripts/check-required-files.mjs
?? package-lock.json
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool>
```

8/5/2026, 2:31:21 PM



That is correct. `npm.cmd ci` did not create any unexpected changes; the repository still contains only the fixes we have intentionally made.

Now run the full check against the clean installation:

```
npm.cmd run check
```

When it finishes successfully, update the CI workflow:

```
(Get-Content .gitea/workflows/ci.yml) -replace 'run: npm install', 'run: npm ci' | Set-Content
.gitea/workflows/ci.yml
```

Then format and verify the workflow:

```
npx.cmd prettier --write .gitea/workflows/ci.yml
```

```
Get-Content .gitea/workflows/ci.yml
```

```
npm.cmd run check
```

The installation section should now read:

```
- name: Install dependencies
  run: npm ci
```

Send the final result of `npm.cmd run check` and the displayed `ci.yml`.

8/5/2026, 2:31:31 PM



```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> Get-Content
.gitea/workflows/ci.yml
name: Continuous Integration
```

```
on:
  push:
    branches:
      - main
  pull_request:

jobs:
  quality:
    runs-on: ubuntu-latest
    timeout-minutes: 15

    steps:
      - name: Check out repository
        uses: actions/checkout@v4

      - name: Set up Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20

      - name: Install dependencies
        run: npm ci

      - name: Run repository checks
        run: npm run check
```

```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> npm.cmd run check
```

```
> sport-analytics-tool@0.1.0 check
> npm run structure:check && npm run format:check && npm run lint && npm run
typecheck && npm run test && npm run build
```

```
> sport-analytics-tool@0.1.0 structure:check
> node scripts/check-required-files.mjs
```

Repository structure check passed (12 required files).

```
> sport-analytics-tool@0.1.0 format:check
> prettier --check .
```

Checking formatting...  
All matched files use Prettier code style!

```
> sport-analytics-tool@0.1.0 lint
> npm run lint --workspaces --if-present
```

```
> @sport-analytics/backend@0.1.0 lint
> eslint src tests --ext .ts
```

```
> @sport-analytics/frontend@0.1.0 lint
> eslint src --ext .ts,.tsx
```

```

> @sport-analytics/contracts@0.1.0 lint
> eslint src --ext .ts

> sport-analytics-tool@0.1.0 typecheck
> npm run typecheck --workspaces --if-present

> @sport-analytics/backend@0.1.0 typecheck
> tsc --noEmit

> @sport-analytics/frontend@0.1.0 typecheck
> tsc --noEmit

> @sport-analytics/contracts@0.1.0 typecheck
> tsc --noEmit

> sport-analytics-tool@0.1.0 test
> npm run test --workspaces --if-present

> @sport-analytics/backend@0.1.0 test
> vitest run

```

RUN v2.1.9 C:/Users/shayn/OneDrive/Desktop/Sport-Analytics-Tool/apps/backend

```

{"level":30,"time":1785933134801,"pid":7256,"hostname":"ShaynaU-
Laptop","req":{"id":1,"method":"GET","url":"/api/v1/health","query":{"params":
{},"headers":{"host":"127.0.0.1:62626","accept-encoding":"gzip,
deflate","connection":"close"},"remoteAddress":"::ffff:127.0.0.1","remotePort":62627}}
{"statusCode":200,"headers":{"content-security-policy":"default-src 'self';base-uri
'self';font-src 'self' https: data:;form-action 'self';frame-ancestors 'self';img-src 'self'
data:;object-src 'none';script-src 'self';script-src-attr 'none';style-src 'self' https:
'unsafe-inline';upgrade-insecure-requests","cross-origin-opener-policy":"same-
origin","cross-origin-resource-policy":"same-origin","origin-agent-cluster":"?
1","referrer-policy":"no-referrer","strict-transport-security":"max-age=31536000;
includeSubDomains","x-content-type-options":"nosniff","x-dns-prefetch-
control":"off","x-download-options":"noopen","x-frame-options":"SAMEORIGIN","x-
permitted-cross-domain-policies":"none","x-xss-
protection":"0","vary":"Origin","access-control-allow-credentials":"true","content-
type":"application/json; charset=utf-8","content-length":"86","etag":"W/\"56-
JXuKWSFaKfxRLcXkWnsQsgGOzlU\""},\"responseTime\":3,\"msg\":\"request
completed\"}
✓ tests/health.test.ts (1)
✓ GET /api/v1/health (1)
  ✓ returns the API health state

```

Test Files 1 passed (1)

Tests 1 passed (1)

Start at 14:32:13

Duration 1.44s (transform 246ms, setup 0ms, collect 1.03s, tests 21ms, environment 0ms, prepare 128ms)

```

> @sport-analytics/frontend@0.1.0 test

```

```

> vitest run

RUN v2.1.9 C:/Users/shayn/OneDrive/Desktop/Sport-Analytics-Tool/apps/frontend

✓ src/App.test.tsx (1)
✓ App (1)
  ✓ shows the application heading and successful API state

Test Files  1 passed (1)
Tests       1 passed (1)
Start at    14:32:15
Duration    2.43s (transform 48ms, setup 225ms, collect 373ms, tests 49ms,
environment 1.43s, prepare 105ms)

> @sport-analytics/contracts@0.1.0 test
> vitest run --passWithNoTests

RUN v2.1.9 C:/Users/shayn/OneDrive/Desktop/Sport-Analytics-
Tool/packages/contracts

include: **/*.test.spec.?(c|m)[jt]s?(x)
exclude: **/node_modules/**, **/dist/**, **/cypress/**, **/.
{idea,git,cache,output,temp}/**,
**/{karma,rollup,webpack,vite,vitest,jest,ava,babel,nyc,cypress,tsup,build,eslint,pretti
No test files found, exiting with code 0

> sport-analytics-tool@0.1.0 build
> npm run build --workspace=@sport-analytics/contracts && npm run build --
workspace=@sport-analytics/backend && npm run build --workspace=@sport-
analytics/frontend

> @sport-analytics/contracts@0.1.0 build
> tsc -p tsconfig.json

> @sport-analytics/backend@0.1.0 build
> tsc -p tsconfig.json

> @sport-analytics/frontend@0.1.0 build
> tsc -b && vite build

vite v5.4.21 building for production...
✓ 32 modules transformed.
dist/index.html          0.62 kB | gzip: 0.37 kB
dist/assets/index-DqRmXqCP.css 1.21 kB | gzip: 0.65 kB
dist/assets/index-RwBW4OZx.js 144.02 kB | gzip: 46.51 kB
✓ built in 446ms

```

8/5/2026, 2:32:38 PM



Thinking

- **Reviewing local changes**

I'm treating the checks as successful while noting the dependency warnings separately. Before committing, I'm reviewing the diff for unintended semantic changes, confirming the branch, then I'll commit and push so CI can reveal any runner issues.

- **Reviewed local changes**

- **Reviewing substantive edits**

I'm separating likely formatting-only changes from meaningful configuration, dependency, API, and CI edits. I'm checking those focused diffs first, then I'll stage changes in logical commits and push only after the content looks intentional.

- **Reviewed substantive edits**

✓ Worked for 42s

Excellent — the repository now passes the **complete quality pipeline after a clean `npm ci` installation**. That proves the lockfile and CI command are reproducible.

Before committing, we need to review and separate the substantive changes from the mechanical formatting changes.

## Review the changes

First confirm the branch:

```
git branch --show-current
```

It must show:

```
ci/10-restore-green-ci
```

Then display the five substantive changes:

```
git diff -- .eslintrc.cjs .prettierrc.json package.json apps/frontend/src/api/client.ts .gitea/workflows/ci.yml
```

Now show the overall change summary:

```
git diff --stat
```

Finally verify that there are no whitespace errors:

```
git diff --check
```

`git diff --check` should produce no errors; the LF replacement notices are harmless warnings.

Send me those outputs. We will then divide the work into small atomic commits rather than putting all 18 files into one commit.

8/5/2026, 2:33:05 PM



```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> git diff --check
warning: in the working copy of '.eslintrc.cjs', CRLF will be replaced by LF the next
time Git touches it
warning: in the working copy of '.gitea/workflows/ci.yml', CRLF will be replaced by
LF the next time Git touches it
warning: in the working copy of '.prettierrc.json', CRLF will be replaced by LF the
next time Git touches it
warning: in the working copy of 'apps/frontend/src/App.tsx', CRLF will be replaced
by LF the next time Git touches it
warning: in the working copy of 'apps/frontend/src/api/client.ts', CRLF will be
replaced by LF the next time Git touches it
warning: in the working copy of 'apps/frontend/src/styles.css', CRLF will be
replaced by LF the next time Git touches it
warning: in the working copy of 'apps/frontend/tsconfig.json', CRLF will be replaced
by LF the next time Git touches it
warning: in the working copy of 'docs/development/dependencies.md', CRLF will be
replaced by LF the next time Git touches it
warning: in the working copy of 'docs/development/testing.md', CRLF will be
replaced by LF the next time Git touches it
warning: in the working copy of 'docs/git-methodology.md', CRLF will be replaced
by LF the next time Git touches it
warning: in the working copy of 'docs/project_methodology.md', CRLF will be
replaced by LF the next time Git touches it
warning: in the working copy of 'scripts/check-required-files.mjs', CRLF will be
replaced by LF the next time Git touches it
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> git diff --check
```

8/5/2026, 2:34:38 PM



## Thinking

- **Reviewing the diff**

I'm seeing no whitespace errors; the line-ending warnings are expected normalization notices. Before committing, I need the branch, full diff summary, and substantive-file diff so I can group changes into clean atomic commits.

- **Reviewed the diff**

✓ Worked for 16s

The check passed. Those messages are only Git confirming that `.gitattributes` will store the files with `LF` line endings. There are no whitespace errors.

Before staging anything, please run the three missing review commands:

```
git branch --show-current
```

Expected:

```
ci/10-restore-green-ci
```

Then:

```
git diff -- .eslintrc.cjs .prettierrc.json package.json apps/frontend/src/api/client.ts .gitea/workflows/ci.yml
```

And:

```
git diff --stat
```

Also confirm the untracked lockfile exists:

```
Get-Item package-lock.json | Select-Object Name, Length
```

Send those outputs. We will then separate the changes into atomic commits, likely covering formatting, dependency reproducibility, lint configuration, the frontend build fix and the CI workflow.

*8/5/2026, 2:34:48 PM*



```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> git branch --show-current
ci/10-restore-green-ci
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> git diff -- .eslintrc.cjs
.prettierrc.json package.json apps/frontend/src/api/client.ts .gitea/workflows/ci.yml
warning: in the working copy of '.eslintrc.cjs', CRLF will be replaced by LF the next
time Git touches it
warning: in the working copy of '.gitea/workflows/ci.yml', CRLF will be replaced by
LF the next time Git touches it
warning: in the working copy of '.prettierrc.json', CRLF will be replaced by LF the
next time Git touches it
warning: in the working copy of 'apps/frontend/src/api/client.ts', CRLF will be
replaced by LF the next time Git touches it
diff --git a/.eslintrc.cjs b/.eslintrc.cjs
index 3ac0ff7..300dbbd 100644
--- a/.eslintrc.cjs
+++ b/.eslintrc.cjs
@@ -11,6 +11,9 @@ module.exports = {
  },
  plugins: ['@typescript-eslint'],
  extends: ['eslint:recommended', 'plugin:@typescript-eslint/recommended'],
+ rules: {
+   '@typescript-eslint/no-unused-vars': ['error', { argsIgnorePattern: '^_' }],
+ },
  ignorePatterns: ['**/dist/**', '**/coverage/**', 'site/**'],
  overrides: [
    {
diff --git a/.gitea/workflows/ci.yml b/.gitea/workflows/ci.yml
index 888e445..5080ee1 100644
--- a/.gitea/workflows/ci.yml
+++ b/.gitea/workflows/ci.yml
@@ -21,7 +21,7 @@ jobs:
   node-version: 20

   - name: Install dependencies
-    run: npm install
+    run: npm ci

   - name: Run repository checks
    run: npm run check
diff --git a/.prettierrc.json b/.prettierrc.json
index 6fc6d66..c54406e 100644
--- a/.prettierrc.json
+++ b/.prettierrc.json
@@ -2,5 +2,6 @@
  "singleQuote": true,
  "trailingComma": "all",
  "semi": true,
- "printWidth": 100
+ "printWidth": 100,
+ "endOfLine": "auto"
}
diff --git a/apps/frontend/src/api/client.ts b/apps/frontend/src/api/client.ts
index c41f9c0..5c1a431 100644
--- a/apps/frontend/src/api/client.ts
+++ b/apps/frontend/src/api/client.ts
@@ -11,7 +11,7 @@ export async function getHealth(signal?: AbortSignal):
Promise<HealthResponse> {
```

```

    headers: {
      Accept: 'application/json',
    },
    - signal,
    + signal: signal ?? null,
  });

  if (!response.ok) {
diff --git a/package.json b/package.json
index a444aba..f88167b 100644
--- a/package.json
+++ b/package.json
@@ -29,6 +29,6 @@
   "eslint": "^8.57.1",
   "eslint-plugin-react-hooks": "^4.6.2",
   "prettier": "^3.3.3",
-  "typescript": "^5.6.3"
+  "typescript": "5.5.4"
  }
}

```

PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> git diff --stat

warning: in the working copy of '.eslintrc.cjs', CRLF will be replaced by LF the next time Git touches it

warning: in the working copy of '.gitea/workflows/ci.yml', CRLF will be replaced by LF the next time Git touches it

warning: in the working copy of '.prettierrc.json', CRLF will be replaced by LF the next time Git touches it

warning: in the working copy of 'apps/frontend/src/App.tsx', CRLF will be replaced by LF the next time Git touches it

warning: in the working copy of 'apps/frontend/src/api/client.ts', CRLF will be replaced by LF the next time Git touches it

warning: in the working copy of 'apps/frontend/src/styles.css', CRLF will be replaced by LF the next time Git touches it

warning: in the working copy of 'apps/frontend/tsconfig.json', CRLF will be replaced by LF the next time Git touches it

warning: in the working copy of 'docs/development/dependencies.md', CRLF will be replaced by LF the next time Git touches it

warning: in the working copy of 'docs/development/testing.md', CRLF will be replaced by LF the next time Git touches it

warning: in the working copy of 'docs/git-methodology.md', CRLF will be replaced by LF the next time Git touches it

warning: in the working copy of 'docs/project\_methodology.md', CRLF will be replaced by LF the next time Git touches it

warning: in the working copy of 'scripts/check-required-files.mjs', CRLF will be replaced by LF the next time Git touches it

```

.eslintrc.cjs | 3 +
.gitea/ISSUE_TEMPLATE/bug.md | 16 ++---
.gitea/ISSUE_TEMPLATE/feature.md | 18 ++++---
.gitea/ISSUE_TEMPLATE/task.md | 6 +-
.gitea/PULL_REQUEST_TEMPLATE.md | 4 +-
.gitea/workflows/ci.yml | 2 +-
.prettierrc.json | 3 +-
apps/frontend/src/App.tsx | 4 +-
apps/frontend/src/api/client.ts | 2 +-
apps/frontend/src/styles.css | 8 ++-
apps/frontend/tsconfig.json | 5 +-
docs/development/dependencies.md | 20 ++++---
docs/development/testing.md | 18 ++++---
docs/git-methodology.md | 68 ++++++++-----
docs/project_methodology.md | 55 +++++-----

```

```
.../decisions/ADR-001-initial-repository-setup.md | 2 +-  
package.json | 2 +-  
scripts/check-required-files.mjs | 4 +-  
18 files changed, 103 insertions(+), 137 deletions(-)
```

```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> git diff --checkGet-Item  
package-lock.json | Select-Object Name, Length  
error: invalid option: --checkGet-Item  
usage: git diff [<options>] [<commit>] [--] [<path>...]  
or: git diff [<options>] --cached [--merge-base] [<commit>] [--] [<path>...]  
or: git diff [<options>] [--merge-base] <commit> [<commit>...] <commit> [--]  
[<path>...]  
or: git diff [<options>] <commit>...<commit> [--] [<path>...]  
or: git diff [<options>] <blob> <blob>  
or: git diff [<options>] --no-index [--] <path> <path> [<paths-spec>...]
```

common diff options:

- z output diff raw with lines terminated with NUL.
- p output patch format.
- u synonym for -p.
- patch-with-raw  
output both a patch and the diff raw format.
- stat show diffstat instead of patch.
- numstat show numeric diffstat instead of patch.
- patch-with-stat  
output a patch and prepend its diffstat.
- name-only show only names of changed files.
- name-status show names and status of changed files.
- full-index show full object name on index lines.
- abbrev=<n> abbreviate object names in diff-tree header and diff raw.
- R swap input file pairs.
- B detect complete rewrites.
- M detect renames.
- C detect copies.
- find-copies-harder  
try unchanged files as candidate for copy detection.
- l<n> limit rename attempts up to <n> paths.
- O<file> reorder diffs according to the <file>.
- S<string> find filepair whose only one side contains the string.
- pickaxe-all  
show all files diff when -S is used and hit is found.
- a --text treat all files as text.

```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool>
```

8/5/2026, 2:35:52 PM



## Thinking

- **Organizing commits**

The combined command isn't a problem; it just means the results run together. I'm grouping the changes into clean commits: formatting, dependency locking and TypeScript compatibility, lint fixes, and CI's reproducible install behavior.

- **Organized commits**

✓ Worked for 38s

The error is harmless—you accidentally joined two commands together. Nothing changed.

Your changes divide cleanly into atomic commits. Start with the formatting corrections.

## Commit 1: Repository formatting

Stage only the formatting configuration and files Prettier corrected:

```
git add .prettierrc.json .gitea/ISSUE_TEMPLATE/bug.md .gitea/ISSUE_TEMPLATE/feature.md .gitea/ISSUE_TEMPLATE/task.md
.gitea/PULL_REQUEST_TEMPLATE.md
```

```
git add apps/frontend/src/App.tsx apps/frontend/src/styles.css apps/frontend/tsconfig.json scripts/check-required-
files.mjs
```

```
git add docs/development/dependencies.md docs/development/testing.md docs/git-methodology.md
docs/project_methodology.md evidence/decisions/ADR-001-initial-repository-setup.md
```

Confirm exactly what is staged:

```
git diff --cached --stat
```

It should show the formatting-related files, but **not**:

- `.eslintrc.cjs`
- `.gitea/workflows/ci.yml`
- `apps/frontend/src/api/client.ts`
- `package.json`
- `package-lock.json`

Then commit:

```
git commit -m "style: align repository formatting" -m "Refs #10" -m "Assisted-by: ChatGPT-Web[GPT-5.6 Thinking]"
```

After committing, run:

```
git status --short
```

The remaining files should be:

```
M .eslintrc.cjs
M .gitea/workflows/ci.yml
M apps/frontend/src/api/client.ts
M package.json
?? package-lock.json
```

Send the output of `git status --short` after the commit.

8/5/2026, 2:36:06 PM

[next](#)

8/5/2026, 2:37:42 PM



## Commit 2: Reproducible dependencies

First confirm the remaining changes:

```
git status --short
```

You should see:

```
M .eslintrc.cjs
M .gitea/workflows/ci.yml
M apps/frontend/src/api/client.ts
M package.json
?? package-lock.json
```

Stage only the dependency files:

```
git add package.json package-lock.json
```

Check what is staged:

```
git diff --cached --stat
```

Then commit:

```
git commit -m "build: lock reproducible dependency versions" -m "Refs #10" -m "Assisted-by: ChatGPT-Web[GPT-5.6 Thinking]"
```

Finally:

```
git status --short
```

The remaining files should now be:

```
M .eslintrc.cjs
M .gitea/workflows/ci.yml
M apps/frontend/src/api/client.ts
```

8/5/2026, 2:37:54 PM

[next](#)

8/5/2026, 2:38:29 PM



### Commit 3: Allow intentionally unused Express parameters

Stage only the ESLint configuration:

```
git add .eslintrc.cjs
```

Check the staged change:

```
git diff --cached
```

It should only show the rule allowing underscore-prefixed unused parameters.

Commit it:

```
git commit -m "chore(lint): allow intentionally unused parameters" -m "Refs #10" -m "Assisted-by: ChatGPT-Web[GPT-5.6 Thinking]"
```

Then check what remains:

```
git status --short
```

Expected remaining files:

```
M .gitea/workflows/ci.yml
M apps/frontend/src/api/client.ts
```

8/5/2026, 2:38:37 PM

[next](#)

8/5/2026, 2:39:04 PM



## Commit 4: Fix the frontend production build

Stage only the API client change:

```
git add apps/frontend/src/api/client.ts
```

Review the staged diff:

```
git diff --cached
```

It should only show:

```
signal: signal ?? null,
```

Commit it:

```
git commit -m "fix(frontend): handle optional abort signal" -m "Refs #10" -m "Assisted-by: ChatGPT-Web[GPT-5.6 Thinking]"
```

Then check what remains:

```
git status --short
```

Expected:

```
M .gitea/workflows/ci.yml
```

8/5/2026, 2:39:11 PM

[next](#)

8/5/2026, 2:39:37 PM



## Commit 5: Use reproducible installation in CI

Stage the workflow change:

```
git add .gitea/workflows/ci.yml
```

Review it:

```
git diff --cached
```

It should only show:

```
- run: npm install
+ run: npm ci
```

Commit it:

```
git commit -m "ci: use reproducible dependency installation" -m "Refs #10" -m "Assisted-by: ChatGPT-Web[GPT-5.6 Thinking]"
```

Then confirm the branch is clean:

```
git status
```

Expected:

```
nothing to commit, working tree clean
```

8/5/2026, 2:39:45 PM



next  
do we need to update documentation b4 we push?

8/5/2026, 2:40:18 PM



### Thinking

- **Reviewing documentation needs**

Yes, I'm checking this before the push. Because the changes affect CI, dependency installation, TypeScript configuration, linting, and development setup, I'm treating documentation updates as part of the issue—not optional cleanup.

I'm first reviewing the existing setup, dependency, and testing guides so I can make only targeted edits. I'm also updating the AI-use register before committing.

- **Reviewed documentation needs**

✓ Worked for 13s

Yes — **a small, focused documentation update is appropriate before pushing**. The technical behaviour changed in ways a new teammate needs to know:

- `package-lock.json` is now committed.
- Normal setup should use `npm ci`.
- TypeScript is pinned to `5.5.4`.
- `npm run check` is the local equivalent of the CI quality job.
- The AI usage register should record the assistance on Issue #10.

We do **not** need to rewrite the broader methodology documents again.

Before editing, inspect the relevant files:

```
Get-Content docs/development/setup.md
```

```
Get-Content docs/development/dependencies.md
```

```
Get-Content docs/development/testing.md
```

Then check the AI evidence location:

```
Test-Path evidence/ai/ai-usage-register.csv
```

If that returns `True`, run:

```
Get-Content evidence/ai/ai-usage-register.csv
```

Send those outputs. We will make only the necessary documentation changes, run the full check again, and create one final documentation commit before pushing.

8/5/2026, 2:40:29 PM



```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> Get-Content  
docs/development/setup.md  
# Local development setup
```

### ## Prerequisites

- Node.js 20 or later
- npm 10 or later
- Python 3 for the documentation site
- Access to the selected PostgreSQL development environment

### ## Install

```
```bash  
git clone https://sdp.ms.wits.ac.za/git-push-pray/Sport-Analytics-Tool.git  
cd Sport-Analytics-Tool  
npm install  
cp apps/backend/.env.example apps/backend/.env  
cp apps/frontend/.env.example apps/frontend/.env  
```
```

Do not place real secrets in committed files.

### ## Run

Backend:

```
```bash  
npm run dev:backend  
```
```

Frontend:

```
```bash  
npm run dev:frontend  
```
```

Documentation:

```
```bash  
python -m pip install -r requirements-docs.txt  
mkdocs serve  
```
```

### ## Verify before a Pull Request

```
```bash  
npm run check  
```
```

The first `npm install` should create and commit `package-lock.json`. Once it exists, CI should be changed from `npm install` to `npm ci` for reproducible installs.

### ## Configuration ownership

- Frontend variables must be public-safe and use the `VITE\_` prefix.
- Database URLs, provider secrets, and external API keys belong only in

backend/deployment secret stores.

- Environment variables must be validated at backend startup.
- Document each variable, its owner, and which deployment supplies it.

```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> Get-Content docs/development/dependencies.md
```

# Dependencies

## ## Selection principles

A dependency must have a clear project purpose, an acceptable licence, active maintenance, security support, and a cost lower than implementing and maintaining the capability safely yourselves.

## ## Initial dependencies

| Area                  | Dependency                         | Purpose                                           |
|-----------------------|------------------------------------|---------------------------------------------------|
| Frontend              | React                              | Component-based web user interface                |
| Frontend tooling      | Vite                               | Development server and production build           |
| Backend middleware    | Express                            | Hand-written HTTP routing and                     |
| Validation            | Zod                                | Runtime validation and TypeScript type derivation |
| HTTP security         | Helmet                             | Secure response-header defaults                   |
| Logging               | Pino HTTP                          | Structured request logging foundation             |
| Testing               | Vitest, Testing Library, Supertest | Component, unit, and API integration tests        |
| Documentation website | MkDocs Material                    | Public static documentation                       |

## ## Review requirements

Before each milestone:

- review direct and transitive vulnerabilities;
- remove unused packages;
- document newly added third-party services and code;
- verify licences and attribution needs;
- record significant dependency decisions in an ADR; and
- retest the application after upgrades.

Exact installed versions will be recorded by `package-lock.json` after the first verified installation.

```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> Get-Content docs/development/testing.md
```

# Testing strategy

## ## Test layers

| Layer           | Location                      | Purpose                              |
|-----------------|-------------------------------|--------------------------------------|
| Unit            | Near frontend/backend modules | Verify isolated rules and components |
| API integration | `apps/backend/tests`          | Verify routes, validation, errors,   |

```

and persistence boundaries |
| UI component | `apps/frontend/src/**/*.test.tsx` | Verify user-visible behaviour
and accessibility semantics |
| End-to-end | `tests/e2e` | Verify critical user journeys against
deployed-like services |
| Accessibility | `tests/accessibility` | Automate and record accessibility
checks |
| Performance | `tests/performance` | Verify stated API response-time
and load targets |
| User testing | `evidence/user-testing` | Record formal tasks, participants,
observations, and evaluated changes |

```

## ## Required practice

- Add meaningful tests for every behavioural change.
- Test success, validation failure, unauthorised/forbidden access, conflict, not-found, dependency failure, and retry/idempotency behaviour where relevant.
- Compare derived statistics against independently prepared reference results.
- Use deterministic fixtures and isolate tests from production data.
- Do not rely only on generated/default tests.
- Record known gaps as Gitea issues with risk and planned resolution.

## ## Current scaffold tests

The scaffold contains:

- a backend health-endpoint integration test; and
- a frontend rendering/API-state component test.

These prove the test runners are wired, not that the product is adequately tested.

## ## CI expectation

Pull Requests should fail when formatting, linting, type checking, tests, structure validation, or builds fail. Deployment jobs should run only from reviewed branches/tags and should use environment-specific secrets.

```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> Test-Path
evidence/ai/ai-usage-register.csv
```

```
True
```

```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> Get-Content
evidence/ai/ai-usage-register.csv
```

```
Date,Team member,Tool,Model,Purpose,Brief task,Output used,Verification or
adaptation,Related evidence
```

```
2026-08-04,Dean Feldman,ChatGPT Web,GPT-5.6 Thinking,Planning; technical
guidance; documentation; code assistance,"Restructure the existing repository
while preserving its methodology documents","Initial monorepo scaffold; frontend
and backend health applications; CI; documentation; evidence templates","Review,
test, and adapt all generated files before merge; verification is pending at
generation time","Gitea issue and Pull Request to be added by the team"
```

```
2026-08-05,Shayna Unterslak,ChatGPT Web,GPT-5.6 Thinking,Planning;
documentation; review; Git and Gitea guidance,"Single project session used from
2026-08-04 to 2026-08-05 to develop the Git Methodology, Project Methodology,
Gitea workflow, Issue #7 governance correction, ADR-001, issue templates, Pull
Request template and related repository setup guidance","Git Methodology; Project
Methodology; Issue #7 description; ADR-001; issue templates; Pull Request
template; AI register wording; repository workflow instructions","Reviewed,
adapted and verified each item before use; no application code was generated or
used from this session","Unedited ChatGPT transcript retained for the project;
```



### Thinking

- **Updating documentation**

Yes—I'm updating three docs before pushing: `setup.md` to require `npm ci`, `dependencies.md` to explain lockfile enforcement, and `testing.md` to document local CI parity. I'm also adding this dependency-management decision to the AI register.

- **Updated documentation**

- **Applying doc edits**

I'm replacing `npm install` with `npm ci`, clarifying reproducible installs, adding the TypeScript compatibility note, documenting clean-checkout verification, and appending the CI-debugging session to the AI register.

- **Applied document edits**

✓ Worked for 37s

Yes. The documentation still describes the old state, so update it before pushing.

## 1. Update docs/development/setup.md

Change:

```
npm install
```

to:

```
npm ci
```

Replace this outdated paragraph:

The first `npm install` should create and commit `package-lock.json`. Once it exists, CI should be changed from `npm install` to `npm ci` for reproducible installs.

with:

The committed `package-lock.json` defines the exact dependency versions. Use `npm ci` for clean, reproducible installations locally and in CI.

The `npm run check` command runs the same structure, formatting, linting, type-checking, testing, and build checks used by the CI quality job.

## 2. Update docs/development/dependencies.md

Replace the final sentence:

Exact installed versions will be recorded by `package-lock.json` after the first verified installation.

with:

Exact installed versions are recorded in the committed `package-lock.json`. Use `npm ci` rather than `npm install` when validating a clean checkout or running CI.

Then add:

### ## TypeScript compatibility

TypeScript is pinned to version `5.5.4` because the current `@typescript-eslint` version supports TypeScript versions below `5.6.0`. TypeScript and `@typescript-eslint` should be reviewed and upgraded together.

## 3. Update docs/development/testing.md

Add this immediately before `## CI expectation`:

### ## Local CI verification

Before opening or updating a Pull Request, validate the repository from a reproducible installation:

```
```bash
npm ci
npm run check
```

### ## 4. Add the Issue #10 AI register entry

Append this as one new line to `evidence/ai/ai-usage-register.csv`:

```
```csv
2026-08-05,Shayna Unterslak,ChatGPT Web,GPT-5.6 Thinking,CI debugging; code and configuration assistance;
documentation,"Issue #10: restore green CI and reproducible installation; diagnose formatting, lint, TypeScript
compatibility, frontend build and workflow failures","Package lockfile; npm ci workflow; ESLint unused-argument rule;
TypeScript 5.5.4 pin; optional AbortSignal fix; development documentation","Reviewed each change; successfully ran
npm ci and the complete npm run check pipeline; separated changes into atomic commits","Issue #10; linked Pull
Request; same unedited ChatGPT transcript retained for the project"
```

## 5. Format and verify

```
npx.cmd prettier --write docs/development/setup.md docs/development/dependencies.md docs/development/testing.md
```

```
npm.cmd run check
```

```
git diff --check
```

## 6. Commit the documentation

```
git add docs/development/setup.md docs/development/dependencies.md docs/development/testing.md evidence/ai/ai-usage-
register.csv
```

```
git commit -m "docs: record reproducible CI workflow" -m "Refs #10" -m "Assisted-by: ChatGPT-Web[GPT-5.6 Thinking]"
```

Then confirm:

```
git status
```

It should report a clean working tree.

8/5/2026, 2:41:34 PM



Pasted markdown(1).md

correct?? also, this ai chat is the same one i used for the previous stuff, so what should i add to that?

8/5/2026, 2:45:16 PM



## Thinking

- **Reviewing the AI entry**

I'm checking the existing row against the repository evidence. It is correct: the setup steps match the lockfile, the TypeScript note matches the warning, and the Prettier fix is documented. Add only the observed results, not more speculation.

- **Reviewed the AI entry**

- **Updating the audit entry**

I'm noticing the existing row is now incomplete: this session generated application and configuration changes. I'd update that row rather than add a duplicate, include the later issues and commits, and verify the README has the required AI-use declaration.

- **Updated the audit entry**

- **Validated setup documentation and formatted the full project with Prettier**

✓ Worked for 1m 3s

Yes, the documentation changes are correct. The setup now uses the committed lockfile, the dependency document explains the TypeScript pin, and the testing document records the local CI procedure. `Pasted markdown.md`

There is only one small formatting correction in `docs/development/testing.md`: add a blank line between the closing code fence and `## CI expectation`:

```
npm ci
npm run check
```

## CI expectation

Prettier should correct this automatically.

*## AI register: same chat, separate activities*

Because this is the **same continuous transcript**, you do not need a separate transcript. However, I recommend using **two register rows** because Issue #7 and Issue #10 were distinct activities. Both rows can point to the same unedited transcript.

Your existing row currently says:

> no application code was generated or used from this session

That is no longer accurate because this same chat assisted with the frontend TypeScript fix, ESLint configuration and CI workflow.

### Replace the existing Shayna row with this

```
```csv
```



```
2026-08-05,Shayna Unterslak,ChatGPT Web,GPT-5.6 Thinking,Planning; documentation; review; Git and Gitea guidance,"Issue #7 governance and methodology portion of a continuous project session used from 2026-08-04 to 2026-08-05","Git Methodology; Project Methodology; Issue #7 description; ADR-001; issue templates; Pull Request template; AI register wording; repository workflow instructions","Reviewed, adapted and verified each item before use; no application code was generated or used for the Issue #7 governance portion","Unedited ChatGPT transcript retained for the project; Issue #7 and linked Pull Request"
```

## Add this new row directly underneath it

```
2026-08-05,Shayna Unterslak,ChatGPT Web,GPT-5.6 Thinking,CI debugging; code and configuration assistance; documentation,"Issue #10: restore green CI and reproducible installation; diagnose dependency, formatting, lint, TypeScript compatibility and frontend build failures","Package lockfile; npm ci workflow; ESLint unused-argument rule; TypeScript 5.5.4 pin; optional AbortSignal fix; development documentation","Reviewed each proposed change; successfully ran npm ci and the complete npm run check pipeline; organised the work into atomic commits","Issue #10 and linked Pull Request; same unedited ChatGPT transcript retained for the project"
```

That is more transparent than stretching one row across unrelated issues.

## README declaration

Because this session now included code and configuration assistance, the AI policy also requires the repository-level README declaration to reflect AI code generation/review, in addition to the `Assisted-by` commit footers. `ai_policy_20_jul.pdf`

First check what is already present:

```
Select-String -Path README.md -Pattern "AI|artificial intelligence|code generation|code review" -Context 2,5
```

Do not duplicate an existing section. If none exists, add:

```
## AI usage
```

```
This repository makes use of AI code generation using the following tools: ChatGPT-Web[GPT-5.6 Thinking].
```

```
This repository does not use AI in-line editing tools.
```

```
This repository makes use of AI code review using the following tools: ChatGPT-Web[GPT-5.6 Thinking].
```

Then format:

```
npx.cmd prettier --write docs/development/setup.md docs/development/dependencies.md docs/development/testing.md README.md
```

8/5/2026, 2:45:18 PM



## ## AI usage

This repository makes use of AI code generation using the following tools: ChatGPT-Web[GPT-5.6 Thinking].

This repository does not currently use AI in-line editing tools.

This repository does not currently use AI code review.

See [``evidence/ai/ai-usage-register.csv``](evidence/ai/ai-usage-register.csv) for task-level records.

TypeScript is pinned to version ``5.5.4`` because the current ``@typescript-eslint`` version supports TypeScript versions below ``5.6.0``. TypeScript and ``@typescript-eslint`` should be reviewed and upgraded together.

PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> Select-String -Path README.md -Pattern "AI|artificial intelligence|code generation|code review" -Context 2,5

README.md:13:database	Migrations, seeds, and schema documentation
README.md:14:docs	Source for the public MkDocs documentation site
> README.md:15:evidence	Stakeholder, sprint, testing, decision, and AI
evidence	
README.md:16:infra	Deployment and infrastructure documentation
README.md:17:scripts	Repository validation scripts
README.md:18:tests	Cross-application and non-unit testing assets
README.md:19:``	
README.md:20:	
README.md:21:	The frontend and backend are separate applications. The frontend must access application data through the backend HTTP API. The database must not be exposed to the frontend through generated Supabase or Firebase data endpoints.
README.md:22:	
> README.md:23:	See [Repository Structure](docs/architecture/repository-structure.md) for the detailed tree and boundaries.
README.md:24:	
README.md:25:## Prerequisites	
README.md:26:	
README.md:27:- Node.js 20 or later	
README.md:28:- npm 10 or later	
README.md:71:	
README.md:72:- Create a Gitea issue before significant work begins.	
> README.md:73:- Create a short-lived branch from the latest <code>`main`</code> .	
> README.md:74:- Use Pull Requests and peer review; do not develop directly on <code>`main`</code> .	
README.md:75:- Keep API, database, security, testing, and deployment documentation aligned with the implementation.	
README.md:76:- Add or update tests for behavioural changes.	
README.md:77:- Record stakeholder decisions and feedback under <code>`evidence/`</code> .	
> README.md:78:- Attribute AI-assisted code in commit messages and maintain the AI register.	
README.md:79:	
> README.md:80:## AI usage	
README.md:81:	
> README.md:82:	This repository makes use of AI code generation using the following tools: ChatGPT-Web[GPT-5.6 Thinking].
README.md:83:	

```
> README.md:84:This repository does not currently use AI in-line editing tools.
  README.md:85:
> README.md:86:This repository does not currently use AI code review.
  README.md:87:
> README.md:88:See [evidence/ai/ai-usage-register.csv](evidence/ai/ai-usage-register.csv) for task-level records.
```

```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool>
```

8/5/2026, 2:49:24 PM



Yes — the **README AI section is correct as written**:

### ## AI usage

This repository makes use of AI code generation using the following tools: ChatGPT-Web[GPT-5.6 Thinking].

This repository does not currently use AI in-line editing tools.

This repository does not currently use AI code review.

See [`evidence/ai/ai-usage-register.csv`]([evidence/ai/ai-usage-register.csv](#)) for task-level records.

The debugging and suggested code/configuration changes in this chat count as **AI code generation/assistance**, not necessarily formal AI code review. Keep “does not currently use AI code review” unless you specifically use AI to review a completed PR or code change as a reviewer.

The TypeScript paragraph belongs in `docs/development/dependencies.md`, not in the README AI section.

## Same chat transcript

Use the **same transcript as evidence**, but keep two CSV rows because the work relates to two different issues.

Revise the existing Issue #7 row so it refers specifically to that portion of the chat:

```
2026-08-05,Shayna Unterslak,ChatGPT Web,GPT-5.6 Thinking,Planning; documentation; review; Git and Gitea
guidance,"Issue #7 governance and methodology portion of a continuous project session used from 2026-08-04 to 2026-
08-05","Git Methodology; Project Methodology; Issue #7 description; ADR-001; issue templates; Pull Request template;
AI register wording; repository workflow instructions","Reviewed, adapted and verified each item before use; no
application code was generated or used for the Issue #7 governance portion","Unedited ChatGPT transcript retained for
the project; Issue #7 and linked Pull Request"
```

Then add this Issue #10 row beneath it:

```
2026-08-05,Shayna Unterslak,ChatGPT Web,GPT-5.6 Thinking,CI debugging; code and configuration assistance;
documentation,"Issue #10: restore green CI and reproducible installation; diagnose dependency, formatting, lint,
TypeScript compatibility and frontend build failures","Package lockfile; npm ci workflow; ESLint unused-argument
rule; TypeScript 5.5.4 pin; optional AbortSignal fix; development documentation","Reviewed each proposed change;
successfully ran npm ci and the complete npm run check pipeline; organised the work into atomic commits","Issue #10
and linked Pull Request; same unedited ChatGPT transcript retained for the project"
```

Also ensure there is a blank line before `## CI expectation` in `docs/development/testing.md`.

8/5/2026, 2:49:47 PM



Date,Team member,Tool,Model,Purpose,Brief task,Output used,Verification or adaptation,Related evidence

2026-08-04,Dean Feldman,ChatGPT Web,GPT-5.6 Thinking,Planning; technical guidance; documentation; code assistance,"Restructure the existing repository while preserving its methodology documents","Initial monorepo scaffold; frontend and backend health applications; CI; documentation; evidence templates","Review, test, and adapt all generated files before merge; verification is pending at generation time","Gitea issue and Pull Request to be added by the team"

2026-08-05,Shayna Unterslak,ChatGPT Web,GPT-5.6 Thinking,Planning; documentation; review; Git and Gitea guidance,"Issue #7 governance and methodology portion of a continuous project session used from 2026-08-04 to 2026-08-05","Git Methodology; Project Methodology; Issue #7 description; ADR-001; issue templates; Pull Request template; AI register wording; repository workflow instructions","Reviewed, adapted and verified each item before use; no application code was generated or used for the Issue #7 governance portion","Unedited ChatGPT transcript retained for the project; Issue #7 and linked Pull Request"

2026-08-05,Shayna Unterslak,ChatGPT Web,GPT-5.6 Thinking,CI debugging; code and configuration assistance; documentation,"Issue #10: restore green CI and reproducible installation; diagnose dependency, formatting, lint, TypeScript compatibility and frontend build failures","Package lockfile; npm ci workflow; ESLint unused-argument rule; TypeScript 5.5.4 pin; optional AbortSignal fix; development documentation","Reviewed each proposed change; successfully ran npm ci and the complete npm run check pipeline; organised the work into atomic commits","Issue #10 and linked Pull Request; same unedited ChatGPT transcript retained for the project"

good?

8/5/2026, 2:50:18 PM



## Thinking

- **Checking CSV quality**

I'm seeing a solid register entry. I'd remove the blank line between records and slightly reword the output as "Guidance resulting in a committed package-lock.json" for maximum accuracy.

- **Checked CSV quality**

✓ Worked for 14s

Yes, that is good. One small correction: **remove the blank line between Dean's row and your first row** so the CSV does not contain an empty record.

I would also slightly refine "Package lockfile" because npm generated the file; the AI guided the process. Use:

2026-08-05,Shayna Unterslak,ChatGPT Web,GPT-5.6 Thinking,CI debugging; code and configuration assistance; documentation,"Issue #10: restore green CI and reproducible installation; diagnose dependency, formatting, lint, TypeScript compatibility and frontend build failures","Reproducible dependency setup; npm ci workflow; ESLint unused-argument rule; TypeScript 5.5.4 pin; optional AbortSignal fix; development documentation","Reviewed each proposed change; successfully ran npm ci and the complete npm run check pipeline; organised the work into atomic commits","Issue #10 and linked Pull Request; same unedited ChatGPT transcript retained for the project"

Everything else is accurate and transparent.

8/5/2026, 2:50:33 PM





k, lets commit and push? are we ready?

8/5/2026, 2:51:12 PM



### Thinking

- **Finalizing changes**

Yes, I'm ready to commit and push, provided the working tree contains only the intended documentation and AI-register edits. I'd run the final formatter/check, inspect the staged diff, commit, then push the branch and open the PR.

- **Finalized changes**

✓ Worked for 13s

Yes — we are ready after one final verification and documentation commit.

## 1. Format the documentation

```
npx.cmd prettier --write docs/development/setup.md docs/development/dependencies.md docs/development/testing.md
```

## 2. Run the final checks

```
npm.cmd run check
```

Then:

```
git diff --check
```

Warnings about CRLF being replaced by LF are acceptable. There should be no actual whitespace errors.

## 3. Review the remaining changes

```
git status --short
```

The remaining changes should be limited to:

```
docs/development/setup.md
docs/development/dependencies.md
docs/development/testing.md
evidence/ai/ai-usage-register.csv
```

Review them:

```
git diff -- docs/development/setup.md docs/development/dependencies.md docs/development/testing.md evidence/ai/ai-usage-register.csv
```

## 4. Commit the documentation

```
git add docs/development/setup.md docs/development/dependencies.md docs/development/testing.md evidence/ai/ai-usage-register.csv
```

Check the staged files:

```
git diff --cached --stat
```

Commit:

```
git commit -m "docs: record reproducible CI workflow" -m "Refs #10" -m "Assisted-by: ChatGPT-Web[GPT-5.6 Thinking]"
```

## 5. Confirm everything is clean

```
git status
```

Expected:

```
nothing to commit, working tree clean
```

You can also review the commits:

```
git log --oneline -6
```

## 6. Push the branch

```
git push -u origin ci/10-restore-green-ci
```

After the push completes, send me the output. We will then open the Pull Request, inspect the online Actions run, and diagnose whether a Gitea runner is available.

*8/5/2026, 2:51:24 PM*



```
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool> git push -u origin ci/10-restore-green-ci
Enumerating objects: 89, done.
Counting objects: 100% (89/89), done.
Delta compression using up to 16 threads
Compressing objects: 100% (52/52), done.
Writing objects: 100% (55/55), 61.30 KiB | 4.38 MiB/s, done.
Total 55 (delta 37), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a new pull request for 'ci/10-restore-green-ci':
remote:  https://sdp.ms.wits.ac.za/git-push-pray/Sport-Analytics-Tool/pulls/new/ci/10-restore-green-ci
remote:
remote: . Processing 1 references
remote: Processed 1 references in total
To https://sdp.ms.wits.ac.za/git-push-pray/Sport-Analytics-Tool.git
* [new branch]    ci/10-restore-green-ci -> ci/10-restore-green-ci
branch 'ci/10-restore-green-ci' set up to track 'origin/ci/10-restore-green-ci'.
PS C:\Users\shayn\OneDrive\Desktop\Sport-Analytics-Tool>
```

8/5/2026, 2:53:14 PM



Great — the branch pushed successfully and is tracking the remote branch.

Open the Pull Request link shown in the output and use:

## PR title

ci: restore green CI and reproducible installation

## PR description

### ## Summary

- add a committed `package-lock.json` for reproducible dependency installation
- update CI to use `npm ci`
- pin TypeScript to `5.5.4` for compatibility with the current `@typescript-eslint` version
- allow intentionally unused underscore-prefixed function parameters
- fix the optional `AbortSignal` frontend production-build error
- align existing files with the repository's Prettier configuration
- update development and AI usage documentation

### ## Verification

- [x] `npm ci`
- [x] `npm run check`
- [x] repository structure check passed
- [x] formatting passed
- [x] linting passed
- [x] type checking passed
- [x] backend and frontend tests passed
- [x] all production builds passed

### ## Notes

The dependency installation currently reports five audit findings. These were not automatically changed because `npm audit fix --force` may introduce breaking dependency upgrades. They should be reviewed in a separate issue.

Closes #10

After creating the PR:

1. Assign a teammate as reviewer.
2. Move Issue #10 from **In Progress** to **In Review**.
3. Open the PR's checks or Actions section.

Do not merge yet. Send me a screenshot of the PR and the workflow status, especially if it remains yellow or says it is waiting for a runner.

*8/5/2026, 2:53:29 PM*



caa4dfa3-2e41-4bb9-9b6a-90...

8/5/2026, 2:55:14 PM



This strongly indicates a **runner problem**, not a workflow-code problem:

- the new run is **Waiting**
- duration is still **0s**
- the previous runs also remained at **0s**
- your local `npm ci` and `npm run check` both passed

So Gitea has accepted the workflow, but no compatible runner has started the `ubuntu-latest` job.

## Next check

Click the newest run:

docs: record reproducible CI workflow

Then open the `quality` job. Look for a message such as:

Waiting for a runner to pick up this job

or wording mentioning the required label:

ubuntu-latest

After that, go to:

Settings → Actions → Runners

Check whether any runner is listed and whether its labels include `ubuntu-latest`.

Do not change `runs-on` yet. Send a screenshot of the run-detail page or the Runners page, and we will determine whether the runner is missing entirely or only has a label mismatch.

8/5/2026, 2:55:29 PM



5b3426b6-f826-439c-ab4a-15...

8/5/2026, 2:56:36 PM



## Thinking

- **Assessing runner availability**

I'm reading the screenshot as showing no runner currently registered for the repository, so CI jobs are waiting. I'd create a runner only on an approved machine; otherwise, the Wits server administrator needs to provide one.

- **Assessed runner availability**



- **Searching the web**

[docs.gitea.com](https://docs.gitea.com) [merriam-webster.com](https://www.merriam-webster.com) [synonymer.se](https://synonymer.se) [dictionary.cambridge.org](https://dictionary.cambridge.org) [wise.se](https://wise.se) [youtube.com](https://www.youtube.com) [komvuxutbildningar.se](https://komvuxutbildningar.se)  
[grammaticus.blog](https://grammaticus.blog) [collinsdictionary.com](https://www.collinsdictionary.com)

- **Confirming the blocker**

The official documentation confirms that jobs need an available runner. Since the repository shows none, I'd leave the pull request open, comment that CI is blocked, and ask the server administrator to provision an approved shared runner rather than registering a personal machine.

- **Confirmed the blocker**

- **Searched 9 websites**

✓ Worked for 40s

That confirms the blocker: **there is no runner available to execute the workflow**. Gitea accepts the workflow, but jobs only run when a separate Gitea Actions runner picks them up. A registration token alone does not run anything; a runner program must be installed and connected on a suitable machine. ([Gitea Documentation](#))

Do **not** create a runner on your personal laptop yet. A runner executes repository workflow code, so it should be provisioned on an approved, controlled machine. Gitea's own documentation warns that independently deployed runners have security implications. ([Gitea Documentation](#))

## What to do now

Keep the Pull Request open and do not merge it yet.

Move Issue #10 to **Blocked**, and add this comment:

Local CI restoration is complete. A clean `npm ci` installation followed by the full `npm run check` pipeline passes, including formatting, linting, type checking, tests and production builds.

The Gitea Actions workflow is currently blocked by infrastructure: repository Settings → Actions → Runners shows 0 available runners, and all workflow runs remain waiting at 0 seconds.

The Pull Request will remain open until an approved organisation or instance runner is provisioned and the workflow completes successfully.

Add a similar note to the Pull Request:

CI infrastructure blocker: the repository currently has no available Gitea Actions runner. Local verification passes with `npm ci` and `npm run check`. Awaiting confirmation from the SDP/Gitea administrator about whether a shared runner should be available or whether teams are expected to provision one.

## Message to the SDP administrator or lecturer

Hi,

Our Gitea Actions workflow is being accepted but remains in "Waiting" at 0 seconds. Under Repository Settings → Actions → Runners, the repository shows "Total: 0" and "No runners available".

The workflow requires `ubuntu-latest`. Locally, a clean `npm ci` followed by `npm run check` passes successfully.

Could you please confirm whether an organisation or instance runner should be available for student repositories, or whether each team is expected to provision its own repository runner?

Thank you.

Do not share any runner registration token publicly or paste it into the issue or Pull Request.

### Sources

[Act Runner](https://docs.gitea.com) docs.gitea.com

8/5/2026, 2:56:54 PM



the admin is the lecturer, so write it d=formaly enough for me

8/5/2026, 2:59:34 PM



**Subject: Gitea Actions Runner Required for CI Workflow**

Dear [Lecturer's Name],

I hope you are well.

Our team has completed the work for Issue #10, which focused on restoring the repository's CI process and ensuring reproducible dependency installation.

The workflow is being recognised by Gitea, but all Actions runs remain in a "Waiting" state at 0 seconds. Under **Repository Settings → Actions → Runners**, the repository shows that there are currently no runners available.

The workflow is configured to use `ubuntu-latest`. We have verified the process locally using a clean `npm ci` installation followed by `npm run check`, and all formatting, linting, type checking, tests and production builds pass successfully.

Please could you advise whether a shared organisation or instance runner should be available for student repositories, or whether each team is expected to provision its own repository runner?

Thank you for your assistance.

Kind regards

Shayna Underslak

Git Push Pray

8/5/2026, 2:59:46 PM



dont say issue #10 bc he doesnt know what that is

8/5/2026, 3:00:34 PM



**Subject: Gitea Actions Runner Required for CI Workflow**

Dear [Lecturer's Name],

I hope you are well.

Our team has configured the repository's CI workflow and verified the full process locally. A clean `npm ci` installation followed by `npm run check` completes successfully, including formatting, linting, type checking, testing and production builds.

However, the workflow remains in a "Waiting" state at 0 seconds on Gitea. Under **Repository Settings → Actions → Runners**, the repository shows that there are currently no runners available.

The workflow is configured to use `ubuntu-latest`. Please could you advise whether a shared organisation or instance runner should be available for student repositories, or whether each team is expected to provision its own repository runner?

Thank you for your assistance.

Kind regards

Shayna Unterslak

Git Push Pray

8/5/2026, 3:00:39 PM



can i make it that we merge this commit, but dont close the issue?

8/5/2026, 3:01:59 PM



### Thinking



#### Searching the web

[docs.gitea.com](https://docs.gitea.com)

#### • Searched 1 website

✓ Worked for 7s

Yes. Edit the PR description before merging and change:

Closes #10

to:

Refs #10

Closes #10 is an actionable reference, so Gitea will close the issue when the PR merges. A normal reference such as Refs #10 links the PR to the issue without closing it. ([Gitea Documentation](https://docs.gitea.com))

Then you can:

1. Merge the PR after the required teammate approval.
2. Leave Issue #10 open.
3. Move the issue to **Blocked**.
4. Add a comment explaining that the repository changes are merged, but completion is blocked until a Gitea Actions runner is provided and the online workflow passes.

Use this issue comment:

The repository changes have been reviewed, locally verified and merged into `main`.

A clean `npm ci` installation followed by the full `npm run check` pipeline passes locally, including formatting, linting, type checking, tests and production builds.

This issue remains open and blocked because the Gitea Actions workflow cannot currently execute. Repository Settings → Actions → Runners shows no available runners, so workflow runs remain waiting at 0 seconds.

The issue can be closed once an approved runner is available and the workflow completes successfully on Gitea.

### Sources

[Automatically Linked References | Gitea Documentation](https://docs.gitea.com) docs.gitea.com

8/5/2026, 3:02:01 PM



**\*\*Subject: Gitea Actions Runner Required for CI Workflow\*\***

Dear [Lecturer's Name],

I hope you are well.

Our team has configured the repository's CI workflow and verified the full process locally. A clean `npm ci` installation followed by `npm run check` completes successfully, including formatting, linting, type checking, testing and production builds.

However, the workflow remains in a "Waiting" state at 0 seconds on Gitea. Under **\*\*Repository Settings → Actions → Runners\*\***, the repository shows that there are currently no runners available.

The workflow is configured to use `ubuntu-latest`. Please could you advise whether a shared organisation or instance runner should be available for student repositories, or whether each team is expected to provision its own repository runner?

Thank you for your assistance.

Kind regards  
Shayna Unterslak  
Git Push Pray  
maker much ;ess formal and less AI

8/5/2026, 3:02:52 PM



**Subject: Gitea Actions Runner**

Hi [Lecturer's Name],

We have set up our CI workflow and tested it locally. Everything passes when we run `npm ci` followed by `npm run check`.

On Gitea, however, the workflow stays on "Waiting" at 0 seconds. In the repository settings, it also shows that there are no runners available.

Could you please let us know whether there should be a shared runner available for student repositories, or whether each team needs to set up its own runner?

Thanks so much.

Kind regards  
Shayna Unterslak  
Git Push Pray

8/5/2026, 3:03:00 PM



no, ask as if he hasnt seen our repo at all,

8/5/2026, 3:03:45 PM



**Subject: Gitea Actions Runner**

Hi [Lecturer's Name],

We are setting up automated checks for our project repository on Gitea so that our code is tested whenever we push changes or open a pull request.

We have tested the checks on our own computers and everything passes, but on Gitea the workflow stays on "Waiting" at 0 seconds. In the repository settings, it also says that there are no runners available.

Could you please let us know whether there is meant to be a shared runner for student repositories, or whether each group needs to set up its own?

Thanks so much.

Kind regards

Shayna Underslak

Git Push Pray

8/5/2026, 3:03:51 PM



the person doing azure from the 6 issues from b4 wants to do frontend at the same time, make me a frontend azure issue then (if that makes sense to do)

8/5/2026, 3:07:39 PM



Yes, it makes sense to create a **separate frontend Azure issue**. The same person may work on both, but keeping frontend and backend deployment separate gives you clearer scope, evidence, reviews and rollback.

## Issue title

Establish Azure frontend deployment foundation

## Issue type

Task

## Description

### ## Objective

Establish a basic Azure deployment foundation for the Vite frontend so that the application can be built and accessed through a public HTTPS URL.

This issue covers frontend hosting only. Backend deployment remains part of the separate Azure backend deployment issue.

### ## Scope

- evaluate and select an appropriate Azure hosting option for the Vite frontend
- configure the frontend production build for deployment
- deploy the frontend to Azure
- configure the deployed frontend to communicate with the deployed backend through `VITE\_API\_BASE\_URL`
- ensure no secrets or backend-only credentials are exposed in frontend configuration
- document the deployment process and required environment variables
- record evidence of the successful deployment

### ## Acceptance criteria

- [ ] The selected Azure frontend hosting approach is documented and briefly motivated.
- [ ] The frontend builds successfully using the committed dependency lockfile.
- [ ] A production build is created using the repository's existing frontend build command.
- [ ] The frontend is deployed to a publicly accessible HTTPS URL.
- [ ] The deployed frontend loads successfully without console-breaking errors.
- [ ] `VITE\_API\_BASE\_URL` is configured for the deployed environment.
- [ ] No database credentials, API secrets or backend-only variables are included in the frontend.
- [ ] Single-page application routing is handled correctly where applicable.
- [ ] Deployment and configuration instructions are added to the repository documentation.
- [ ] The deployed URL and supporting evidence are recorded in the issue or Pull Request.
- [ ] `npm run check` passes before the Pull Request is opened.

### ## Out of scope

- backend hosting and backend environment configuration
- database provisioning
- production authentication configuration

- advanced monitoring, scaling or custom-domain configuration
- exposing the database directly to the frontend

### ## Dependencies

- Link this issue to the Azure backend deployment issue.
- Final API integration depends on the deployed backend URL being available.
- Automated deployment may depend on the availability of a Gitea Actions runner.

### ## Evidence required

- public frontend URL
- screenshot showing the deployed application
- screenshot or log showing a successful production build
- documentation describing the Azure service selected and deployment steps

## Suggested relationship to the backend issue

Add this comment to both issues:

The frontend and backend Azure deployment tasks are related but will remain separate for clearer scope, review and evidence. They may be assigned to the same team member and worked on in parallel.

They should still use separate branches and Pull Requests, for example:

```
infra/<frontend-issue-number>-azure-frontend-deployment
```

and

```
infra/<backend-issue-number>-azure-backend-deployment
```

8/5/2026, 3:08:02 PM



labels?

8/5/2026, 3:10:13 PM



Use:

- **Task**
- **Frontend**
- **Infrastructure**
- **Azure**
- **Deployment**
- **Sprint 1**

Priority: **Medium**, unless the backend deployment issue is already marked High—in that case, match it for consistency.

I would not label it **Feature**, because this issue establishes deployment infrastructure rather than adding user-facing functionality.

8/5/2026, 3:10:26 PM



noo, like type and area

8/5/2026, 3:10:38 PM



Use:

- **Type:** Task
- **Area:** Frontend

Because the issue is specifically for deploying the frontend. The Azure and deployment details belong in the title and description rather than needing separate labels.

8/5/2026, 3:10:48 PM



wait, do it properly, look back to the other one and do it like that!!

8/5/2026, 3:11:10 PM



### Thinking



#### Remembering

- Reviewed Azure issues, labels, and Sport Analytics template

✓ Worked for 18s

You're right — use the same label format as the other foundation issues:

- **Type:** type: infrastructure
- **Area:** area: frontend
- **Milestone:** Sprint 1

So this is infrastructure work specifically for the frontend.

8/5/2026, 3:11:12 PM



and the issue decr etc? do thag again

8/5/2026, 3:11:42 PM



### Thinking

- **Used personal\_context tool**
- ✓ Worked for 22s

## Issue title

Establish Azure frontend deployment foundation

## Labels

- **Type:** type: infrastructure
- **Area:** area: frontend
- **Priority:** priority: high

## Milestone

Sprint 1

## Assignee

Assign it to the team member handling the Azure deployment work.

### ## Description

Establish the initial Azure deployment foundation for the Sport Analytics Tool frontend.

The frontend should be deployed from the existing Vite application and made available through a public HTTPS URL. The deployment must use environment-specific configuration so that the frontend can later communicate with the deployed backend without exposing backend secrets or database credentials.

The selected Azure hosting service and deployment approach must be documented and motivated.

### ## Motivation

An early frontend deployment provides the team with a stable environment for stakeholder demonstrations, integration testing and user testing.

Keeping frontend deployment separate from backend deployment also provides clearer ownership, evidence and troubleshooting. Although the same team member may work on both Azure issues, each deployment should have its own branch, Pull Request and documentation.

### ## Acceptance Criteria

- [ ] An appropriate Azure hosting service for the Vite frontend has been selected.
- [ ] The selected service and the reason for choosing it have been documented.
- [ ] The frontend installs successfully using `npm ci`.
- [ ] The frontend production build completes successfully.
- [ ] The frontend is deployed to a publicly accessible HTTPS URL.

- [ ] The deployed application loads successfully without blocking browser-console errors.
- [ ] The deployed frontend displays the existing application interface correctly.
- [ ] `VITE\_API\_BASE\_URL` can be configured for the deployed environment.
- [ ] No backend secrets, database credentials or private API keys are exposed through frontend environment variables.
- [ ] Frontend deployment instructions are added to the repository documentation.
- [ ] The public deployment URL is recorded in the documentation or Pull Request.
- [ ] `npm run check` passes before the Pull Request is opened.
- [ ] The work is completed through a reviewed Pull Request linked to this issue.

### ## Dependencies

- The final frontend-to-backend connection depends on the Azure backend deployment issue providing a working backend URL.
- Automated deployment may depend on a Gitea Actions runner becoming available.
- This issue does not include database provisioning or backend hosting.

### ## Evidence

Link or attach:

- the Pull Request implementing the deployment;
- the public Azure frontend URL;
- a screenshot of the deployed frontend;
- evidence of a successful production build;
- the updated deployment documentation; and
- any Azure configuration evidence that does not expose secrets.

## Suggested branch

infra/<issue-number>-azure-frontend-deployment

The frontend and backend Azure issues can be assigned to the same person and worked on at the same time, but they should remain separate issues and Pull Requests.

8/5/2026, 3:11:44 PM



ok, now we couldn figure out how to 'aprove' merges so i changed therule to 0, is that an issue and what do i need to do to make that ok>?

8/5/2026, 3:13:54 PM



## Thinking

- **Clarifying review controls**

I'm distinguishing enforced settings from actual review evidence. Keeping approvals at zero can be acceptable temporarily if a teammate explicitly reviews and approves each PR, authors never self-merge, and the methodology records this exception.

- **Clarified review controls**

- **Evaluated Gitea approval requirements and branch protection compliance**



### Searching the web

[docs.gitea.com](https://docs.gitea.com) [dictionary.cambridge.org](https://dictionary.cambridge.org) [en.wikipedia.org](https://en.wikipedia.org) [github.com](https://github.com) [github.nih.gov](https://github.nih.gov) [merriam-webster.com](https://merriam-webster.com) [medium.com](https://medium.com)  
[collinsdictionary.com](https://collinsdictionary.com) [w3c.hexaschool.com](https://w3c.hexaschool.com)

- **Resolving the mismatch**

I'm treating zero required approvals as a temporary enforcement gap, not a replacement for peer review. I'd restore one approval when possible; meanwhile, require a teammate's explicit approval comment, prohibit self-merging, record the workaround, and update the methodology only if this becomes permanent.

- **Resolved the mismatch**

- **Searched 9 websites**

✓ Worked for 1m 0s

It is **not ideal to leave the required approvals at 0 permanently**, because your Git Methodology says that the PR author may not merge their own PR without approval from another team member. It also lists peer-review comments and approvals as evidence that the methodology was followed. `Git_Methodology.md` `Git_Methodology.md`

However, you can make 0 acceptable **temporarily** if you still require and record a genuine peer review manually. The platform setting would no longer enforce the rule, but your team process still would.

## First try to make formal approval work

The reviewer must:

1. Use their **own Gitea account**.
2. Have **Write access** to the repository.
3. Open the Pull Request.
4. Open the changed-files or review section.
5. Submit a review and choose **Approve**.

Gitea protected branches count approvals from users with write access unless approval restrictions have been configured differently. ([Gitea Documentation](#)) Gitea supports reviews that approve changes, request changes or leave comments. ([Gitea Documentation](#))

The PR author cannot count as their own peer reviewer.

## Temporary process while the rule is set to 0

Before every merge, another teammate must review the PR and leave this comment from their own account:



Peer review completed by @reviewer.

I reviewed the changed files, the linked issue, the testing evidence and the documentation updates. I approve this Pull Request for merge.

Then:

- the PR author may merge only after that comment;
- use a merge commit, as required by your methodology;
- do not delete or edit the approval comment;
- ensure the PR clearly links to its issue;
- keep a screenshot only if needed for assessment evidence.

For your current PR, the reviewer should also acknowledge the CI limitation:

Peer review completed by @reviewer.

I reviewed the changes and the local verification evidence. `npm ci` and the complete `npm run check` pipeline passed locally.

I approve this Pull Request for merge. I understand that the online workflow is still waiting because no Gitea Actions runner is currently available.

## Record why the setting is 0

Add this comment to the branch-protection issue or governance record:

The required approval count has temporarily been set to 0 because the team has not yet been able to use Gitea's formal approval function successfully.

This does not remove the team's peer-review requirement. Before any Pull Request is merged, another team member must review the changes from their own account and leave an explicit approval comment on the Pull Request.

The branch-protection approval count will be restored to 1 once the formal Gitea approval process has been confirmed.

You do **not** need to rewrite the Git Methodology, because you are still requiring approval from another team member. You are only using an explicit review comment as temporary evidence instead of Gitea's automated approval counter. The course material also emphasises that someone else should review a PR before merging. [lecture\\_two.pdf](#)

### Sources

[Protected branches](#) docs.gitea.com

8/5/2026, 3:14:34 PM



96454461-bc87-40d2-b3b6-c6...

8/5/2026, 3:20:02 PM



That's helpful. It confirms the runner problem is known and the lecturer is arranging one, but it may take a week or two.

You can now:

- merge the PR after another teammate reviews it and leaves an approval comment;
- keep Issue #10 open;
- move it to **Blocked**;
- note that the code is complete, but online CI is waiting for the runner.

Use this issue comment:

The CI configuration and repository fixes have been completed and verified locally. A clean `npm ci` installation followed by `npm run check` passes successfully.

The Pull Request has been merged, but this issue will remain open and blocked until the Gitea Actions runner is available and the workflow completes successfully online.

The lecturer has confirmed that runners are being arranged, although this may take one to two weeks.

You could reply to the lecturer:

Hi Brendan,

Thank you for letting me know. We will continue with the local checks for now and keep the CI task open until the runner is available.

Kind regards  
Shayna

8/5/2026, 3:20:15 PM



ok, we have our first standup today, what do we need to do/say?

8/6/2026, 9:03:56 AM



### Thinking

- **Preparing standup guidance**

I'm treating this as a practical first standup: align on yesterday, today, blockers, and ownership. I'll tailor prompts to the CI, security, Azure, and UI issues, while keeping discussion concise and actionable.

- **Prepared standup guidance**

- **Prepared a lightweight Scrumban standup update and agenda**

✓ Worked for 20s

For your **first Thursday standup**, keep it to about **15 minutes**. The purpose is coordination, not solving technical problems during the meeting. Each person answers four questions: what they completed, what they are working on, what they will do next, and what is blocking them. Then quickly review the Gitea board and agree on next actions. `Pasted markdown.md`

### Before the meeting

Open the Gitea Project board and ask everyone to make sure:

- their issue is assigned to them;
- it has the correct Sprint 1 milestone and labels;
- its board status is accurate;
- any blocker is written as a comment;
- each person normally has only one main issue in progress.

Choose one person to facilitate. For the first meeting, you can do it, and then rotate weekly.

### What you can say to open

*Thanks, everyone. This is our first weekly standup, so we will keep it brief and follow the process in our Project Methodology.*

*Each person should tell us:*

- 1. what you have completed so far;*
- 2. what you are currently working on;*
- 3. what you will complete next;*
- 4. whether anything is blocking you.*

*We will then review the board, confirm priorities and note any actions. Any detailed technical discussion can happen after the standup with the relevant people.*

### Suggested speaking order

Go around the group one person at a time. Each update should take about one to two minutes.

They can use this format:

**Completed:**

*I completed...*

**Current:**

*I am currently working on...*

**Next:**

*Before the next standup, I plan to...*

**Blockers:**

*I am blocked by... / I currently have no blockers.*

## What you can say for your update

Adjust this depending on whether the PR has already been merged:

**Completed:** *I set up and documented our Git and Project Methodologies, aligned the repository governance, restored the local CI checks and created the reproducible dependency installation. The full `npm ci` and `npm run check` process now passes locally.*

**Current:** *The CI changes are in review or have been merged, but the issue remains open because Gitea currently has no Actions runner.*

**Next:** *I will keep the CI issue documented as blocked, follow up when the runner becomes available and help ensure the remaining Sprint 1 issues follow the correct issue, branch and PR process.*

**Blocker:** *The online CI workflow cannot run until the lecturer provides a Gitea Actions runner. This may take one to two weeks.*

## Questions to ask each person

For each of the Sprint 1 foundation issues, confirm:

- Is the issue still achievable within Sprint 1?
- Is the acceptance criteria clear?
- Has the person created the correct branch?
- Is there anything they need from another team member?
- Does any issue depend on the backend, database or stakeholder?
- Is their issue in the correct board column?

For the Azure person specifically:

*Are the frontend and backend Azure issues being kept as separate branches and Pull Requests?  
Is either deployment blocked by credentials, Azure access or the unavailable Gitea runner?*

## Review the board together

Move issues during the meeting where needed:

- **Ready** - fully defined but not started
- **In Progress** - actively being worked on
- **In Review** - Pull Request open and awaiting review
- **Blocked** - work cannot continue
- **Done** - acceptance criteria met, reviewed, merged and closed

Your CI issue should remain **Blocked** until the online workflow runs successfully, even if the code has already been merged.

## End the meeting with a clear recap

Say:

*Before we finish, I am going to confirm the actions for the week.*

*Each person has one main issue in progress. Please keep your issue and board status updated during the week, not only at the next standup.*

*The key blocker is the missing Gitea Actions runner. Any technical discussions that need more time can continue after this meeting with the relevant people.*

Then read out each person's next action and deadline.

## Brief standup record

Save a short note such as:

```
# Sprint 1 Standup

**Date:** 6 August 2026
**Facilitator:** Shayna Unterslak
**Attendees:**
-
-
-

## Progress

- Repository governance and methodology work completed.
- Local CI passes using `npm ci` and `npm run check`.
- Sprint 1 foundation issues assigned and active.
- Frontend and backend Azure deployment work separated into individual issues.

## Blockers

- Gitea Actions cannot run because no runner is currently available.
- Lecturer has confirmed that runners are being arranged and may take one to two weeks.

## Decisions

- CI changes may be merged after peer review, while the related issue remains open and blocked.
- Peer review will be recorded through an explicit teammate approval comment until formal Gitea approvals are resolved.
- Frontend and backend Azure deployment will use separate issues, branches and Pull Requests.

## Actions

| Team member | Issue | Next action | Due |
| --- | --- | --- | --- |
| Shayna | CI | Maintain blocker record and follow up on runner | Next standup |
| | | | |
| | | | |

## Issues requiring separate discussion

-
```

Save it somewhere consistent, for example:

```
evidence/sprints/sprint-1/standup-2026-08-06.md
```

Your methodology requires a brief Thursday standup summary as evidence, including progress, blockers, decisions, responsibility changes and next actions. `Pasted markdown.md` The active board, blocked issues and standup summaries also

help demonstrate that the methodology is actually being followed, rather than merely documented. Pasted markdown.md

8/6/2026, 9:04:18 AM



what sport is best for us to use

8/6/2026, 9:10:49 AM



## Thinking

- **Choosing a suitable sport**

I'm comparing cricket, baseball, tennis, basketball, and soccer by event detail, statistical depth, dataset availability, and integration potential. Cricket seems especially strong because ball-by-ball data naturally supports rich derived metrics and manageable scope.

- **Chose a suitable sport**

- **Selected the best sport for the analytics tool**



### Searching official Cricsheet ball-by-ball JSON data cricket open data

[cricsheet.org](https://cricsheet.org) [github.com](https://github.com) [retrosheet.org](https://retrosheet.org) [reddit.com](https://reddit.com) [cran.r-project.org](https://cran.r-project.org) [kaggle.com](https://kaggle.com) [apify.com](https://apify.com) [medium.com](https://medium.com)  
[en.wikipedia.org](https://en.wikipedia.org) [x.com](https://x.com) [pkg.robjhyndman.com](https://pkg.robjhyndman.com) [observablehq.com](https://observablehq.com) [mlb.com](https://mlb.com) [oreilly.com](https://oreilly.com) [thedata.school.co.uk](https://thedata.school.co.uk)  
[baseball-reference.com](https://baseball-reference.com) [arxiv.org](https://arxiv.org)

- **Selecting T20 cricket**

I'm recommending T20 cricket, ideally a South African professional league, because ball-by-ball events support rich derived statistics, corrections, scaling, and a clear event-driven architecture. I'd scope the MVP tightly around match, player, team, bowling, and batting metrics.

- **Selected T20 cricket**

- **Searched 17 websites**

✓ Worked for 47s

## The best sport for your project is T20 cricket, ideally the SA20 league.

It fits the brief extremely well because the platform must store ordered event data and calculate statistics from those events rather than manually entered totals. [sport\\_analytics.pdf](#)

## Why T20 cricket works best

- **Every ball is a clear event.** You can record the batter, bowler, runs, extras, wicket and dismissal type.
- **Statistics are easy to derive:** batting runs, strike rate, boundaries, bowling economy, wickets, dot-ball percentage, run rate and partnerships.
- **Corrections are straightforward.** Fixing one delivery can automatically recalculate every affected statistic.
- **The scope is manageable.** A T20 match normally contains about 240 legal deliveries, giving you meaningful event volume without the complexity of multi-day cricket.
- **It is locally relevant.** SA20 gives the project a clear South African professional-league identity.
- **Real data is available.** Cricsheet currently provides structured JSON ball-by-ball data for 130 SA20 matches and reports complete SA20 coverage. ([cricsheet.org](https://cricsheet.org))

## Why I would not choose the others

Football is attractive, but defining and validating passes, pressure, possession and positional events becomes much more complicated, while free professional event data only covers selected competitions. ([GitHub](#)) Rugby data is harder to obtain in a consistent open event format. Tennis would be manageable, but it gives you fewer team and multi-player analytics opportunities.



## Recommended project scope

Use:

***An event-driven analytics platform for the SA20 professional T20 cricket league.***

Start with these delivery events:

```
legal delivery
runs scored
boundary
wide
no-ball
bye or leg bye
wicket
dismissal type
```

Then derive:

```
batting runs and strike rate
fours and sixes
bowling wickets and economy
dot-ball percentage
team score and run rate
powerplay statistics
player and team season totals
```

Do not build for Test, ODI and T20 cricket together. Choosing **SA20 only** will keep the schema, rules and statistics focused enough to complete properly.

At today's standup, propose SA20 T20 cricket as the team's preferred option, then take it to Tuesday's stakeholder meeting for confirmation before finalising the event schema.

### Sources

[Match data - Cricsheet](https://cricsheet.org) cricsheet.org

[hudl/open-data: Free football data from StatsBomb](https://github.com/hudl/open-data) github.com

8/6/2026, 9:11:09 AM





can clouflare link to gitea? and how?

8/6/2026, 9:15:08 AM



## Thinking



### Searching the web

[developers.cloudflare.com](https://developers.cloudflare.com) [merriam-webster.com](https://www.merriam-webster.com) [apple.com](https://apple.com) [git-scm.com](https://git-scm.com) [wrangler.com](https://wrangler.com) [dictionary.cambridge.org](https://dictionary.cambridge.org)  
[instagram.com](https://www.instagram.com) [docs.gitea.com](https://docs.gitea.com) [vocabulary.com](https://www.vocabulary.com) [nextdirect.com](https://nextdirect.com) [youtube.com](https://www.youtube.com) [en.wikipedia.org](https://en.wikipedia.org) [grammaticus.blog](https://grammaticus.blog)  
[eu.wrangler.com](https://eu.wrangler.com) [english.stackexchange.com](https://english.stackexchange.com) [reddit.com](https://www.reddit.com) [thesaurus.com](https://www.thesaurus.com) [en.wiktionary.org](https://en.wiktionary.org) [yoox.com](https://yoox.com) [wordwebonline.com](https://wordwebonline.com)

- **Preparing deployment guidance**

I'm outlining a practical two-stage path: deploy the frontend manually first, then automate future deployments through repository secrets and a workflow. I'll keep backend configuration separate until its public URL is available.

- **Prepared deployment guidance**

- **Searched 20 websites**

✓ Worked for 37s

**Not directly.** Cloudflare Pages' built-in Git connection currently supports **GitHub and GitLab**, not a self-hosted Gitea repository. But your Gitea repository can still deploy to Cloudflare Pages using **Direct Upload + Wrangler**. ([Cloudflare Docs](#))

## How it would work for your project

```
Gitea push to main
    ↓
Gitea Actions runner
    ↓
npm ci
npm run check
build Vite frontend
    ↓
Wrangler uploads apps/frontend/dist
    ↓
Cloudflare Pages
```

Because your Gitea runner is not available yet, you can deploy manually now and automate it once the lecturer provides the runner.

## Option 1: deploy manually now

In Cloudflare:

1. Open **Workers & Pages**.
2. Create a **Pages** application.
3. Choose **Direct Upload**, not "Connect to Git".
4. Give it a project name such as:

sport-analytics-tool

From the repository root in PowerShell:

```
npm.cmd ci
npm.cmd run build --workspace=@sport-analytics/frontend
npx.cmd wrangler login
npx.cmd wrangler pages deploy apps/frontend/dist --project-name=sport-analytics-tool
```

Your Vite frontend produces its static site in `apps/frontend/dist`, and Wrangler uploads that folder to Cloudflare Pages. The deployed site will receive a `pages.dev` address. ([Cloudflare Docs](#))

When the Azure backend exists, build with its URL:

```
$env:VITE_API_BASE_URL="https://your-backend-url/api/v1"
npm.cmd run build --workspace=@sport-analytics/frontend
npx.cmd wrangler pages deploy apps/frontend/dist --project-name=sport-analytics-tool
```

`VITE_API_BASE_URL` is public frontend configuration, not a secret.

## Option 2: automate deployment from Gitea later

Once the Gitea runner is available:

1. Create a Cloudflare API token with **Cloudflare Pages: Edit** permission.
2. Add these under **Gitea → Repository Settings → Actions → Secrets**:

```
CLOUDFLARE_API_TOKEN
CLOUDFLARE_ACCOUNT_ID
```

Gitea repository secrets are designed for sensitive Actions values, and Cloudflare recommends those two credentials for CI deployment. ([Gitea Documentation](#))

Then create:

```
.gitea/workflows/deploy-frontend.yml
```

with:

```
name: Deploy Frontend

on:
  push:
    branches:
      - main

jobs:
  deploy:
    runs-on: ubuntu-latest
    timeout-minutes: 15

    steps:
      - name: Check out repository
        uses: actions/checkout@v4

      - name: Set up Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20

      - name: Install dependencies
        run: npm ci
```

```
- name: Run repository checks
  run: npm run check

- name: Deploy frontend to Cloudflare Pages
  run: npx wrangler pages deploy apps/frontend/dist --project-name=sport-analytics-tool
  env:
    CLOUDFLARE_API_TOKEN: ${ secrets.CLOUDFLARE_API_TOKEN }
    CLOUDFLARE_ACCOUNT_ID: ${ secrets.CLOUDFLARE_ACCOUNT_ID }
```

Cloudflare officially supports deploying prebuilt assets through `wrangler pages deploy`, which makes this suitable for CI systems other than its direct Git integrations. ([Cloudflare Docs](#))

So the answer for your team is:

***Cloudflare cannot connect directly to Gitea, but Gitea Actions can build the frontend and deploy it to Cloudflare Pages through Wrangler. Until the runner is available, deploy manually using Direct Upload.***

## Sources

[Git integration · Cloudflare Pages docs](#) [developers.cloudflare.com](#)

[Secrets](#) [docs.gitea.com](#)

8/6/2026, 9:15:10 AM



can you make an issue for me for doing that and connecting to wrangler etc

8/6/2026, 9:46:10 AM



## Thinking

- 🌐 **Searching the web**
    - [docs.gitea.com](https://docs.gitea.com)
    - [developers.cloudflare.com](https://developers.cloudflare.com)
  - **Clarifying Cloudflare scope**

I'm framing this as a Cloudflare Pages deployment for the public MkDocs documentation site, connected to Gitea through Wrangler. I'm also flagging that it should not duplicate the separate Azure frontend-hosting issue.
  - **Clarified Cloudflare scope**
  - **Searched 2 websites**
- ✓ Worked for 22s

Yes. Because you already have a separate **Azure frontend deployment issue**, this Cloudflare issue should cover the **public MkDocs documentation site**, not the application frontend.

Cloudflare Pages can receive prebuilt static files through Direct Upload using Wrangler. Cloudflare's automatic Git integration currently supports GitHub and GitLab rather than Gitea, so deployment from Gitea must use Wrangler manually at first and later through Gitea Actions once a runner is available. ([Cloudflare Docs](#))

## Issue title

Deploy MkDocs documentation to Cloudflare Pages using Wrangler

## Labels

- **Type:** type: infrastructure
- **Area:** area: documentation
- **Priority:** priority: medium

## Milestone

Sprint 1

## Description

### ## Description

Deploy the project's MkDocs documentation site to Cloudflare Pages using a Direct Upload workflow through Wrangler.

Cloudflare Pages does not connect directly to the project's Gitea repository. The documentation must therefore be built locally and uploaded using Wrangler. Once a Gitea Actions runner becomes available, the same process should be automated from the repository.

The deployment must use the existing `mkdocs.yml` configuration and documentation stored under `docs/`.

### ## Motivation

The project requires a public documentation website that remains aligned with the repository.

Publishing the MkDocs site early will give the team and stakeholder a stable location for viewing the project methodology, architecture, setup instructions, testing strategy and other project documentation.

Using Wrangler also provides a deployment path that is compatible with the team's Gitea repository even though Cloudflare Pages does not provide direct Gitea integration.

### ## Acceptance Criteria

- [ ] A Cloudflare Pages project has been created using the Direct Upload option.
- [ ] The MkDocs documentation builds successfully from the repository.
- [ ] The generated static site is deployed using Wrangler.
- [ ] The documentation is available through a public HTTPS `pages.dev` URL.
- [ ] The deployed site loads successfully and its navigation works.
- [ ] The deployed content matches the documentation currently stored in the repository.
- [ ] No Cloudflare API token, account identifier or other sensitive credential is committed to the repository.
- [ ] The Cloudflare project name and public documentation URL are recorded in the repository documentation.
- [ ] Clear manual build and deployment instructions are documented.
- [ ] A Gitea Actions workflow for automated deployment is prepared or documented for implementation once a runner is available.
- [ ] Required Cloudflare secrets are identified for later storage in Gitea Actions Secrets.
- [ ] `npm run check` and the MkDocs build both pass before the Pull Request is opened.
- [ ] The work is completed through a reviewed Pull Request linked to this issue.

### ## Implementation Guidance

The initial manual deployment should follow this process:

```
```bash
python -m pip install -r requirements-docs.txt
mkdocs build
npx wrangler login
npx wrangler pages deploy site --project-name=<cloudflare-project-name>
```

The `site/` directory is generated by MkDocs and should be uploaded to Cloudflare Pages. It should not be treated as the editable documentation source.

Once a Gitea Actions runner is available, automation should:

1. check out the repository;
2. install the documentation dependencies;
3. build the MkDocs site;
4. deploy the generated `site/` directory using Wrangler.

## Required Secrets for Future Automation

The automated workflow is expected to use repository Actions secrets such as:

- `CLOUDFLARE_API_TOKEN`
- `CLOUDFLARE_ACCOUNT_ID`

The API token must have only the permissions required to deploy the Cloudflare Pages project.

## Dependencies and Blockers

- Manual deployment can be completed before a Gitea Actions runner is available.
- Automated deployment from Gitea is blocked until the lecturer provides a runner.
- This issue is separate from the Azure frontend and backend deployment issues.

- The documentation deployment must not contain application or database secrets.

## Evidence

Link or attach:

- the Pull Request implementing the deployment;
- the public Cloudflare Pages documentation URL;
- a screenshot of the deployed documentation site;
- terminal output showing a successful MkDocs build;
- terminal output showing a successful Wrangler deployment;
- the updated deployment documentation; and
- the prepared Gitea Actions workflow or documented automation plan.

## Out of Scope

- hosting the Vite application frontend;
- deploying the Express backend;
- provisioning the PostgreSQL database;
- configuring a custom domain;
- advanced Cloudflare analytics or access controls.

```
## Suggested branch
```

```
```text
infra/<issue-number>-cloudflare-docs-deployment
```

## Suggested comment for the runner blocker

Manual Cloudflare Pages deployment through Wrangler can proceed. Automated deployment from Gitea will remain blocked until an approved Gitea Actions runner becomes available.

Wrangler's documented deployment command accepts a generated directory and Pages project name, while Cloudflare recommends storing API credentials as CI secrets rather than committing them. ([Cloudflare Docs](#))

### Sources

[Direct Upload · Cloudflare Pages docs](#) developers.cloudflare.com

8/6/2026, 9:46:11 AM



give me the issue to copy paste, make sure ur markdown works well make sure to add the AI decleration, etc.

8/6/2026, 9:49:30 AM



### Thinking

- **Structuring deployment issue**

I'm drafting a self-contained issue with milestones, acceptance criteria, Windows-friendly commands, manual Cloudflare deployment, and an optional Gitea workflow. I'm keeping automated deployment clearly marked as prepared but dependent on runner availability.

- **Structured the deployment issue**

✓ Worked for 24s

## Issue title

Deploy MkDocs documentation to Cloudflare Pages using Wrangler

## Labels

- type: infrastructure
- area: documentation
- priority: medium

## Milestone

Sprint 1

## Issue description

Copy and paste everything inside this box:

### ## Description

Deploy the Sport Analytics Tool's MkDocs documentation website to Cloudflare Pages using Wrangler.

The documentation source is stored in `docs/` and configured through `mkdocs.yml`. Because Cloudflare Pages does not connect directly to the team's Gitea repository, the site should first be deployed using Cloudflare Pages Direct Upload and Wrangler.

Once a Gitea Actions runner becomes available, the same build and deployment process should be automated from the repository.

### ## Motivation

A public documentation website will provide the team, lecturer and stakeholder with a stable location for viewing:

- the Git and Project Methodologies;
- architecture and repository structure;
- development setup instructions;
- testing and deployment documentation;
- database and API documentation; and
- project evidence and decisions where appropriate.

Deploying the documentation early also ensures that documentation remains part of the team's normal development and review process.

### ## Scope

This issue includes:

- creating a Cloudflare Pages project;
- installing and configuring Wrangler where required;
- building the existing MkDocs site;
- deploying the generated `site/` directory;
- recording the public documentation URL;
- documenting the manual deployment process;
- identifying the credentials required for future automated deployment; and
- preparing or documenting the Gitea Actions deployment process for use once a runner is available.

### ## Acceptance Criteria

- [ ] A Cloudflare Pages project has been created using the Direct Upload option.
- [ ] The existing MkDocs documentation builds successfully.
- [ ] The documentation build completes without broken navigation or configuration errors.
- [ ] The generated `site/` directory is deployed using Wrangler.
- [ ] The documentation is available through a public HTTPS `pages.dev` URL.
- [ ] The deployed home page and navigation links load correctly.
- [ ] The deployed content reflects the documentation currently committed to the repository.
- [ ] The Cloudflare Pages project name and public URL are recorded in the repository documentation.
- [ ] Clear manual build and deployment instructions are added to the repository.
- [ ] The future Gitea Actions deployment process is either prepared or clearly documented.
- [ ] Required Cloudflare secrets for future automation are identified.
- [ ] No API token, account credential or other secret is committed to the repository.
- [ ] The generated `site/` directory is not committed unless this is specifically required and documented.
- [ ] `npm run check` passes before the Pull Request is opened.
- [ ] The MkDocs production build passes before the Pull Request is opened.
- [ ] The work is completed through a reviewed Pull Request linked to this issue.
- [ ] AI usage is declared according to the repository's AI policy.

### ## Suggested Manual Deployment Process

From the repository root:

```
```bash
python -m pip install -r requirements-docs.txt
python -m mkdocs build --strict
npx wrangler login
npx wrangler pages deploy site --project-name=<cloudflare-project-name>
```
```

On Windows PowerShell, `npx.cmd` may be used instead of `npx` if PowerShell blocks the npm script wrapper.

Example:

```
```powershell
npx.cmd wrangler pages deploy site --project-name=<cloudflare-project-name>
```
```

### ## Future Automated Deployment

Once a Gitea Actions runner is available, the deployment workflow should:

1. check out the repository;
2. install the documentation dependencies;
3. build the MkDocs site using strict mode;
4. deploy the generated `site/` directory through Wrangler; and
5. run only after the required repository checks pass.

The future workflow may be stored at:

```
```text
.gitea/workflows/deploy-docs.yml
```
```

### ## Required Secrets for Future Automation

The following values should be stored under Gitea repository Actions secrets and must not be committed:

```
```text
CLOUDFLARE_API_TOKEN
CLOUDFLARE_ACCOUNT_ID
```
```

The Cloudflare API token should be limited to the permissions required to deploy the relevant Pages project.

### ## Dependencies and Blockers

- Manual deployment can proceed without a Gitea Actions runner.
- Automated deployment from Gitea is blocked until a runner is provided.
- The current runner blocker is tracked separately under Issue #10.
- This issue is separate from the Azure frontend and backend deployment work.
- The documentation deployment must not contain application, database or infrastructure secrets.

### ## Evidence Required

The Pull Request or issue must include:

- the public Cloudflare Pages URL;
- a screenshot of the deployed documentation site;
- terminal output showing a successful MkDocs build;
- terminal output showing a successful Wrangler deployment;
- the Cloudflare Pages project name;
- the updated deployment documentation; and
- evidence that no credentials were committed.

### ## AI Usage and Attribution

This issue description was drafted with assistance from:

```
```text
ChatGPT-Web[GPT-5.6 Thinking]
```
```

This work forms part of the existing continuous project transcript retained by the team.

When completing this issue:

- update `evidence/ai/ai-usage-register.csv` with the actual AI assistance used;
- distinguish between planning, documentation, code generation, configuration assistance and code review;
- include the exact tool and model used;
- retain or link the relevant unedited transcript; and
- add the following footer to commits containing AI-assisted configuration, workflow code or documentation:

```
```text
```

```
Assisted-by: ChatGPT-Web[GPT-5.6 Thinking]
```

```
```
```

The assignee must review, test and adapt all AI-assisted output before it is committed.

### ## Out of Scope

- deploying the Vite application frontend;
- deploying the Express backend;
- provisioning the PostgreSQL database;
- configuring a custom domain;
- advanced Cloudflare analytics;
- Cloudflare Access configuration; and
- storing secrets directly in repository files.

## Suggested branch

```
infra/<issue-number>-cloudflare-docs-deployment
```

8/6/2026, 9:49:51 AM



now make an issue to create folders for AI transcripts for each person

8/6/2026, 9:53:39 AM



### Thinking

- **Drafting repository issue**

I'm shaping a documentation issue for organizing AI transcripts while preserving context. I'm including explicit, explained redaction for secrets or personal data, AI-use attribution, suggested labels, sprint placement, and a consistent branch name.

- **Drafted repository issue**

✓ Worked for 23s

### Issue title

Create per-team-member folders for AI transcripts

### Labels

- type: documentation
- area: documentation
- priority: medium

### Milestone

Sprint 1

### Issue description

Copy and paste everything inside this box:

#### ## Description

Create a consistent repository structure for storing AI transcripts used by each team member during the project.

Each team member must have their own folder under the project's AI evidence directory. The structure should make it easy to connect an AI usage register entry to the relevant transcript, issue, Pull Request and commit.

The transcript folders are evidence of how AI tools were used during planning, documentation, coding, configuration, debugging and review.

#### ## Proposed Structure

Create the following structure:

```
```text
evidence/
├─ ai/
│   ├── README.md
│   ├── ai-usage-register.csv
│   └── transcripts/
│       ├── README.md
│       ├── <team-member-name>/
│       │   └── README.md
```

```
├── <team-member-name>/
│   └── README.md
└── <team-member-name>/
    └── README.md
...

```

Use lowercase kebab-case for team member folder names.

Example:

```
```text
evidence/ai/transcripts/shayna-unterslak/
evidence/ai/transcripts/dean-feldman/
```

```

Create a folder for every current team member.

Each folder should contain a `README.md` so that the directory remains in Git even before transcripts are added.

### ## Transcript Naming Convention

Use the following naming format:

```
```text
YYYY-MM-DD-issue-<number>-<short-description>.<extension>
```

```

Examples:

```
```text
2026-08-05-issue-7-repository-governance.md
2026-08-06-issue-10-ci-debugging.md
2026-08-06-issue-14-cloudflare-deployment.pdf
```

```

Where one continuous transcript covers several issues, the same transcript may be stored once and referenced by several rows in the AI usage register.

Example:

```
```text
2026-08-04-to-2026-08-06-continuous-project-session.pdf
```

```

The register entries should identify which portion of the transcript relates to each issue.

### ## Root AI README Requirements

Update `evidence/ai/README.md` or create it if it does not exist.

It should explain:

- the purpose of the AI evidence directory;
- where the AI usage register is stored;
- where individual transcripts are stored;
- the transcript naming convention;
- how continuous transcripts covering multiple tasks should be referenced;
- how transcripts should be linked to issues and Pull Requests;
- how AI-assisted commits must be attributed;
- that all AI-assisted output must be reviewed and verified by a team member; and

- that secrets and sensitive personal information must not be committed.

### ## Transcript Folder README Template

Each team member's folder should contain a `README.md` using this structure:

```markdown

# AI transcripts: <Team Member Name>

This folder contains AI transcripts retained as evidence for work completed by <Team Member Name>.

#### ## Naming convention

`YYYY-MM-DD-issue-<number>-<short-description>.<extension>`

#### ## Register

Each transcript stored here must have a corresponding entry in:

`evidence/ai/ai-usage-register.csv`

#### ## Requirements

- Record the exact AI tool and model used.
- Link the transcript to the relevant issue or Pull Request.
- State how the output was reviewed, tested or adapted.
- Do not commit passwords, API tokens, private keys or other credentials.
- Clearly mark and explain any necessary redactions.

```

### ## Acceptance Criteria

- [ ] An `evidence/ai/transcripts/` directory has been created.
- [ ] A separate folder exists for every current team member.
- [ ] Team member folder names use lowercase kebab-case.
- [ ] Every team member folder contains a `README.md`.
- [ ] A clear transcript naming convention has been documented.
- [ ] Continuous transcripts covering several issues can be referenced without unnecessary duplication.
- [ ] Existing retained transcripts are placed in the correct team member folder where available.
- [ ] Existing AI usage register entries are updated with accurate transcript paths where possible.
- [ ] The root AI evidence README explains the folder structure and required process.
- [ ] Transcript files can be linked clearly to issues, Pull Requests and commits.
- [ ] No passwords, tokens, private keys or other secrets are committed.
- [ ] Any necessary redactions are clearly identified and explained.
- [ ] `npm run check` passes before the Pull Request is opened.
- [ ] The work is completed through a reviewed Pull Request linked to this issue.
- [ ] AI assistance used to complete this issue is declared in the AI usage register and relevant commits.

### ## Privacy and Security

Transcripts must be checked before they are committed.

Do not commit:

- passwords;
- API tokens;
- authentication cookies;
- private keys;
- database credentials;
- private repository access links containing credentials;

- confidential stakeholder information; or
- unnecessary personal information.

Transcripts should remain unedited evidence where practical. If information must be removed for privacy or security, the redaction must be visibly marked and briefly explained.

### ## Evidence Required

The Pull Request should include:

- the completed folder structure;
- the root AI evidence README;
- the README template used in each team member folder;
- at least one example transcript path;
- updated register references where applicable; and
- confirmation that the repository contains no exposed credentials.

### ## AI Usage and Attribution

This issue description was drafted with assistance from:

```
```text
ChatGPT-Web[GPT-5.6 Thinking]
```
```

When completing this issue:

- record the actual AI assistance used in `evidence/ai/ai-usage-register.csv`;
- retain or reference the relevant transcript;
- review and verify all AI-assisted output before committing it; and
- add the following footer to commits containing AI-assisted documentation or configuration:

```
```text
Assisted-by: ChatGPT-Web[GPT-5.6 Thinking]
```
```

### ## Out of Scope

- changing the project's general Git Methodology;
- changing the AI policy requirements;
- adding automated transcript-export tooling;
- storing AI account credentials;
- uploading transcripts to an external storage provider; and
- reviewing the technical accuracy of every historical AI response.

## Suggested branch

docs/<issue-number>-organise-ai-transcripts

8/6/2026, 9:54:04 AM